

C++ Training @ R&S

2. C++ Software Design

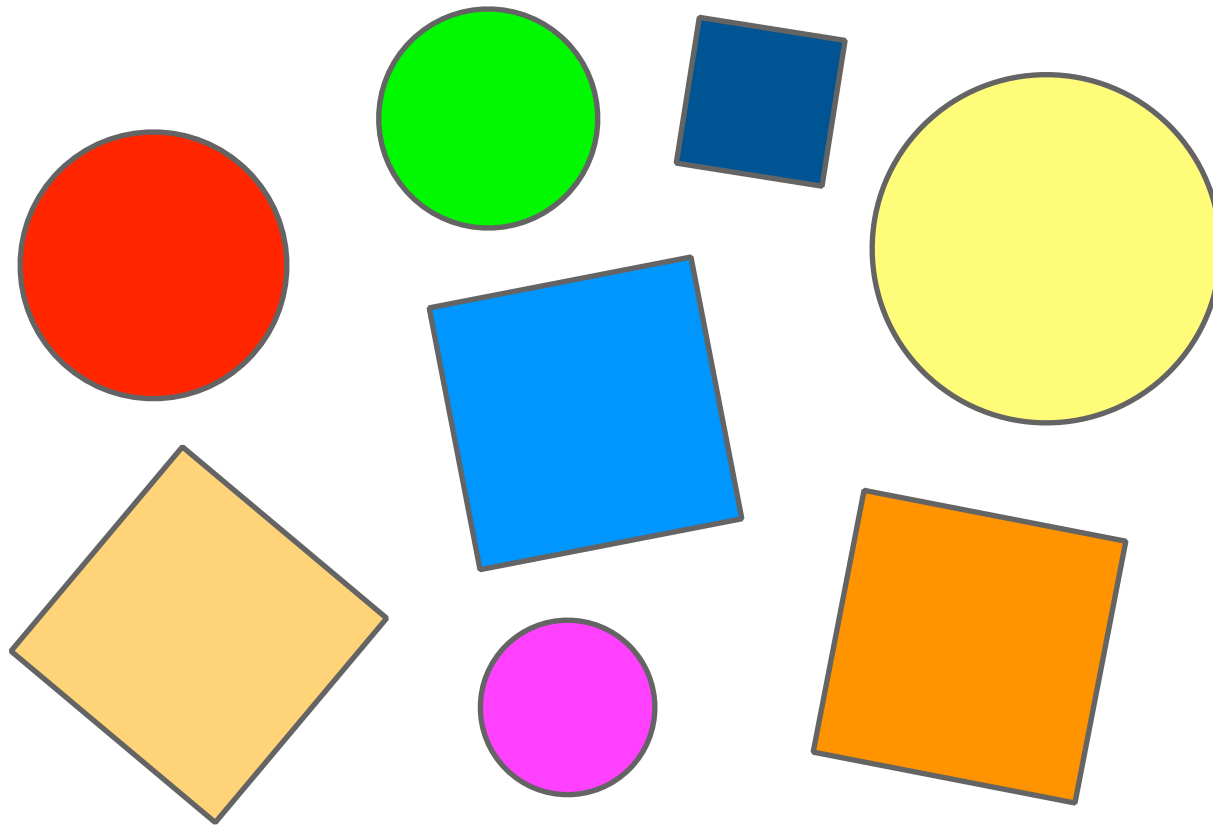
Klaus Iglberger
27.4.-29.4.2026

Content

1. Motivation
2. Software Architecture vs. Design
3. Visitor
4. Interlude 1: The Expression Problem
5. Strategy
6. Interlude 2: Value Semantics
7. External Polymorphism
8. Type Erasure
9. Prototype
10. Bridge
11. Decorator
12. CRTP
13. Expression Templates

2.1. Motivation

Our Toy Problem: Drawing Shapes



An Example

Task (C_Cpp_Software_Design/Motivation/Procedural): Evaluate the given design with respect to changeability and extensibility.

A Procedural Solution

```
enum ShapeType
{
    circle,
    square
};

class Shape
{
public:
    explicit Shape( ShapeType t )
        : type{ t }
    {}

    virtual ~Shape() = default;
    ShapeType getType() const noexcept;

private:
    ShapeType type;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad )
        : Shape{ circle }
        , radius{ rad }
        , // ... Remaining data members
    {}

    double getRadius() const noexcept;
```

A Procedural Solution

```
enum ShapeType
{
    circle,
    square
};

class Shape
{
public:
    explicit Shape( ShapeType t )
        : type{ t }
    {}

    virtual ~Shape() = default;
    ShapeType getType() const noexcept;

private:
    ShapeType type;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad )
        : Shape{ circle }
        , radius{ rad }
        , // ... Remaining data members
    {}

    double getRadius() const noexcept;
```

A Procedural Solution

```
enum ShapeType
{
    circle,
    square
};

class Shape
{
public:
    explicit Shape( ShapeType t )
        , type{ t }
    {}

    virtual ~Shape() = default;
    ShapeType getType() const noexcept;

private:
    ShapeType type;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad )
        : Shape{ circle }
        , radius{ rad }
        , // ... Remaining data members
    {}

    double getRadius() const noexcept;
```

A Procedural Solution

```
};

class Circle : public Shape
{
public:
    explicit Circle( double rad )
        : Shape{ circle }
        , radius{ rad }
        , // ... Remaining data members
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...
    // ...

private:
    double radius;
    // ... Remaining data members
};

void translate( Circle&, Vector2D const& );
void rotate( Circle&, double const& );
void draw( Circle const& );

class Square : public Shape
{
public:
    explicit Square( double s )
        : Shape{ square }
        , side{ s }
```

A Procedural Solution

```
void draw( Circle const& );

class Square : public Shape
{
public:
    explicit Square( double s )
        : Shape{ square }
        , side{ s }
        , // ... Remaining data members
    {}

    double getSide() const noexcept;
    // ... getCenter(), getRotation(), ...
    // ...

private:
    double side;
    // ... Remaining data members
};

void translate( Square&, Vector2D const& );
void rotate( Square&, double const& );
void draw( Square const& );

void drawAllShapes( std::vector<std::unique_ptr<Shape>> const& shapes )
{
    for( auto const& s : shapes )
    {
        switch ( s->getType() )
        {
```

A Procedural Solution

```
};

void translate( Square&, Vector2D const& );
void rotate( Square&, double const& );
void draw( Square const& );

void drawAllShapes( std::vector<std::unique_ptr<Shape>> const& shapes )
{
    for( auto const& s : shapes )
    {
        switch ( s->getType() )
        {
            case circle:
                draw( *static_cast<Circle const*>( s.get() ) );
                break;
            case square:
                draw( *static_cast<Square const*>( s.get() ) );
                break;
        }
    }
}

int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( std::make_unique<Circle>( 2.0 ) );
    shapes.emplace_back( std::make_unique<Square>( 1.5 ) );
}
```

A Procedural Solution

```
        case square:
            draw( *static_cast<Square const*>( s.get() ) );
            break;
        }
    }
}

int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( std::make_unique<Circle>( 2.0 ) );
    shapes.emplace_back( std::make_unique<Square>( 1.5 ) );
    shapes.emplace_back( std::make_unique<Circle>( 4.2 ) );

    // Drawing all shapes
    drawAllShapes( shapes );
}
```


A Procedural Solution

```
enum ShapeType
{
    circle,
    square,
    rectangle
};

class Shape
{
public:
    explicit Shape( ShapeType t )
        : type{ t }
    {}

    virtual ~Shape() = default;
    ShapeType getType() const noexcept;

private:
    ShapeType type;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad )
        : Shape{ circle }
        , radius{ rad }
        , // ... Remaining data members
    {}
}
```

A Procedural Solution

```
};

class Circle : public Shape
{
public:
    explicit Circle( double rad )
        : Shape{ circle }
        , radius{ rad }
        , // ... Remaining data members
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...
    // ...

private:
    double radius;
    // ... Remaining data members
};

void translate( Circle&, Vector2D const& );
void rotate( Circle&, double const& );
void draw( Circle const& );

class Square : public Shape
{
public:
    explicit Square( double s )
        : Shape{ square }
        , side{ s }
```

A Procedural Solution

```
void draw( Circle const& );

class Square : public Shape
{
public:
    explicit Square( double s )
        : Shape{ square }
        , side{ s }
        , // ... Remaining data members
    {}

    double getSide() const noexcept;
    // ... getCenter(), getRotation(), ...
    // ...

private:
    double side;
    // ... Remaining data members
};

void translate( Square&, Vector2D const& );
void rotate( Square&, double const& );
void draw( Square const& );

void drawAllShapes( std::vector<std::unique_ptr<Shape>> const& shapes )
{
    for( auto const& s : shapes )
    {
        switch ( s->getType() )
        {
```

A Procedural Solution

```
void draw( Square const& );

void drawAllShapes( std::vector<std::unique_ptr<Shape>> const& shapes )
{
    for( auto const& s : shapes )
    {
        switch ( s->getType() )
        {
            case circle:
                draw( *static_cast<Circle const*>( s.get() ) );
                break;
            case square:
                draw( *static_cast<Square const*>( s.get() ) );
                break;
            case rectangle:
                draw( *static_cast<Rectangle const*>( s.get() ) );
                break;
        }
    }
}

int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( std::make_unique<Circle>( 2.0 ) );
    shapes.emplace_back( std::make_unique<Square>( 1.5 ) );
    shapes.emplace_back( std::make_unique<Circle>( 4.2 ) );
}
```

The Expert's Advice



"This kind of type-based programming has a long history in C, and one of the things we know about it is that it yields programs that are essentially unmaintainable."

(Scott Meyers, More Effective C++, Item 31)

The Problem

There is one constant in **software** development and that is ...

Change

The Problem

The truth in our industry:

**Software must be
adaptable to frequent
changes**

The Problem

The truth in our industry:

Software must be
adaptable to frequent
changes

The Expert's Opinion



"Since change is the dominant cost of software development and understanding code is the dominant cost of change, communicating the structure and intent of working code is one of the most valuable skills you can exercise."

(Kent Beck, Tidy First?)

The Problem

What is the core problem of adaptable software and software development in general?

Dependencies

The Problem

Dependencies ...

- ... complicate **changes/modifications**
- ... impede the **testability** of software
 - ... obstruct **modularity**
 - ... increase **build times**

The Expert's Opinion



*"Dependency is the key problem in software development at all scales."
(Kent Beck, TDD by Example)*

The Expert's Opinion



*"Coupling is a worse enemy for you than DRY.
Coupling is what kills all software."
(Ian Cooper, TDD, Where Did It All Go Wrong)*

The Expert's Opinion



"What "propagates" change? Coupling. So the cost of software is approximately equal to the coupling [...]:

$\text{cost}(\text{software}) \sim \text{cost}(\text{changes}) \sim \text{cost}(\text{big changes}) \sim \text{coupling}.$

Or, to highlight the importance of software design:

$\text{cost}(\text{software}) \sim \text{coupling}."$

(Kent Beck, Tidy First?)

Guidelines

Guideline: When designing software (modules, classes, functions, ...) try to minimize coupling between software components.

Overview

Single-Responsibility Principle (SRP)

Open-Closed Principle (OCP)

Liskov Substitution Principle (LSP)

Interface Segregation Principle (ISP)

Dependency Inversion Principle (DIP)

Overview

Open-Closed Principle (OCP)

Liskov Substitution Principle (LSP)

Dependency Inversion Principle (DIP)

Interface Segregation Principle (ISP)



Robert C. Martin

Overview

Single-Responsibility Principle (SRP)

Open-Closed Principle (OCP)

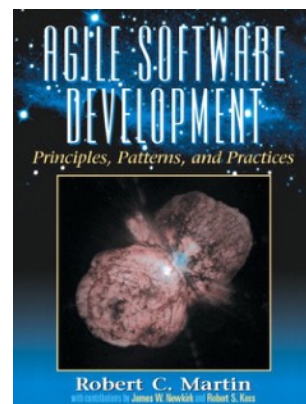
Liskov Substitution Principle (LSP)

Dependency Inversion Principle (DIP)

Interface Segregation Principle (ISP)



Robert C. Martin



2002

Overview

Single-Responsibility Principle (SRP)

Open-Closed Principle (OCP)

Liskov Substitution Principle (LSP)

Interface Segregation Principle (ISP)

Dependency Inversion Principle (DIP)



Robert C. Martin



Michael Feathers

An Example

Task (C_Cpp_Software_Design/Motivation/ObjectOriented): Evaluate the given design with respect to changeability and extensibility.

An Example

```
class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void translate( Vector2D const& ) = 0;
    virtual void rotate( double const& ) = 0;
    virtual void draw() const = 0;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

    void translate( Vector2D const& ) override;
    void rotate( double const& ) override;
    void draw() const override;

    // ...

private:
    double radius;
```

An Example

```
class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void translate( Vector2D const& ) = 0;
    virtual void rotate( double const& ) = 0;
    virtual void draw() const = 0;
};
```

```
class Circle : public Shape
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

    void translate( Vector2D const& ) override;
    void rotate( double const& ) override;
    void draw() const override;

    // ...
```

```
private:
    double radius;
```

An Example

```
    virtual void draw() const = 0;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
        {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

    void translate( Vector2D const& ) override;
    void rotate( double const& ) override;
    void draw() const override;

    // ...

private:
    double radius;
    // ... Remaining data members
};

class Square : public Shape
{
public:
    explicit Square( double s )
        : side{ s }
```

An Example

```
// ... Remaining data members
};

class Square : public Shape
{
public:
    explicit Square( double s )
        : side{ s }
        , // ... Remaining data members
    {}

    double getSide() const noexcept;
    // ... getCenter(), getRotation(), ...

    void translate( Vector2D const& ) override;
    void rotate( double const& ) override;
    void draw() const override;

    // ...

private:
    double side;
    // ... Remaining data members
};

void drawAllShapes( std::vector<std::unique_ptr<Shape>> const& shapes )
{
    for( auto const& s : shapes )
    {
        s->draw();
    }
}
```


An Example

```
void draw() const override;

// ...

private:
    double side;
    // ... Remaining data members
};

void drawAllShapes( std::vector<std::unique_ptr<Shape>> const& shapes )
{
    for( auto const& s : shapes )
    {
        s->draw();
    }
}

int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( std::make_unique<Circle>( 2.0 ) );
    shapes.emplace_back( std::make_unique<Square>( 1.5 ) );
    shapes.emplace_back( std::make_unique<Circle>( 4.2 ) );

    // Drawing all shapes
    drawAllShapes( shapes );
}
```

An Example

```
{  
    for( auto const& s : shapes )  
    {  
        s->draw();  
    }  
}  
  
int main()  
{  
    using Shapes = std::vector<std::unique_ptr<Shape>>;  
  
    // Creating some shapes  
    Shapes shapes;  
    shapes.emplace_back( std::make_unique<Circle>( 2.0 ) );  
    shapes.emplace_back( std::make_unique<Square>( 1.5 ) );  
    shapes.emplace_back( std::make_unique<Circle>( 4.2 ) );  
  
    // Drawing all shapes  
    drawAllShapes( shapes );  
}
```

An Example

```
class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void translate( Vector2D const& ) = 0;
    virtual void rotate( double const& ) = 0;
    virtual void draw() const = 0;
};
```

```
class Circle : public Shape
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

    void translate( Vector2D const& ) override;
    void rotate( double const& ) override;
    void draw() const override;

    // ...
```

```
private:
    double radius;
```

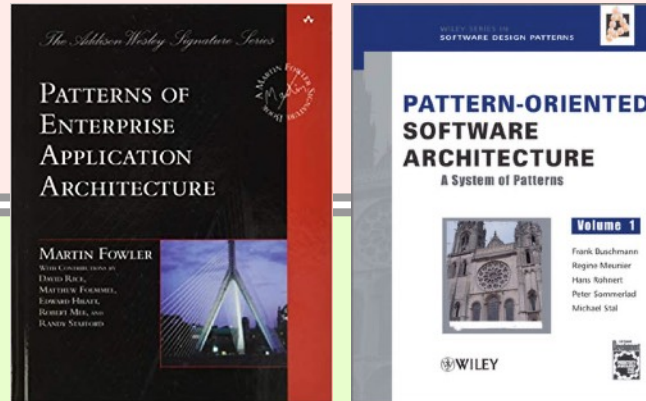
2.2. Software Architecture vs. Design

What is Software Design/Architecture?

2. C++ Software Design - Software Architecture vs. Design

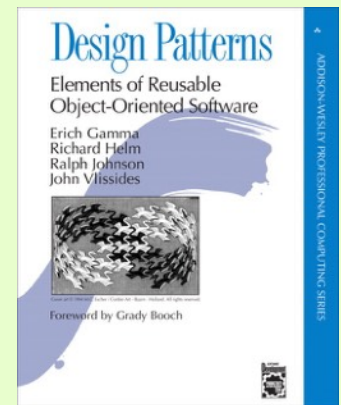
Software Architecture

- How are big entities depending on each other?
- Design decisions that are harder to change
- Architectural patterns
- Examples:
 - Client-Server Architecture
 - Microservices
 - MVC, ...



Software Design

- How are small entities depending on each other?
- Design decisions that are easier to change
- Design patterns
- Examples:
 - GoF Patterns: Visitor, Strategy, Observer, ...
 - External Polymorphism
 - ...



Idioms

- NVI Idiom (Template Method Design Pattern)
- Pimpl Idiom (Bridge Design Pattern)

Implementation Details

- How is a design implemented?
- Which features are used?
- Implementation patterns
- Examples:
 - new, malloc, ...
 - Exception Safety, Performance, ...
 - ...

- Temporary-Swap Idiom
- RAll Idiom
- enable_if
- Factory function

What is Software Design/Architecture?

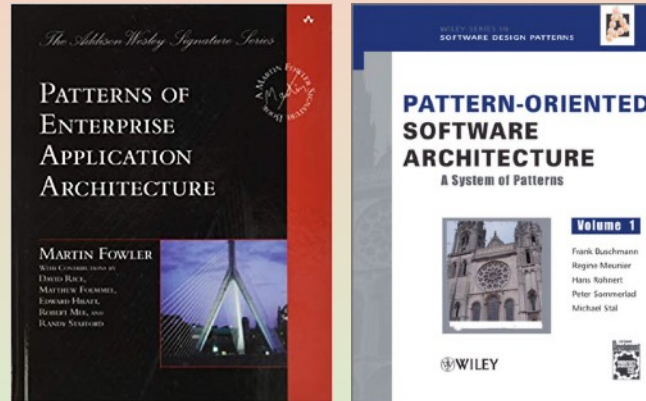


*“..., I’ll assert that there is no difference between [architecture and design]. None at all.”
(Robert C. Martin, Clean Architecture)*

2. C++ Software Design - Software Architecture vs. Design

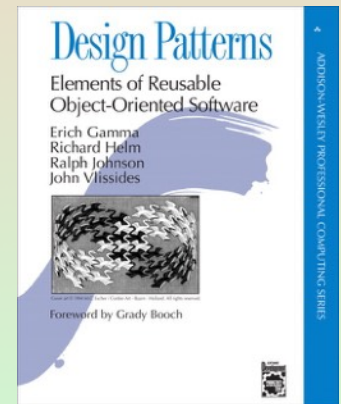
Software Architecture

- How are big entities depending on each other?
- Design decisions that are harder to change
- Architectural patterns
- Examples:
 - Client-Server Architecture
 - Microservices
 - MVC, ...



Software Design

- How are small entities depending on each other?
- Design decisions that are easier to change
- Design patterns
- Examples:
 - GoF Patterns: Visitor, Strategy, Observer, ...
 - External Polymorphism
 - ...



Idioms

- NVI Idiom (Template Method Design Pattern)
- Pimpl Idiom (Bridge Design Pattern)

Implementation Details

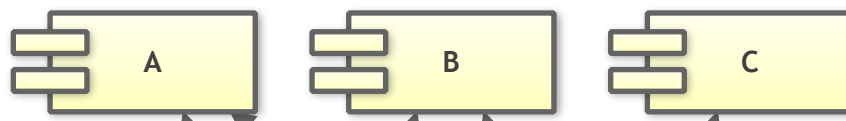
- How is a design implemented?
- Which features are used?
- Implementation patterns
- Examples:
 - new, malloc, ...
 - Exception Safety, Performance, ...
 - ...

- Temporary-Swap Idiom
- RAll Idiom
- enable_if
- Factory function

What is Software Design/Architecture?

High level

(stable, low dependencies)



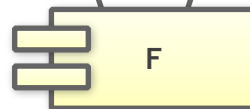
Architectural Boundary

This is an architecture: all arrows point upwards to the next higher level of abstraction. Everything builds on more stable components.



Architectural Boundary

Direct Dependency



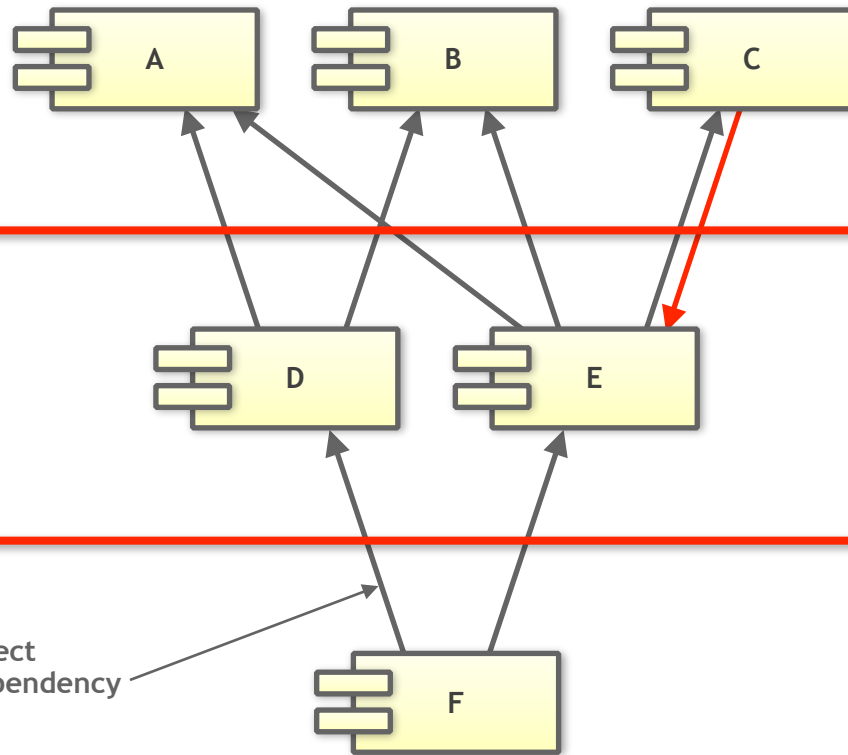
Low level

(volatile, malleable, high dependencies)

What is Software Design/Architecture?

High level

(stable, low dependencies)



This is NOT an architecture: if any arrow points downwards (i.e. if anything “stable” depends on something less stable) the architecture is broken.

Low level

(volatile, malleable, high dependencies)

What is Software Design/Architecture?



"The goal of software architecture is to minimise the human resources required to build and maintain the required system."

(Robert C. Martin, Clean Architecture)

What is Software Design/Architecture?

Software Design is the art of managing interdependencies between software components. It aims at minimizing (technical) **dependencies** and introduces the necessary **abstractions** and compromises.

(Klaus Iglberger)

What is Software Design/Architecture?

Software Design is the art of managing dependencies and abstractions.

The First Law of Software Architecture



"Everything in software architecture is a trade-off."

(Neal Ford, Mark Richards, Head First Software Architecture)



What is a Design Pattern?

What is a Design Pattern?



"A Design Pattern is a named canonical form for a combination of software structures and procedures that have proven to be useful over the years."

(Robert C. Martin, A Little About Patterns, The Clean Code Blog, June, 30th, 2014)

What is a Design Pattern?

A design pattern ...

- ... has a **name**;
- ... has an **intent**;
- ... aims at reducing **dependencies**;
- ... provides some sort of **abstraction**;
- ... has been **proven to work** over the years.

A design pattern is not ...

- ... limited to **object-oriented programming**;
- ... limited to **dynamic polymorphism**;
- ... a **language specific idiom**.

The Value of a Name



Me

“I would use a *Visitor* for that”.

“I don’t know. I thought of using a *Strategy*.”



You



Me

“Yes, you may have a point there. But since we’ll have to extend operations fairly often, we probably should consider a *Decorator* as well.”

The Value of a Name



Me

“I think we should create a system that allows us to extend the operations without the need to modify existing types again and again.”

“I don’t know. Rather than new operations. I would expect new types to be added frequently. So I prefer a solution that allows me to add types easily. But in order to reduce coupling to the implementation details, which is to be expected, I would suggest a way to extract implementation details from existing types by introducing a variation point.”



You



Me

“Yes, you may have a point there. But since we’ll have to extend operations fairly often, we probably should consider designing the system in such a way that we can build on and reuse a given implementation easily.”

Terminology



"The use of [design patterns] provides us in our daily lives with decisive speed advantages for understanding complex structures. This is also why patterns found their way into software development years ago. ... Consistently applied patterns help us deal with the complexity of source code."

(Carola Lilienthal, Software Architecture Metrics)

The Attitude Toward Design Patterns



Agustin P. 1 month ago

Really? Design Patterns in 2021?

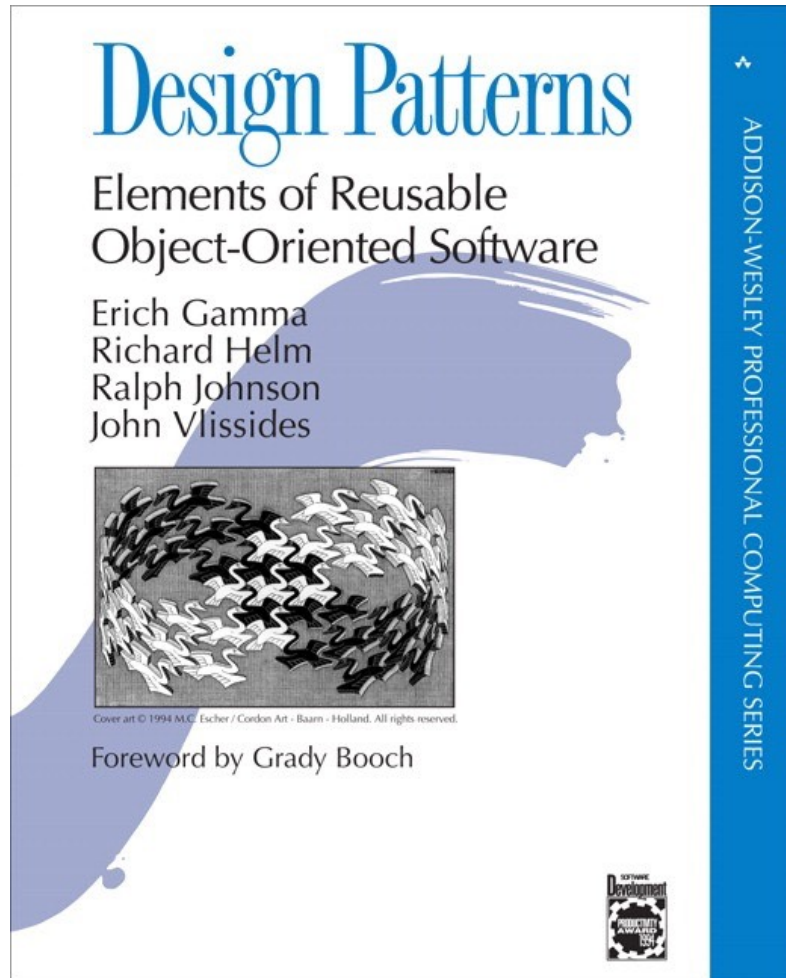


REPLY

Guidelines

Guideline: Design Patterns are Everywhere!

Terminology



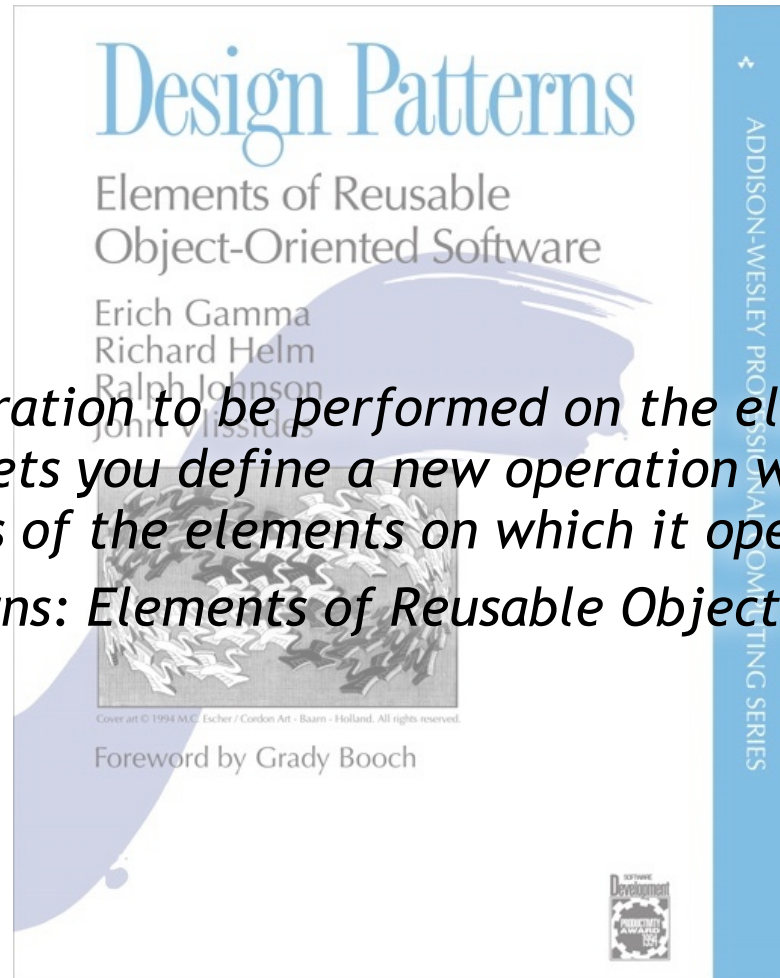
- The Gang of Four (GoF) book
- Published in 1994
- Source of 23 of the most commonly used design patterns
- Almost all design patterns are based on inheritance

2.3. Visitor

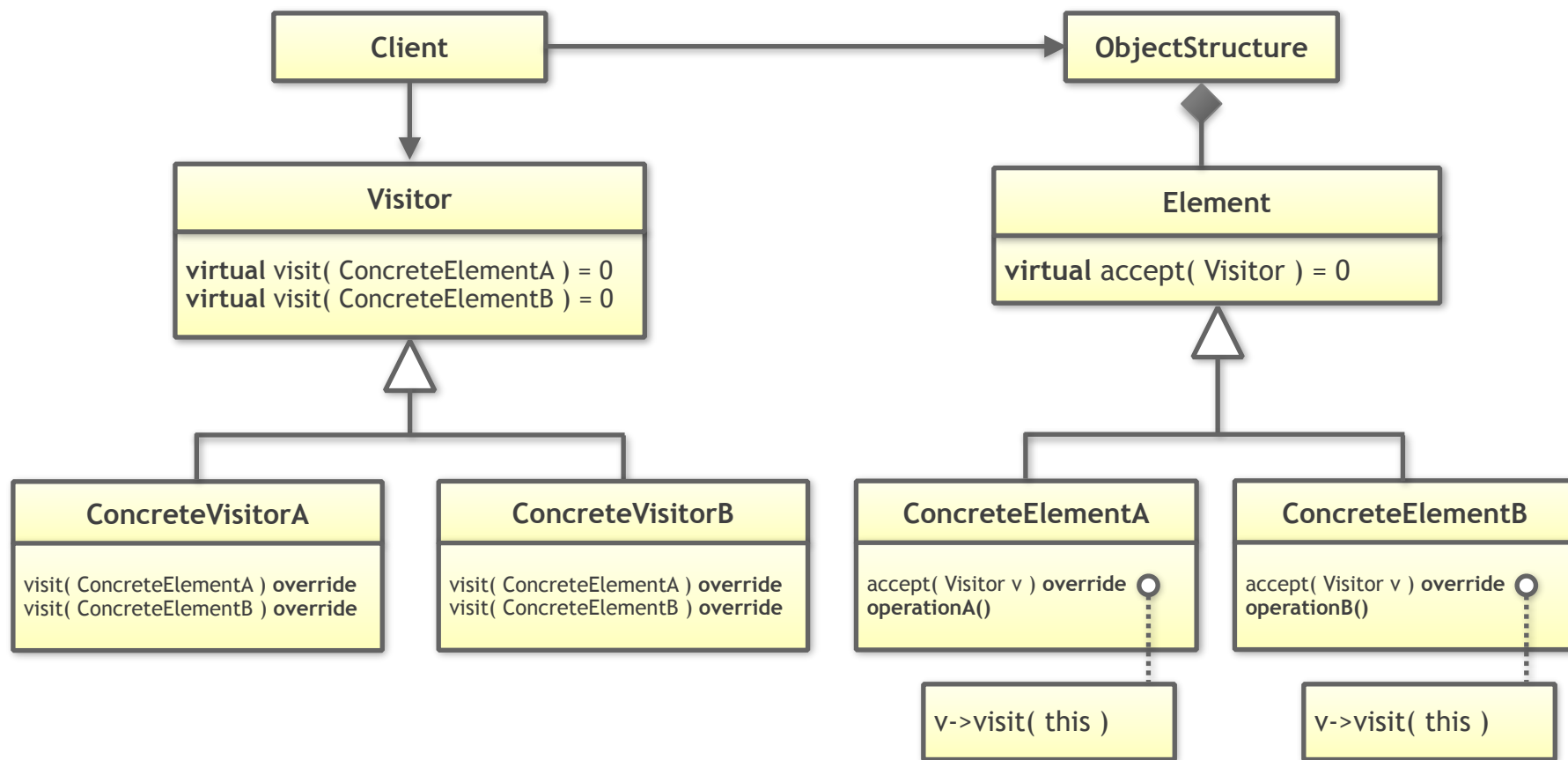
The Intent

”Represent an operation to be performed on the elements of an object structure. Visitor lets you define a new operation without changing the classes of the elements on which it operates.”

(GoF, Design Patterns: Elements of Reusable Object-Oriented Software)

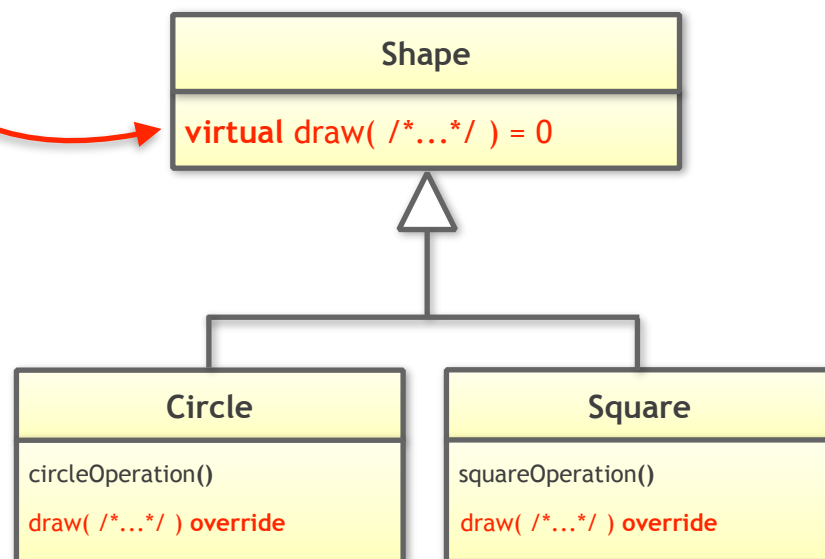


The Classic Visitor Design Pattern

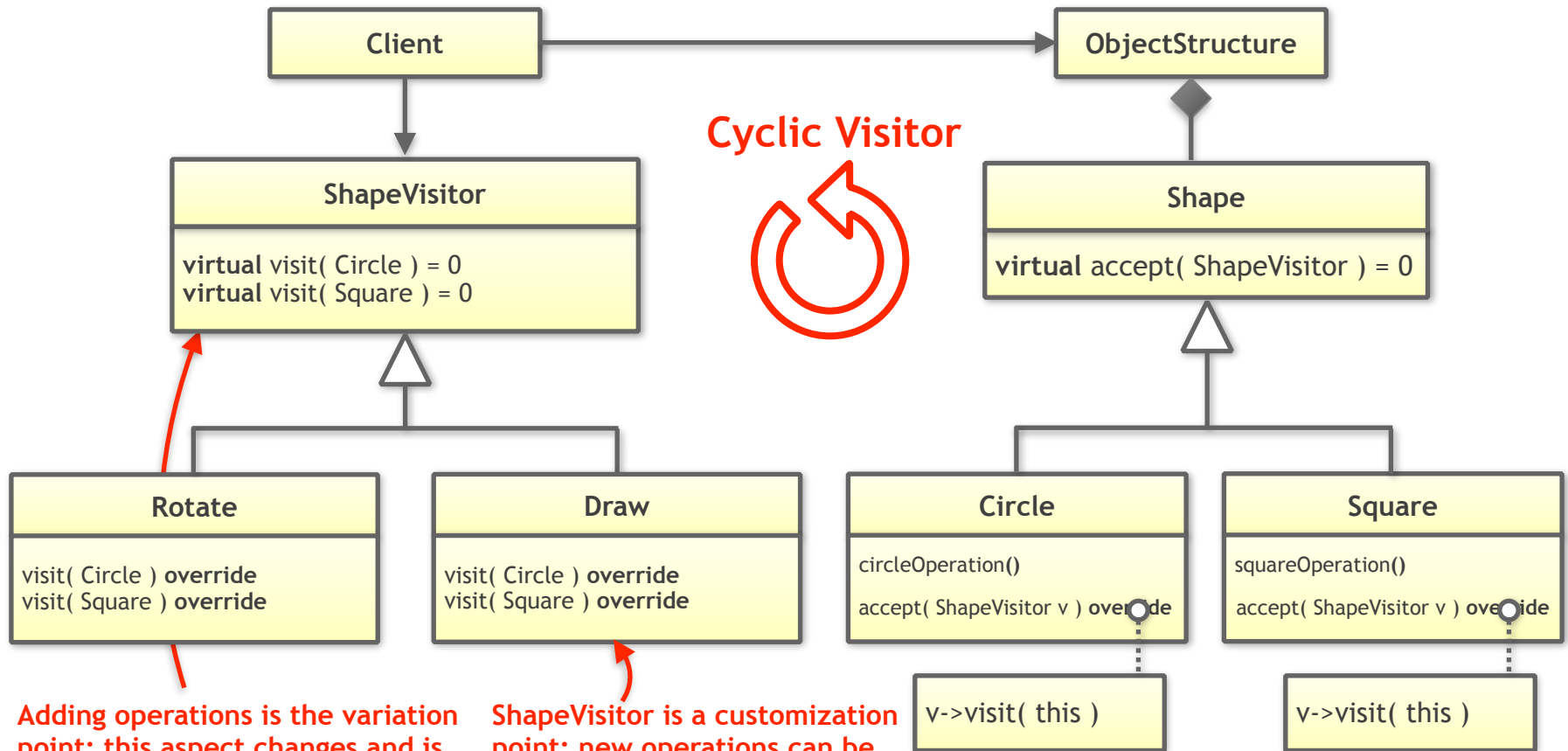


The Classic Visitor Design Pattern

The weakness of Object-Oriented Programming: adding an operation requires an update of all derived classes. This violates the Open-Closed Principle (OCP).



The Classic Visitor Design Pattern



Adding operations is the variation point; this aspect changes and is extracted and isolated; this fulfills the Single-Responsibility Principle (SRP)

ShapeVisitor is a customization point: new operations can be added without modifying any existing code; this fulfills the Open-Closed Principle (OCP)

The Classic Visitor Design Pattern

Task (C_Cpp_Software_Design/Visitor/Visitor_Classic):

Step 1: Refactor the given Shape hierarchy by means of the visitor design pattern to enable the easy addition of operations (e.g. drawing, rotating, serialization, ...).

Step 2: Switch from one to another graphics library. Discuss the feasibility of the change: how easy is the change? How many pieces of code on which level of the architecture have to be touched?

Step 3: Implement the `area()` operations as a classic visitor. Hint: the area of a circle is $\text{radius} \times \text{radius} \times M_PI$, the area of a square is $\text{side} \times \text{side}$.

Step 4: Add the new feature to serialize shapes by means of the `FastSerialization` library. Observe how well the new code can be integrated and how many new dependencies are created.

A Visitor-Based Solution

```
class Circle;
class Square;

class ShapeVisitor
{
public:
    virtual ~ShapeVisitor() = default;

    virtual void visit( Circle const& ) const = 0;
    virtual void visit( Square const& ) const = 0;
};

class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void accept( ShapeVisitor const& ) = 0;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
    {}
}
```

A Visitor-Based Solution

```
class Circle;
class Square;

class ShapeVisitor
{
public:
    virtual ~ShapeVisitor() = default;

    virtual void visit( Circle const& ) const = 0;
    virtual void visit( Square const& ) const = 0;
};

class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void accept( ShapeVisitor const& ) = 0;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
    {}
}
```

A Visitor-Based Solution

```
class Circle;
class Square;

class ShapeVisitor
{
public:
    virtual ~ShapeVisitor() = default;

    virtual void visit( Circle const& ) const = 0;
    virtual void visit( Square const& ) const = 0;
};

class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void accept( ShapeVisitor const& ) = 0;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
    {}
}
```


A Visitor-Based Solution

```
virtual void accept( ShapeVisitor const& ) = 0;  
};
```

```
class Circle : public Shape  
{  
public:  
    explicit Circle( double rad )  
        : radius{ rad }  
        , // ... Remaining data members  
        {}  
  
    double getRadius() const noexcept;  
    // ... getCenter(), getRotation(), ...  
  
    void accept( ShapeVisitor const& ) override;  
  
    // ...  
  
private:  
    double radius;  
    // ... Remaining data members  
};
```

```
class Square : public Shape  
{  
public:  
    explicit Square( double s )  
        : side{ s }  
        , // ... Remaining data members  
        {}  
};
```

A Visitor-Based Solution

```
        // ... Remaining data members
    };

    class Square : public Shape
    {
    public:
        explicit Square( double s )
            : side{ s }
            , // ... Remaining data members
            {}

        double getSide() const noexcept;
        // ... getCenter(), getRotation(), ...

        void accept( ShapeVisitor const& ) override;

        // ...

    private:
        double side;
        // ... Remaining data members
    };

    class Draw : public ShapeVisitor
    {
    public:
        void visit( Circle const& ) const override;
        void visit( Square const& ) const override;
    };
```

A Visitor-Based Solution

```
double side;
// ... Remaining data members
};

class Draw : public ShapeVisitor
{
public:
    void visit( Circle const& ) const override;
    void visit( Square const& ) const override;
};

void drawAllShapes( std::vector<std::unique_ptr<Shape>> const& shapes )
{
    for( auto const& s : shapes )
    {
        s->accept( Draw{} )
    }
}

int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( std::make_unique<Circle>( 2.0 ) );
    shapes.emplace_back( std::make_unique<Square>( 1.5 ) );
}
```

A Visitor-Based Solution

```
void drawAllShapes( std::vector<std::unique_ptr<Shape>> &shapes )  
{  
    for( auto const& s : shapes )  
    {  
        s->accept( Draw{} )  
    }  
}  
  
int main()  
{  
    using Shapes = std::vector<std::unique_ptr<Shape>>;  
  
    // Creating some shapes  
    Shapes shapes;  
    shapes.emplace_back( std::make_unique<Circle>( 2.0 ) );  
    shapes.emplace_back( std::make_unique<Square>( 1.5 ) );  
    shapes.emplace_back( std::make_unique<Circle>( 4.2 ) );  
  
    // Drawing all shapes  
    drawAllShapes( shapes );  
}
```

The Classic Visitor Design Pattern

Task (C_Cpp_Software_Design/Visitor/Visitor_Refactoring):

Step 1: Implement the `area()` operations as a classic visitor. Hint: the area of a circle is `radius*radius*M_PI`, the area of a square is `side*side`.

Step 2: Refactor the classic Visitor solution by a value semantics based solution. Note that the general behavior should remain unchanged.

Step 3: Switch from one to another graphics library. Discuss the feasibility of the change: how easy is the change? How many pieces of code on which level of the architecture have to be touched?

Step 4: Add the new feature to serialize shapes by means of the `FastSerialization` library. Observe how well the new code can be integrated and how many new dependencies are created.

Step 5: Discuss the advantages of the value semantics based solution in comparison to the classic solution.

A “Modern C++” Solution: `std::variant`

```
using Shape = std::variant<Circle, Square>;
```

```
Shape s = ...;
```

```
if( std::holds_alternative<Circle>(s) ) {  
    double radius = std::get<Circle>(s).radius();  
    double area = radius * radius * std::pi_v;  
}  
else if( std::holds_alternative<Square>(s) ) {  
    double side = std::get<Square>(s).side();  
    double area = side * side;  
}  
// ...
```

A “Modern C++” Solution: `std::variant`



“Nothing prevents you to handle a certain case. If you forget to handle one of the cases, it is a bug. And this becomes particularly problematic if, God forbids, somebody ever comes around and adds a new kind of shape to the variant.”

(Andreas Weis, Building C++ Interfaces That Are Hard to Use Incorrectly, ACCU 2023)

A “Modern C++” Solution: `std::variant`

```
double getArea( Circle const& circle );  
double getArea( Square const& square );
```

```
using Shape = std::variant<Circle, Square>;
```

```
Shape s = ...;
```

```
double area =  
    std::visit(  
        []( auto const& shape ){ return getArea(shape); }, s );
```


std::variant - Implementation Details

```
template< typename T, typename V >
constexpr auto make_func() {
    return + [] ( const char* b, V v ) {
        const auto& x = *reinterpret_cast<const T*>(b);
        v(x);
    };
}
```

```
template< typename... Ts, typename V >
void foo( std::size_t i, const char* b, V v ) {
    static constexpr std::array<void*(const char*, V v ),
        sizeof...(Ts)> table = { make_func<Ts,V>()... };
    table[i](b,v);
}
```

Returning from a Visitor — The Classic Way

```
class ShapeVisitor
{
public:
    virtual ~ShapeVisitor() = default;

    virtual void visit( Circle const& ) const = 0;
    virtual void visit( Square const& ) const = 0;
};

class Area : public ShapeVisitor
{
public:
    void visit( Circle const& circle ) const override
    {
        result_ = circle.radius()*circle.radius()*M_PI;
    }

    void visit( Square const& square ) const override
    {
        result_ = square.side()*square.side();
    }

    double getResult() const { return result_; }

private:
    double mutable result_;
};
```

Returning from a Visitor — The Classic Way

```
class ShapeVisitor
{
public:
    virtual ~ShapeVisitor() = default;

    virtual void visit( Circle const& ) const = 0;
    virtual void visit( Square const& ) const = 0;
};

class Area : public ShapeVisitor
{
public:
    void visit( Circle const& circle ) const override
    {
        result_ = circle.radius()*circle.radius()*M_PI;
    }

    void visit( Square const& square ) const override
    {
        result_ = square.side()*square.side();
    }

    double getResult() const { return result_; }

private:
    double mutable result_;
};
```

Returning from a Visitor – The Classic Way

```
class ShapeVisitor
{
public:
    virtual ~ShapeVisitor() = default;

    virtual void visit( Circle const& ) const = 0;
    virtual void visit( Square const& ) const = 0;
};

class Area : public ShapeVisitor
{
public:
    void visit( Circle const& circle ) const override
    {
        result_ = circle.radius()*circle.radius()*M_PI;
    }

    void visit( Square const& square ) const override
    {
        result_ = square.side()*square.side();
    }

    double getResult() const { return result_; }

private:
    double mutable result_;
};
```

Returning from a Visitor — The Classic Way

```
class Area : public ShapeVisitor
{
public:
    void visit( Circle const& circle ) const override
    {
        result_ = circle.radius()*circle.radius()*M_PI;
    }

    void visit( Square const& square ) const override
    {
        result_ = square.side()*square.side();
    }

    double getResult() const { return result_; }

private:
    double mutable result_;
};

int main()
{
    std::unique_ptr<Shape> shape = ...;
    Area area{};

    shape.accept( area );
    double const result = area.getResult();
}
```

Returning from a Visitor — `std::variant`

```
class Area
{
public:
    double operator()( Circle const& circle ) const
    {
        return circle.radius() * circle.radius() * M_PI;
    }
    double operator()( Square const& square ) const
    {
        return square.side() * square.side();
    }
};

using Shape = std::variant<Circle,Square>;

int main()
{
    Shape shape = ...;

    double const area = std::visit( Area{}, shape );
}
```

Note that the every visitor can return some different. This offers much more flexibility than an inheritance hierarchy.

Inheritance Creates a Tight Coupling



"If a class relationship can be expressed in more than one way, use the weakest relationship that's practical. Given that inheritance is nearly the strongest relationship you can express in C++ (second only to friendship), it's only really appropriate when there is no equivalent weaker alternative."

(Herb Sutter, Exceptional C++)

A “Modern C++” Solution: `std::variant`

```
class Circle
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

private:
    double radius;
    // ... Remaining data members
};
```

```
class Square
{
public:
    explicit Square( double s )
        : side{ s }
        , // ... Remaining data members
    {}

    double getSide() const noexcept;
    // ... getCenter(), getRotation(), ...
```

```
private:
    double side;
```


A “Modern C++” Solution: `std::variant`

```
class Circle
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

private:
    double radius;
    // ... Remaining data members
};
```

```
class Square
{
public:
    explicit Square( double s )
        : side{ s }
        , // ... Remaining data members
    {}

    double getSide() const noexcept;
    // ... getCenter(), getRotation(), ...
```

```
private:
    double side;
```

A “Modern C++” Solution: `std::variant`

```
private:
    double radius;
    // ... Remaining data members
};

class Square
{
public:
    explicit Square( double s )
        : side{ s }
        , // ... Remaining data members
        {}

    double getSide() const noexcept;
    // ... getCenter(), getRotation(), ...

private:
    double side;
    // ... Remaining data members
};

using Shape = std::variant<Circle,Square>;

class GLDrawer
{
public:
    void operator()( Circle const& ) const;
    void operator()( Square const& ) const;
```

A “Modern C++” Solution: `std::variant`

```
private:
    double side;
    // ... Remaining data members
};

using Shape = std::variant<Circle, Square>;

class GLDrawer
{
public:
    void operator()( Circle const& ) const;
    void operator()( Square const& ) const;
    // ...
};

void drawAllShapes( std::vector<Shape> const& shapes )
{
    for( auto const& s : shapes )
    {
        std::visit( GLDrawer{}, s );
    }
}

int main()
{
    using Shapes = std::vector<Shape>;
```

A “Modern C++” Solution: `std::variant`

```
private:
    double side;
    // ... Remaining data members
};

using Shape = std::variant<Circle, Square>;

class GLDrawer
{
public:
    void operator()( Circle const& ) const;
    void operator()( Square const& ) const;
    // ...
};

void drawAllShapes( std::vector<Shape> const& shapes )
{
    for( auto const& s : shapes )
    {
        std::visit( GLDrawer{}, s );
    }
}

int main()
{
    using Shapes = std::vector<Shape>;
```

A “Modern C++” Solution: `std::variant`

```
private:
    double side;
    // ... Remaining data members
};

using Shape = std::variant<Circle, Square>;

class GLDrawer
{
public:
    void operator()( Circle const& ) const;
    void operator()( Square const& ) const;
    // ...
};

void drawAllShapes( std::vector<Shape> const& shapes )
{
    for( auto const& s : shapes )
    {
        std::visit( GLDrawer{}, s );
    }
}

int main()
{
    using Shapes = std::vector<Shape>;
```

A “Modern C++” Solution: `std::variant`

```
void drawAllShapes( std::vector<Shape> const& shapes )
{
    for( auto const& s : shapes )
    {
        std::visit( GLDrawer{}, s );
    }
}
```

```
int main()
{
    using Shapes = std::vector<Shape>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( Circle{ 2.0 } );
    shapes.emplace_back( Square{ 1.5 } );
    shapes.emplace_back( Circle{ 4.2 } );

    // Drawing all shapes
    drawAllShapes( shapes );
}
```

Performance Comparison

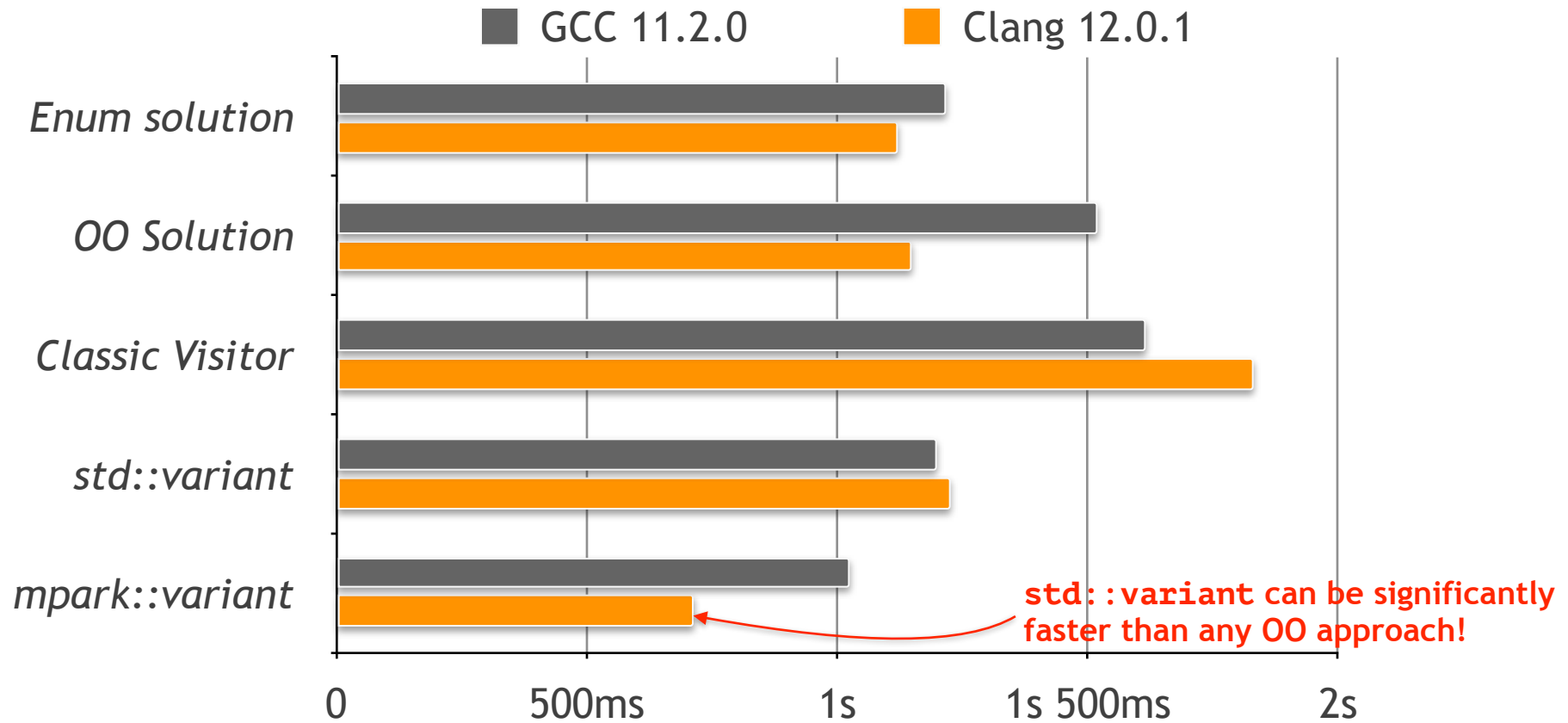
Performance ... *sigh*

Do you promise to not
take the following results
too seriously and as
qualitative results only?

Performance Comparison

- Using four different kinds of shape: circles, squares, ellipses and rectangles
- Using 10000 randomly generated shapes
- Performing 25000 `translate()` operations each
- Benchmarks with GCC-11.2.0 and Clang-12.0.1
- 8-core Intel Core i7 with 3.8 Ghz, 64 GB of main memory

Performance Comparison



Unknown Reviewer on Design Patterns



"I believe that object-oriented programming and especially its theory is overestimated. ... C++ always had templates, and now also has `std::variant`, which makes most of the use of inheritance unnecessary."

(Unknown Reviewer)

A “Modern C++” Solution: A Second Look

```
private:
    double side;
    // ... Remaining data members
};

using Shape = std::variant<Circle, Square>;

class Draw
{
public:
    void operator()( Circle const& ) const;
    void operator()( Square const& ) const;
    // ...
};

void drawAllShapes( std::vector<Shape> const& shapes )
{
    for( auto const& s : shapes )
    {
        std::visit( Draw{}, s );
    }
}

int main()
{
    using Shapes = std::vector<Shape>;
```

A “Modern C++” Solution: A Second Look

```
private:
    double side;
    // ... Remaining data members
};
```

```
using Shape = std::variant<Circle, Square>;
```

High level (stable, low dependencies)

Low level (volatile, malleable, high dependencies)

Architectural
Boundary

```
class Draw
{
public:
    void operator()( Circle const& ) const;
    void operator()( Square const& ) const;
    // ...
};
```

```
void drawAllShapes( std::vector<Shape> const& shapes )
{
    for( auto const& s : shapes )
    {
        std::visit( Draw{}, s );
    }
}
```

A “Modern C++” Solution: A Second Look

```
private:
    double side;
    // ... Remaining data members
};
```

```
using Shape = std::variant<Circle, Square>;
```

Note that the abstraction is on my side of the code. You cannot add any more shape type!



My Code

Architectural
Boundary

Your Code

```
class Draw
{
public:
    void operator()( Circle const& ) const;
    void operator()( Square const& ) const;
    // ...
};

void drawAllShapes( std::vector<Shape> const& shapes )
{
    for( auto const& s : shapes )
    {
        std::visit( Draw{}, s );
    }
}
```

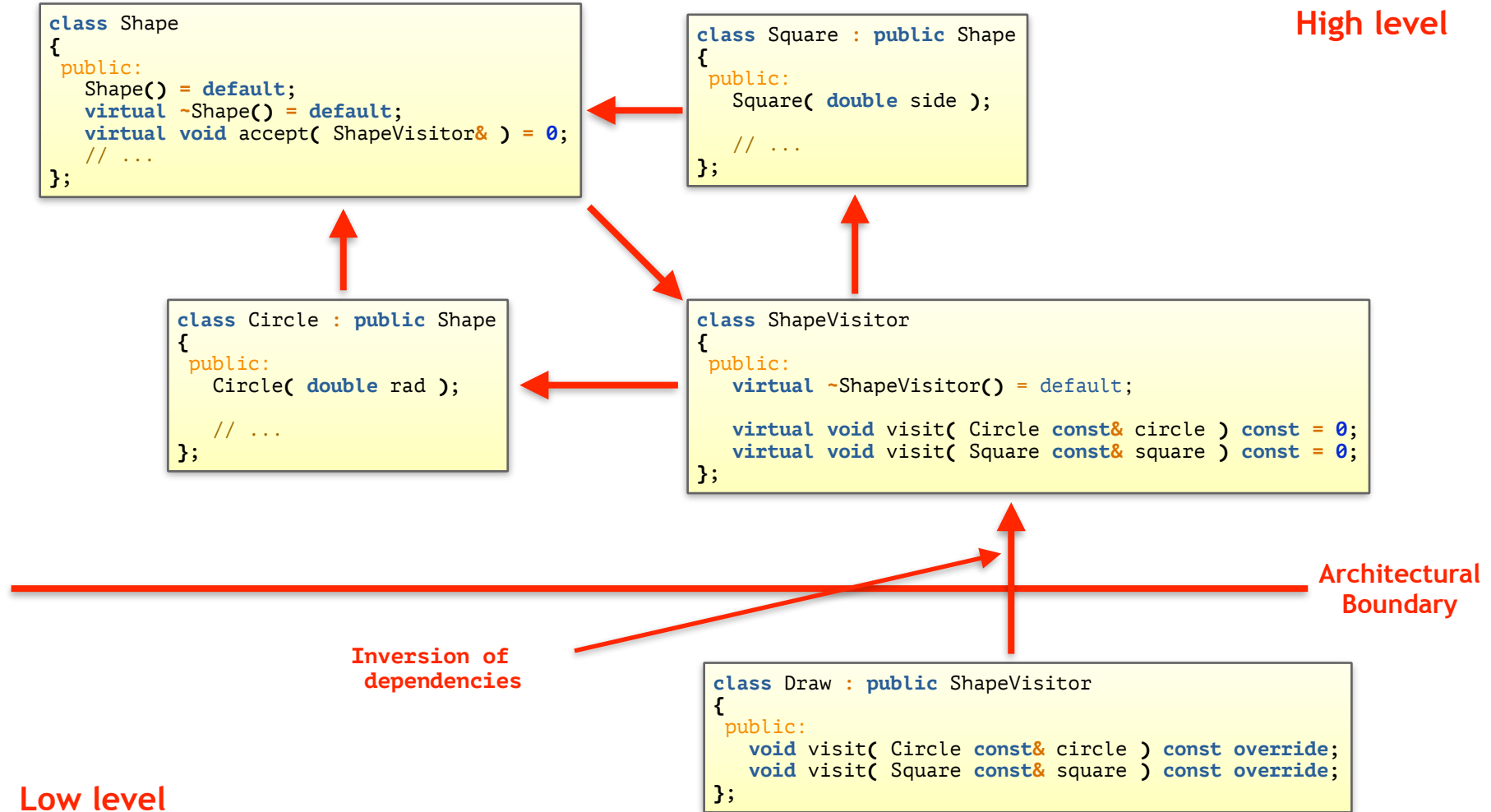
There Is No Silver Bullet



*"For every upside, there is a downside."
(Neal Ford, Mark Richards, Head First Software
Architecture)*



Dependency Structure (Cyclic Visitor)



Dependency Structure (Cyclic Visitor)

High level

```
class Circle
{
public:
    Circle( double rad );

    // ...
};
```

```
class Square
{
public:
    Square( double side );

    // ...
};
```

```
using Shape = std::variant<Circle,Square>;
```

Automatic inversion
of dependencies

```
class Draw
{
public:
    void operator()( Circle const& circle ) const;
    void operator()( Square const& square ) const;
};
```

Architectural
Boundary

Architectural
Boundary

Low level

The Myth: `std::variant` is Static Polymorphism

The Myth: `std::variant` is Static Polymorphism



“... A great many tutorials exist for replacing virtual functions with compile-time polymorphism mechanisms, such as `std::variant` and templates. ...”

(Abstract of CppCon talk)

The Myth: `std::variant` is Static Polymorphism



“... This talk will explore techniques for replacing runtime polymorphism with compile-time polymorphism such that virtual functions are never necessary. ...”

(Abstract of CppCon talk)

`std::variant` is Runtime Polymorphism!

`std::variant` is a runtime
polymorphism mechanism!

It's content is only known at runtime, i.e. it
uses runtime dispatch.

The Classic Visitor Design Pattern

The classic Visitor design pattern ...

- ... requires a base class (dependency);
- ... promotes heap allocation;
- ... requires memory management.

Using `std::variant` instead of the classic Visitor design pattern ...

- ... simplifies code (a lot!);
- ... facilitates comprehension;
- ... reduces dependencies.

std::variant — Advantages/Disadvantages

Use std::variant if ...

- ... you have a **closed set** of known types;
- ... you want to **extend functionality**, not types;
- ... you don't need to abstract from the **concrete types**;
- ... you require **maximum performance**.

Don't use std::variant if ...

- ... you have an **open set** of types;
- ... you want to **extend types**, not functionality;
- ... you want to use the abstraction across **architectural boundaries**;
- ... you want to **hide implementation details** about the alternatives;
- ... you need an **abstraction of the operations** (e.g. via base pointer);
- ... performance is not the **primary concern**.

2. C++ Software Design - Visitor

YouTube

Search

Cppcon | 2019
The C++ Conference
cppcon.org

C++'s evolution priorities

Historical

add things

fix things

simplify

add things

fix things

simplify

A future worth considering?

5

Herb Sutter

De-fragmenting C++:
Making Exceptions and RTTI
More Affordable and Usable
("Simplifying C++" #6 of N)

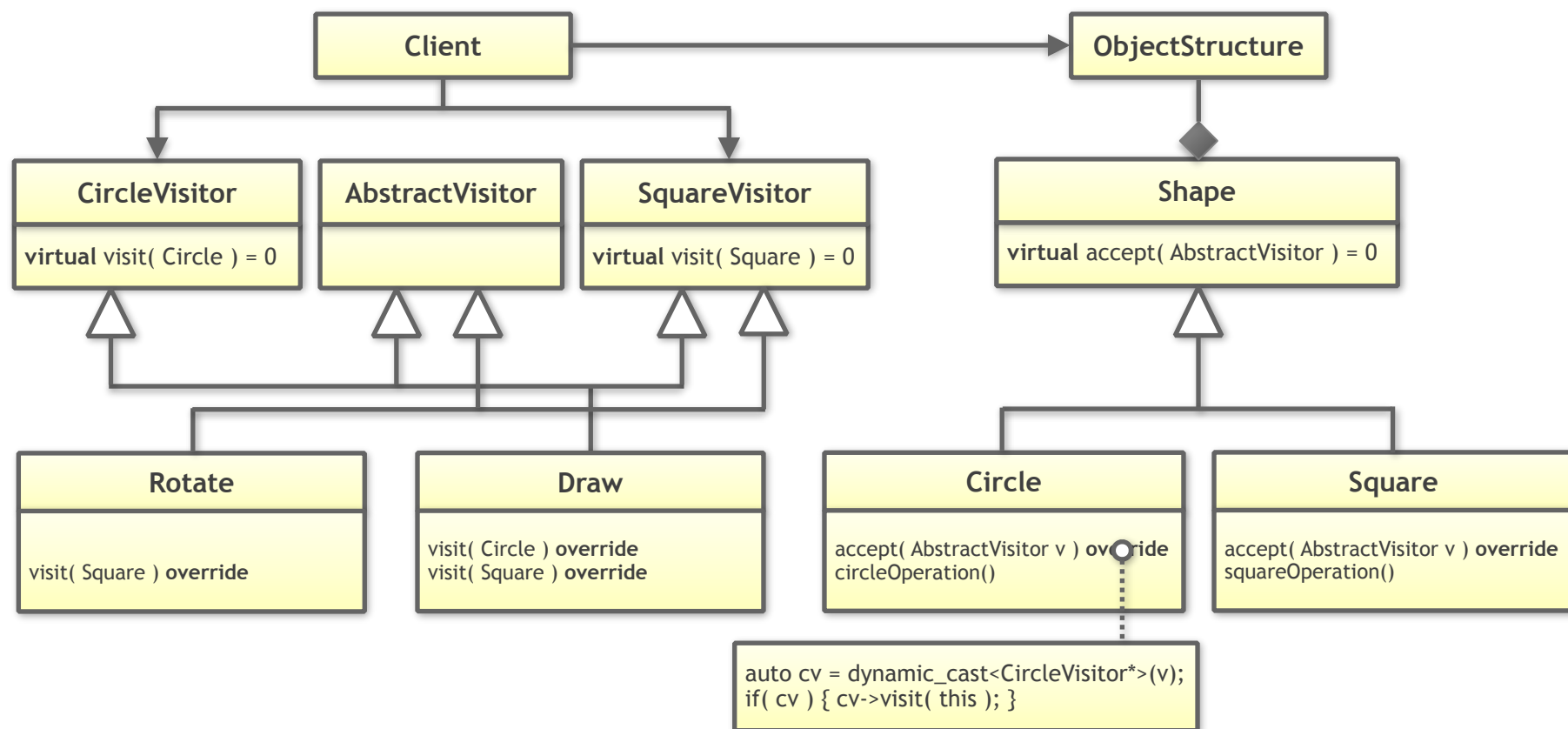
Video Sponsorship Provided By:
ansatz

8:45 / 1:33:11

The Acyclic Visitor Design Pattern

Do we always have to
choose between adding
types or operations?

The Acyclic Visitor Design Pattern



An Acyclic Visitor-Based Solution

```
class AbstractVisitor
{
public:
    virtual ~AbstractVisitor() = default;
};

class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void accept( Visitor const& ) = 0;
};

template< typename T >
class Visitor
{
public:
    virtual ~Visitor() = default;

    virtual void visit( const T& ) const = 0;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad )
```

An Acyclic Visitor-Based Solution

```
class AbstractVisitor
{
public:
    virtual ~AbstractVisitor() = default;
};

class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void accept( Visitor const& ) = 0;
};

template< typename T >
class Visitor
{
public:
    virtual ~Visitor() = default;

    virtual void visit( const T& ) const = 0;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad )
```

An Acyclic Visitor-Based Solution

```
class AbstractVisitor
{
public:
    virtual ~AbstractVisitor() = default;
};

class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void accept( Visitor const& ) = 0;
};

template< typename T >
class Visitor
{
public:
    virtual ~Visitor() = default;

    virtual void visit( const T& ) const = 0;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad )
```

An Acyclic Visitor-Based Solution

```
class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void accept( Visitor const& ) = 0;
};
```

```
template< typename T >
class Visitor
{
public:
    virtual ~Visitor() = default;

    virtual void visit( const T& ) const = 0;
};
```

```
class Circle : public Shape
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...
};
```

An Acyclic Visitor-Based Solution

```
};
```

```
class Circle : public Shape
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
        {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

    void accept( const AbstractVisitor& v ) override {
        if( auto cv = dynamic_cast<const Visitor<Circle>*>(&v) ) {
            cv->visit( *this );
        }
    }

    // ...

private:
    double radius;
    // ... Remaining data members
};
```

```
class Square : public Shape
{
public:
```

An Acyclic Visitor-Based Solution

```
class Square : public Shape
{
public:
    explicit Square( double s )
        : side{ s }
        , // ... Remaining data members
    {}

    double getSide() const noexcept;
    // ... getCenter(), getRotation(), ...

    void accept( const AbstractVisitor& v ) override {
        if( auto sv = dynamic_cast<const Visitor<Square>*>(&v) ) {
            sv->visit( *this );
        }
    }

    // ...

private:
    double side;
    // ... Remaining data members
};
```

```
class Draw : public AbstractVisitor
, public Visitor<Circle>
, public Visitor<Square>
```

An Acyclic Visitor-Based Solution

```
// ... Remaining data members
};

class Draw : public AbstractVisitor
    , public Visitor<Circle>
    , public Visitor<Square>
{
public:
    void visit( Circle const& ) const override;
    void visit( Square const& ) const override;
};

void drawAllShapes( std::vector<std::unique_ptr<Shape>> const& shapes )
{
    for( auto const& s : shapes )
    {
        s->accept( Draw{} )
    }
}

int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( std::make_unique<Circle>( 2.0 ) );
    shapes.emplace_back( std::make_unique<Square>( 1.5 ) );
}
```


An Acyclic Visitor-Based Solution

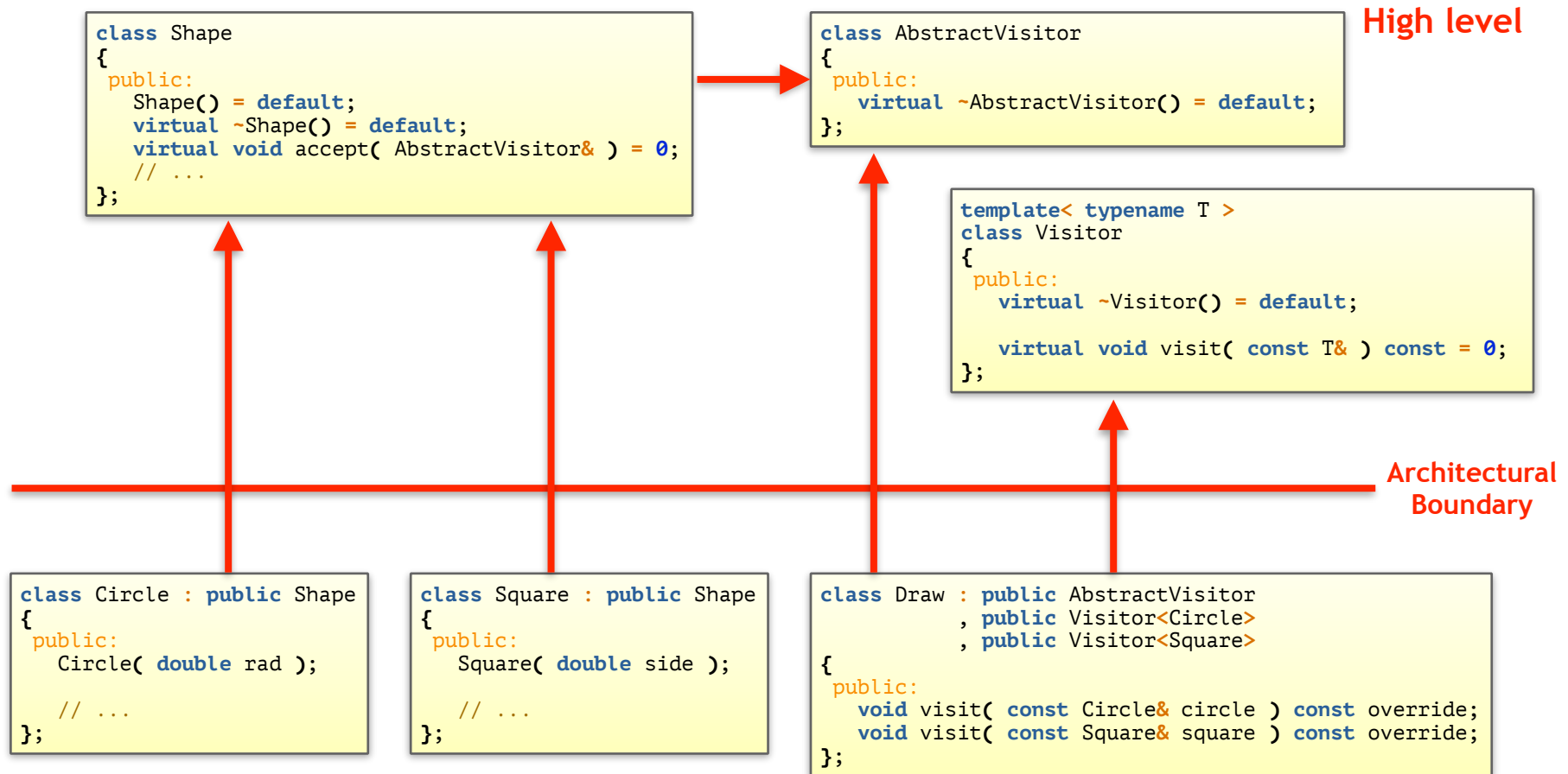
```
void drawAllShapes( std::vector<std::unique_ptr<Shape>> const& shapes )
{
    for( auto const& s : shapes )
    {
        s->accept( Draw{} )
    }
}

int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( std::make_unique<Circle>( 2.0 ) );
    shapes.emplace_back( std::make_unique<Square>( 1.5 ) );
    shapes.emplace_back( std::make_unique<Circle>( 4.2 ) );

    // Drawing all shapes
    drawAllShapes( shapes );
}
```

Dependency Structure (Acyclic Visitor)



Low level

std::variant — Return Values and Parameters

Task (C_Cpp_Software_Design/Visitor/Variant): Implement the `translate()` and `area()` operations for the given Shape variant. Hint: the area of a circle is `radius*radius*M_PI`, the area of a square is `side*side`.

Guidelines

Guideline: Use the Visitor design pattern to enable the extension of operations for a closed set of types.

Guideline: Don't use the Visitor design pattern when you need to frequently extend the set of types.

Guideline: Avoid the over-/abuse of inheritance.

Guideline: Prefer multi-paradigm solutions.

2.4. Interlude 1: The Expression Problem

The Expression Problem



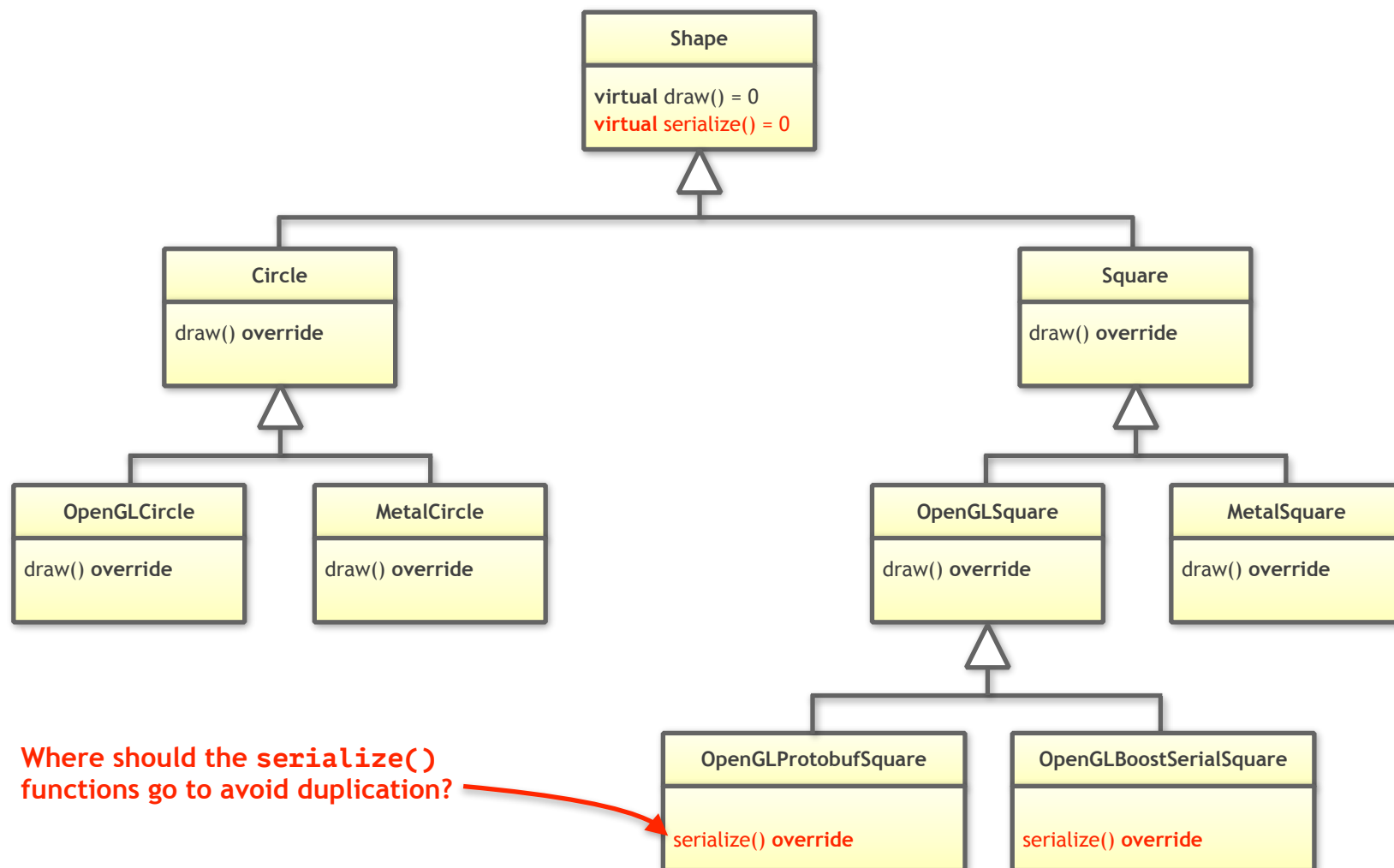
"The Expression Problem is a new name for an old problem. The goal is to define a datatype by cases, where one can add new cases to the datatype and new functions over the datatype, without recompiling existing code, and while retaining static type safety (e.g., no casts)."

Whether a language can solve the Expression Problem is a salient indicator of its capacity for expression. One can think of cases as rows and functions as columns in a table. In a functional language, the rows are fixed (cases in a datatype declaration) but it is easy to add new columns (functions). In an object-oriented language, the columns are fixed (methods in a class declaration) but it is easy to add new rows (subclasses). We want to make it easy to add either rows or columns."

(Philip Wadler, <https://homepages.inf.ed.ac.uk/wadler/papers/expression/expression.txt>)

2.5. Strategy

A Design Challenge



A Design Challenge

```
class OpenGLProtobufSquare : public Square
{
    public:
        // ...
        virtual void draw( Screen& s, /*...*/ ) const;
        virtual void serialize( ByteStream& bs, /*...*/ ) const;
        // ...
};
```

These functions easily result in coupling/dependencies. How do we extract the logic?

Using inheritance to solve our problem easily leads to ...

- ... many derived classes;
- ... ridiculous class names;
- ... deep inheritance hierarchies;
- ... duplication between similar implementations (DRY);
- ... impeded maintenance;
- ... (almost) impossible extensions (OCP);
- ...

A Design Challenge



*"Inheritance is Rarely the Answer."
(Andrew Hunt, David Thomas, The Pragmatic
Programmer)*



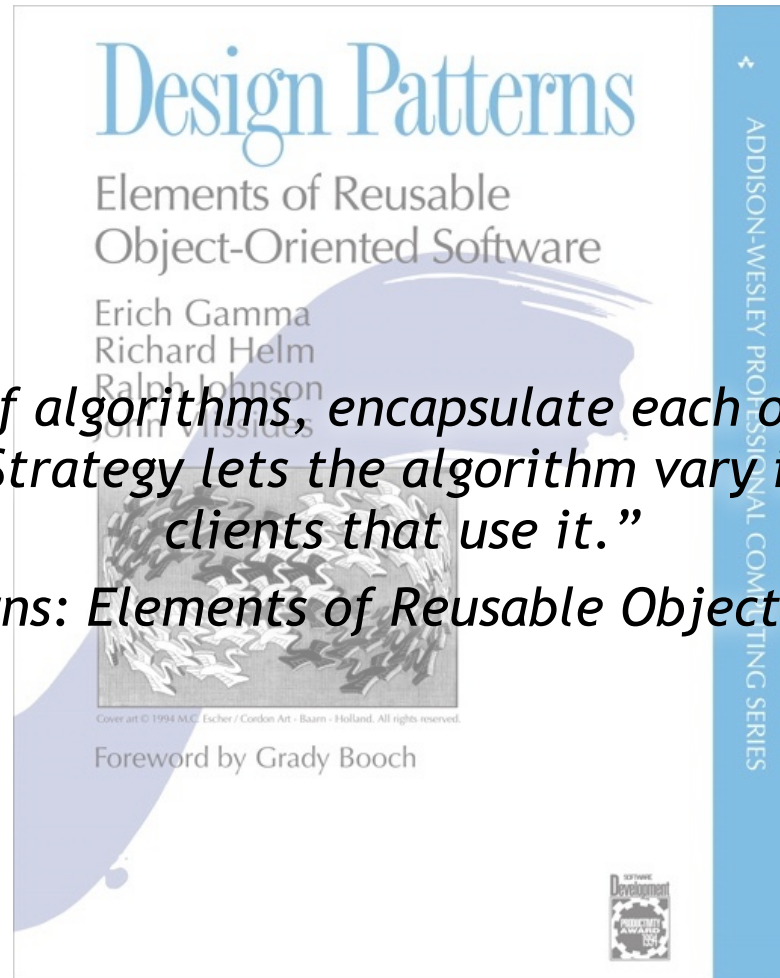
Guidelines

Guideline: Prefer containment (composition/aggregation) to inheritance.

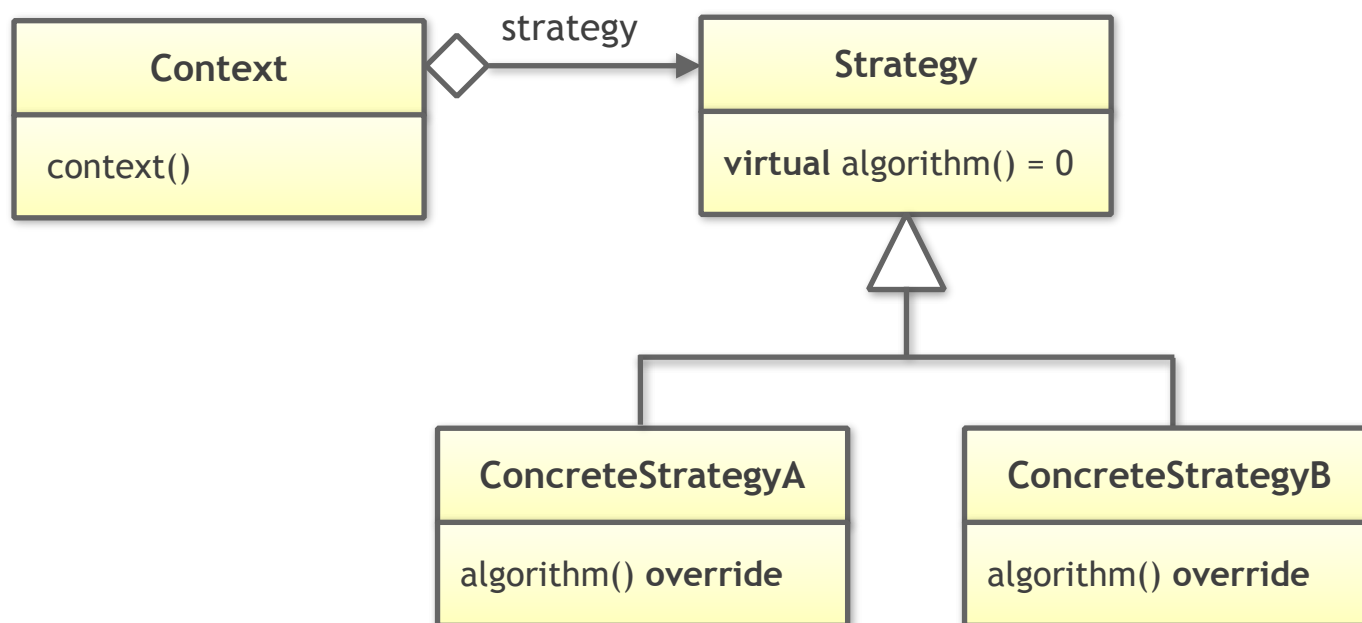
The Intent

"Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it."

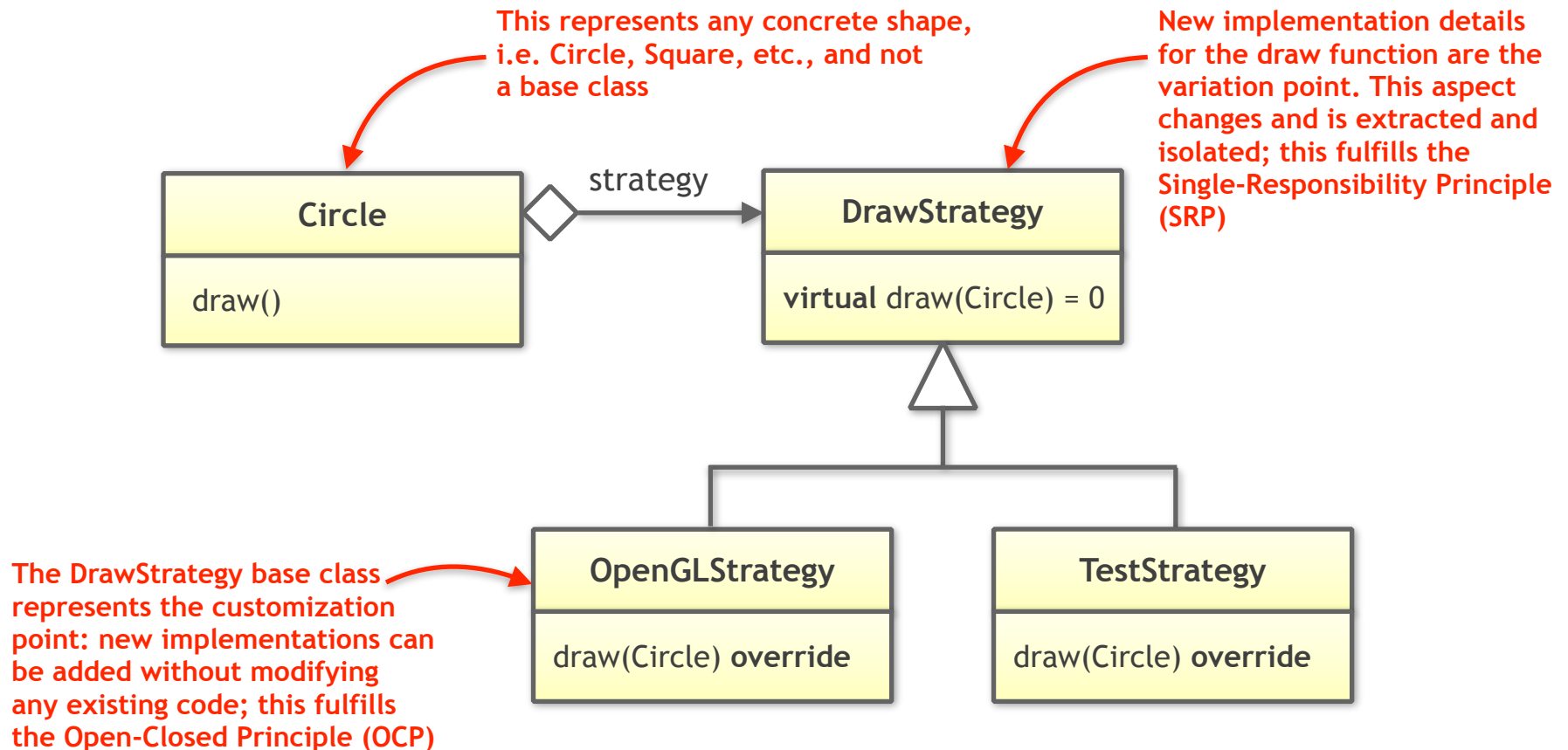
(GoF, Design Patterns: Elements of Reusable Object-Oriented Software)



The Classic Strategy Design Pattern



The Classic Strategy Design Pattern



An Example from the Standard Library

Alternatively we could use static polymorphism:

```
template< typename OP >  
void doSomething( OP strategy );
```

This form of the strategy pattern is used in the standard library:

```
std::vector<int> numbers{ 1, 2, 3, 4, 5, 6, 7 };  
  
std::accumulate( begin(numbers), end(numbers), 0  
                 , std::plus<>{} );
```

The Classic Strategy Design Pattern

Task (C_Cpp_Software_Design/Strategy/Strategy_Classic):

Step 1: Refactor the given Shape hierarchy by means of the strategy design pattern to enable the runtime configuration of the draw() operation.

Step 2: Switch from one to another graphics library. Discuss the feasibility of the change: how easy is the change? How many pieces of code on which level of the architecture have to be touched?

Step 3: Add an area() operations for all shapes. How easy is the change? How many pieces of code on which level of the architecture have to be touched? Hint: the area of a circle is $\text{radius} * \text{radius} * M_PI$, the area of a square is $\text{side} * \text{side}$.

Step 4: Add the new feature to serialize shapes by means of the FastSerialization library. Observe how well the new code can be integrated and how many new dependencies are created.

A Strategy-Based Solution

```
class Circle;
class Square;

class DrawStrategy
{
public:
    virtual ~DrawStrategy() {}

    virtual void draw( const Circle& circle ) const = 0;
    virtual void draw( const Square& square ) const = 0;
};

class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void translate( Vector2D const& ) = 0;
    virtual void rotate( double const& ) = 0;
    virtual void draw() const = 0;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad, std::unique_ptr<DrawStrategy> ds )
        : radius{ rad }
        // Position data members
```

A Strategy-Based Solution

```
class Circle;
class Square;

class DrawStrategy
{
public:
    virtual ~DrawStrategy() {}

    virtual void draw( const Circle& circle ) const = 0;
    virtual void draw( const Square& square ) const = 0;
};

class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void translate( Vector2D const& ) = 0;
    virtual void rotate( double const& ) = 0;
    virtual void draw() const = 0;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad, std::unique_ptr<DrawStrategy> ds )
        : radius{ rad }
        // Position data members
```

A Strategy-Based Solution

```
{
    public:
        virtual ~DrawStrategy() {}

        virtual void draw( const Circle& circle ) const = 0;
        virtual void draw( const Square& square ) const = 0;
};

class Shape
{
    public:
        Shape() = default;
        virtual ~Shape() = default;

        virtual void translate( Vector2D const& ) = 0;
        virtual void rotate( double const& ) = 0;
        virtual void draw() const = 0;
};

class Circle : public Shape
{
    public:
        explicit Circle( double rad, std::unique_ptr<DrawStrategy> ds )
            : radius{ rad }
            , // ... Remaining data members
            , drawer{ std::move(ds) }
        {}

        double getRadius() const noexcept;
};
```

A Strategy-Based Solution

```
class Circle : public Shape
{
public:
    explicit Circle( double rad, std::unique_ptr<DrawStrategy> ds )
        : radius{ rad }
        , // ... Remaining data members
        , drawer{ std::move(ds) }
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

    void translate( Vector2D const& ) override;
    void rotate( double const& ) override;
    void draw() const override;

    // ...

private:
    double radius;
    // ... Remaining data members
    std::unique_ptr<DrawStrategy> drawer;
};
```

```
class Square : public Shape
{
public:
```

A Strategy-Based Solution

```
class Square : public Shape
{
public:
    explicit Square( double s, std::unique_ptr<DrawStrategy> ds )
        : side{ s }
        , // ... Remaining data members
        , drawer{ std::move(ds) }
    {}

    double getSide() const noexcept;
    // ... getCenter(), getRotation(), ...

    void translate( Vector2D const& ) override;
    void rotate( double const& ) override;
    void draw() const override;

    // ...

private:
    double side;
    // ... Remaining data members
    std::unique_ptr<DrawStrategy> drawer;
};

void drawAllShapes( std::vector<std::unique_ptr<Shape>> const& shapes )
{
    for( auto const& s : shapes )
    {
```

A Strategy-Based Solution

```
// ...

private:
    double side;
    // ... Remaining data members
    std::unique_ptr<DrawStrategy> drawer;
};

void drawAllShapes( std::vector<std::unique_ptr<Shape>> const& shapes )
{
    for( auto const& s : shapes )
    {
        s->draw();
    }
}

class GLDrawer : public DrawStrategy
{
public:
    virtual ~GLDrawer() {}

    void draw( Circle const& circle ) const override;
    void draw( Square const& square ) const override;
};

int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;
```

A Strategy-Based Solution

```
void drawAllShapes( std::vector<std::unique_ptr<Shape>> const& shapes )
{
    for( auto const& s : shapes )
    {
        s->draw();
    }
}
```

```
class GLDrawer : public DrawStrategy
{
public:
    virtual ~GLDrawer() {}

    void draw( Circle const& circle ) const override;
    void draw( Square const& square ) const override;
};
```

```
int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;

    Shapes shapes;

    shapes.emplace_back( std::make_unique<Circle>( 2.0
                                                    , std::make_unique<GLDrawer>() ) );
    shapes.emplace_back( std::make_unique<Square>( 1.5
                                                    , std::make_unique<GLDrawer>() ) );
    shapes.emplace_back( std::make_unique<Circle>( 4.2
                                                    , std::make_unique<GLDrawer>() ) );
}
```

A Strategy-Based Solution

```
class GLDrawer : public DrawStrategy
{
public:
    virtual ~GLDrawer() {}

    void draw( Circle const& circle ) const override;
    void draw( Square const& square ) const override;
};

int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;

    Shapes shapes;

    shapes.emplace_back( std::make_unique<Circle>( 2.0
                                                    , std::make_unique<GLDrawer>() ) );
    shapes.emplace_back( std::make_unique<Square>( 1.5
                                                    , std::make_unique<GLDrawer>() ) );
    shapes.emplace_back( std::make_unique<Circle>( 4.2
                                                    , std::make_unique<GLDrawer>() ) );

    drawAllShapes( shapes );
}
```


A Strategy-Based Solution

```
class Circle;
class Square;

class DrawStrategy
{
public:
    virtual ~DrawStrategy() {}

    virtual void draw( const Circle& circle ) const = 0;
    virtual void draw( const Square& square ) const = 0;
};

class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void translate( Vector2D const& ) = 0;
    virtual void rotate( double const& ) = 0;
    virtual void draw() const = 0;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad, std::unique_ptr<DrawStrategy> ds )
        : radius{ rad }
        // ...
};
```

A Strategy-Based Solution

```
template< typename ShapeT >
class DrawStrategy
{
public:
    virtual ~DrawStrategy() = default;

    virtual void draw( ShapeT const& shape ) const = 0;
};

class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void translate( Vector2D const& ) = 0;
    virtual void rotate( double const& ) = 0;
    virtual void draw() const = 0;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad, std::unique_ptr<DrawStrategy<Circle>> ds )
        : radius{ rad }
        , // ... Remaining data members
        , drawer{ std::move(ds) }
    {}
};
```

A Strategy-Based Solution

```
class Circle : public Shape
{
public:
    explicit Circle( double rad, std::unique_ptr<DrawStrategy<Circle>> ds )
        : radius{ rad }
        , // ... Remaining data members
        , drawer{ std::move(ds) }
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

    void translate( Vector2D const& ) override;
    void rotate( double const& ) override;
    void draw() const override;

    // ...

private:
    double radius;
    // ... Remaining data members
    std::unique_ptr<DrawStrategy<Circle>> drawer;
};
```

```
class Square : public Shape
{
public:
```

A Strategy-Based Solution

```
class Square : public Shape
{
public:
    explicit Square( double s, std::unique_ptr<DrawStrategy<Square>> ds )
        : side{ s }
        , // ... Remaining data members
        , drawer{ std::move(ds) }
    {}

    double getSide() const noexcept;
    // ... getCenter(), getRotation(), ...

    void translate( Vector2D const& ) override;
    void rotate( double const& ) override;
    void draw() const override;

    // ...

private:
    double side;
    // ... Remaining data members
    std::unique_ptr<DrawStrategy<Square>> drawer;
};

void drawAllShapes( std::vector<std::unique_ptr<Shape>> const& shapes )
{
    for( auto const& s : shapes )
```

A Strategy-Based Solution

```
}
```

```
class GLDrawer : public DrawStrategy<Circle>
                , public DrawStrategy<Square>
{
public:
    explicit GLDrawer( gl::Color color ) : color_{color} {}

    void draw( Circle const& circle ) const override;
    void draw( Square const& square ) const override;

private:
    gl::Color color_{};
};
```

```
int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( std::make_unique<Circle>( 2.3,
        std::make_unique<GLDrawer>(gl::Color::red) ) );
    shapes.emplace_back( std::make_unique<Square>( 1.2,
        std::make_unique<GLDrawer>(gl::Color::green) ) );
    shapes.emplace_back( std::make_unique<Circle>( 4.1,
        std::make_unique<GLDrawer>(gl::Color::blue) ) );
```

```
    // Drawing all shapes
```

A Strategy-Based Solution

```
int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( std::make_unique<Circle>( 2.3,
        std::make_unique<GLDrawer>(gl::Color::red) ) );
    shapes.emplace_back( std::make_unique<Square>( 1.2,
        std::make_unique<GLDrawer>(gl::Color::green) ) );
    shapes.emplace_back( std::make_unique<Circle>( 4.1,
        std::make_unique<GLDrawer>(gl::Color::blue) ) );

    // Drawing all shapes
    drawAllShapes( shapes );
}
```

The Classic Strategy Design Pattern

Task (C_Cpp_Software_Design/Strategy/Strategy_Refactoring):

Step 1: Refactor the classic Strategy solution by a value semantics based solution. Note that the general behavior should remain unchanged.

Step 2: Switch from one to another graphics library. Discuss the feasibility of the change: how easy is the change? How many pieces of code on which level of the architecture have to be touched?

Step 3: Add an `area()` operations for all shapes. How easy is the change? How many pieces of code on which level of the architecture have to be touched? Hint: the area of a circle is `radius*radius*M_PI`, the area of a square is `side*side`.

Step 4: Add the new feature to serialize shapes by means of the `FastSerialization` library. Observe how well the new code can be integrated and how many new dependencies are created.

`std::function`

- Is a wrapper around a callable with a specific signature
- The canonical example for value semantics
- Not intrusive!
- You can have local variables of type `std::function` (including function parameters) as well as members (not forced for them to be pointers or references)
- `std::function` may allocate as a fallback mechanism

std::function

```
#include <functional>

void foo(int i) {
    std::cout << "foo: " << i << '\n';
}

int main()
{
    std::function<void(int)> f{};

    f = [](int i){ std::cout << "lambda: " << i << '\n'; };

    auto g = f;    // Value semantics: Creates a deep copy

    f = foo;

    f( 1 );
    g( 2 );
}
```

std::function

Note that function is available in ...

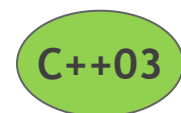
C++11:

```
#include <functional>
using Callback = std::function<void()>;
```



Boost:

```
#include <boost/function.hpp>
typedef boost::function<void()> Callback;
```



A “Modern C++” Solution

```
class Circle;
class Square;

using DrawCircleStrategy = std::function<void(Circle const&)>;
using DrawSquareStrategy = std::function<void(Square const&)>;

class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void translate( Vector2D const& ) = 0;
    virtual void rotate( double const& ) = 0;
    virtual void draw() const = 0;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad, DrawCircleStrategy ds )
        : radius{ rad }
        , // ... Remaining data members
        , drawer{ std::move(ds) }
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...
};
```

A “Modern C++” Solution

```
class Circle;
class Square;

using DrawCircleStrategy = std::function<void(Circle const&)>;
using DrawSquareStrategy = std::function<void(Square const&)>;

class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void translate( Vector2D const& ) = 0;
    virtual void rotate( double const& ) = 0;
    virtual void draw() const = 0;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad, DrawCircleStrategy ds )
        : radius{ rad }
        , // ... Remaining data members
        , drawer{ std::move(ds) }
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...
};
```

A “Modern C++” Solution

```
class Circle;
class Square;

using DrawCircleStrategy = std::function<void(Circle const&)>;
using DrawSquareStrategy = std::function<void(Square const&)>;

class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void translate( Vector2D const& ) = 0;
    virtual void rotate( double const& ) = 0;
    virtual void draw() const = 0;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad, DrawCircleStrategy ds )
        : radius{ rad }
        , // ... Remaining data members
        , drawer{ std::move(ds) }
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...
}
```

A “Modern C++” Solution

```
};
```

```
class Circle : public Shape
{
public:
    explicit Circle( double rad, DrawCircleStrategy ds )
        : radius{ rad }
        , // ... Remaining data members
        , drawer{ std::move(ds) }
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

    void translate( Vector2D const& ) override;
    void rotate( double const& ) override;
    void draw() const override;

    // ...

private:
    double radius;
    // ... Remaining data members
    DrawCircleStrategy drawer;
};
```

```
class Square : public Shape
{
public:
```

A “Modern C++” Solution

```
class Square : public Shape
{
public:
    explicit Square( double s, DrawSquareStrategy ds )
        : side{ s }
        , // ... Remaining data members
        , drawer{ std::move(ds) }
    {}

    double getSide() const noexcept;
    // ... getCenter(), getRotation(), ...

    void translate( Vector2D const& ) override;
    void rotate( double const& ) override;
    void draw() const override;

    // ...

private:
    double side;
    // ... Remaining data members
    DrawSquareStrategy drawer;
};

void drawAllShapes( std::vector<std::unique_ptr<Shape>> const& shapes )
{
    for( auto const& s : shapes )
    {
```

A “Modern C++” Solution

```
{
    for( auto const& s : shapes )
    {
        s->draw();
    }
}

class OpenGLCircleStrategy
{
public:
    void operator()( Circle const& circle ) const;
};

class OpenGLSquareStrategy
{
public:
    void operator()( Square const& square ) const;
};

int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( std::make_unique<Circle>( 2.0
                                                    , OpenGLCircleStrategy{} ) );
    shapes.emplace_back( std::make_unique<Square>( 1.5
                                                    , OpenGLSquareStrategy{} ) );
    shapes.emplace_back( std::make_unique<Circle>( 1.2
                                                    , OpenGLCircleStrategy{} ) );
    shapes.emplace_back( std::make_unique<Square>( 1.0
                                                    , OpenGLSquareStrategy{} ) );
}
```


A “Modern C++” Solution

```
{
    public:
        void operator()( Square const& square ) const;
};

int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( std::make_unique<Circle>( 2.0
                                                    , OpenGLCircleStrategy{} ) );
    shapes.emplace_back( std::make_unique<Square>( 1.5
                                                    , OpenGLSquareStrategy{} ) );
    shapes.emplace_back( std::make_unique<Circle>( 4.2
                                                    , OpenGLCircleStrategy{} ) );

    // Drawing all shapes
    drawAllShapes( shapes );
}
```

A “Modern C++” Solution

```
{
    for( auto const& s : shapes )
    {
        s->draw();
    }
}

class OpenGLCircleStrategy
{
public:
    void operator()( Circle const& circle ) const;
};

class OpenGLSquareStrategy
{
public:
    void operator()( Square const& square ) const;
};

int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( std::make_unique<Circle>( 2.0
                                                    , OpenGLCircleStrategy{} ) );
    shapes.emplace_back( std::make_unique<Square>( 1.5
                                                    , OpenGLSquareStrategy{} ) );
    shapes.emplace_back( std::make_unique<Circle>( 1.2
                                                    , OpenGLCircleStrategy{} ) );
    shapes.emplace_back( std::make_unique<Square>( 0.8
                                                    , OpenGLSquareStrategy{} ) );
}
```

A “Modern C++” Solution

```
{  
    for( auto const& s : shapes )  
    {  
        s->draw();  
    }  
}
```

```
void draw( Circle const& circle ) const;  
void draw( Square const& square ) const;
```

```
int main()  
{  
    using Shapes = std::vector<std::unique_ptr<Shape>>;  
  
    // Creating some shapes  
    Shapes shapes;  
    shapes.emplace_back( std::make_unique<Circle>( 2.0, draw ) );  
    shapes.emplace_back( std::make_unique<Square>( 1.5, draw ) );  
    shapes.emplace_back( std::make_unique<Circle>( 4.2, draw ) );  
  
    // Drawing all shapes  
    for( auto& s : shapes )  
        s->draw();  
}
```

A “Modern C++” Solution

```
{  
    for( auto const& s : shapes )  
    {  
        s->draw();  
    }  
}
```

```
void draw( Circle const& circle ) const;  
void draw( Square const& square ) const;
```

```
struct Draw  
{  
    template< typename T >  
    void operator()( T const& drawable ) const {  
        draw( drawable );  
    }  
};
```

```
int main()  
{  
    using Shapes = std::vector<std::unique_ptr<Shape>>;  
  
    // Creating some shapes  
    Shapes shapes;  
    shapes.emplace_back( std::make_unique<Circle>( 2.0, Draw{} ) );  
    shapes.emplace_back( std::make_unique<Square>( 1.5, Draw{} ) );  
    shapes.emplace_back( std::make_unique<Circle>( 4.2, Draw{} ) );  
  
    // Drawing all shapes  
    for( auto& s : shapes )  
        s->draw();  
}
```

A “Modern C++” Solution

```
{
    template< typename T >
    void operator()( T const& drawable ) const {
        draw( drawable );
    }
};

int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( std::make_unique<Circle>( 2.0, Draw{} ) );
    shapes.emplace_back( std::make_unique<Square>( 1.5, Draw{} ) );
    shapes.emplace_back( std::make_unique<Circle>( 4.2, Draw{} ) );

    // Drawing all shapes
    drawAllShapes( shapes );
}
```

A “Modern C++” Solution

```
{  
    for( auto const& s : shapes )  
    {  
        s->draw();  
    }  
}
```

```
void draw( Circle const& circle ) const;  
void draw( Square const& square ) const;
```

```
struct Draw  
{  
    template< typename T >  
    void operator()( T const& drawable ) const {  
        draw( drawable );  
    }  
};
```

```
int main()  
{  
    using Shapes = std::vector<std::unique_ptr<Shape>>;  
  
    // Creating some shapes  
    Shapes shapes;  
    shapes.emplace_back( std::make_unique<Circle>( 2.0, Draw{} ) );  
    shapes.emplace_back( std::make_unique<Square>( 1.5, Draw{} ) );  
    shapes.emplace_back( std::make_unique<Circle>( 4.2, Draw{} ) );  
  
    // Drawing all shapes  
    for( auto& s : shapes )  
        s->draw();  
}
```

A “Modern C++” Solution

```
{  
    for( auto const& s : shapes )  
    {  
        s->draw();  
    }  
}
```

```
void draw( Circle const& circle ) const;  
void draw( Square const& square ) const;
```

```
int main()  
{  
    using Shapes = std::vector<std::unique_ptr<Shape>>;  
  
    // Creating some shapes  
    Shapes shapes;  
    shapes.emplace_back( std::make_unique<Circle>( 2.0, Draw{} ) );  
    shapes.emplace_back( std::make_unique<Square>( 1.5, Draw{} ) );  
    shapes.emplace_back( std::make_unique<Circle>( 4.2, Draw{} ) );  
  
    // Drawing all shapes  
    for( auto& s : shapes )  
        s->draw();  
}
```

A “Modern C++” Solution

```
class Circle;
class Square;

using DrawCircleStrategy = std::function<void(Circle const&)>;
using DrawSquareStrategy = std::function<void(Square const&)>;

struct Draw
{
    template< typename T >
    void operator()( T const& drawable ) const {
        draw( drawable );
    }
};

class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;

    virtual void translate( Vector2D const& ) = 0;
    virtual void rotate( double const& ) = 0;
    virtual void draw() const = 0;
};

class Circle : public Shape
{
public:
    explicit Circle( double rad, DrawCircleStrategy ds = Draw{} )
```


A “Modern C++” Solution

```
class Circle : public Shape
{
public:
    explicit Circle( double rad, DrawCircleStrategy ds = Draw{} )
        : radius{ rad }
        , // ... Remaining data members
        , drawer{ std::move(ds) }
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

    void translate( Vector2D const& ) override;
    void rotate( double const& ) override;
    void draw() const override;

    // ...

private:
    double radius;
    // ... Remaining data members
    DrawCircleStrategy drawer;
};
```

```
class Square : public Shape
{
public:
```

A “Modern C++” Solution

```
class Square : public Shape
{
public:
    explicit Square( double s, DrawSquareStrategy ds = Draw{} )
        : side{ s }
        , // ... Remaining data members
        , drawer{ std::move(ds) }
    {}

    double getSide() const noexcept;
    // ... getCenter(), getRotation(), ...

    void translate( Vector2D const& ) override;
    void rotate( double const& ) override;
    void draw() const override;

    // ...

private:
    double side;
    // ... Remaining data members
    DrawSquareStrategy drawer;
};

void drawAllShapes( std::vector<std::unique_ptr<Shape>> const& shapes )
{
    for( auto const& s : shapes )
    {
```

A “Modern C++” Solution

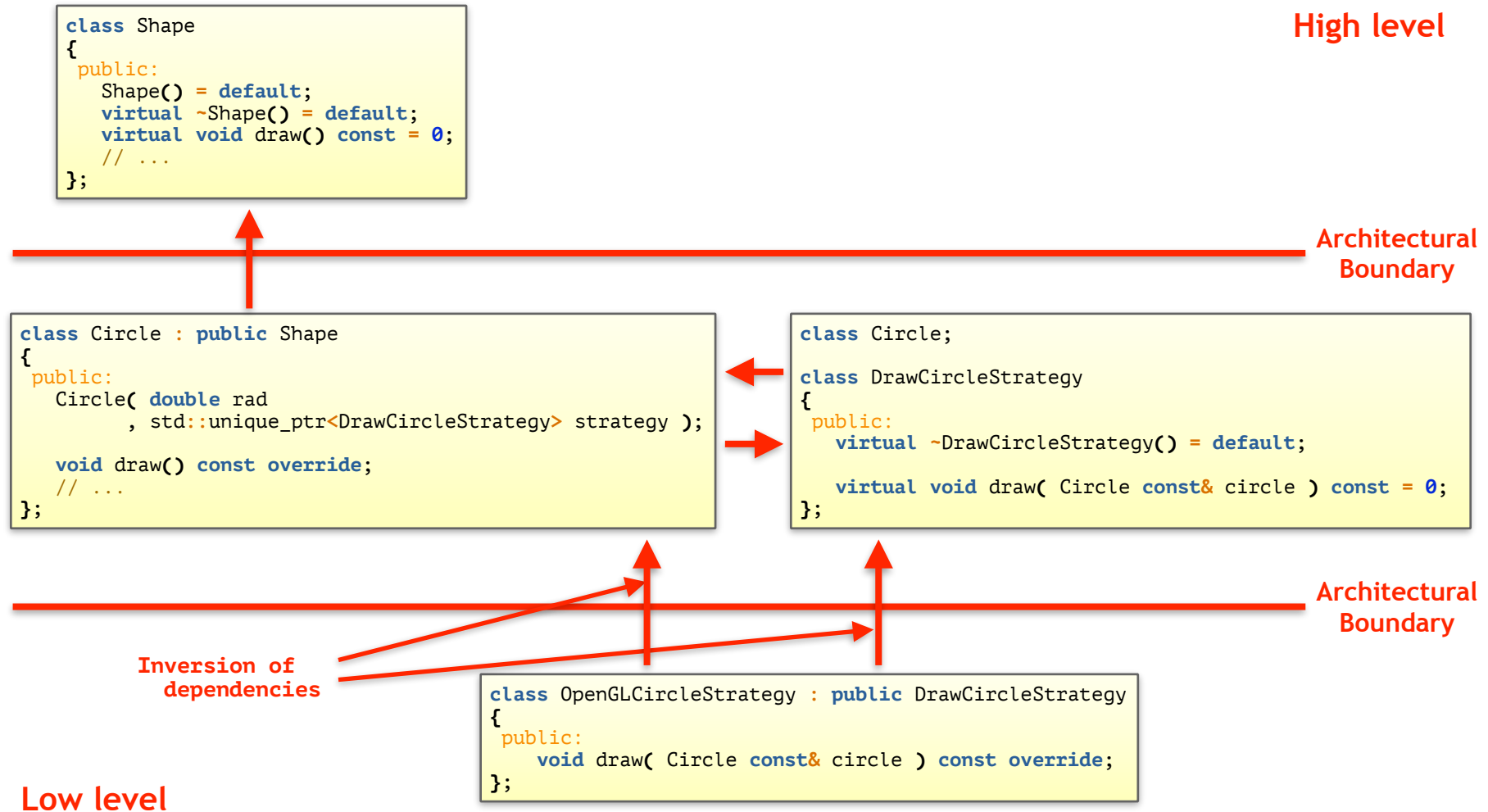
```
void draw( Circle const& circle ) const;
void draw( Square const& square ) const;

int main()
{
    using Shapes = std::vector<std::unique_ptr<Shape>>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( std::make_unique<Circle>( 2.0 ) );
    shapes.emplace_back( std::make_unique<Square>( 1.5 ) );
    shapes.emplace_back( std::make_unique<Circle>( 4.2 ) );

    // Drawing all shapes
    drawAllShapes( shapes );
}
```

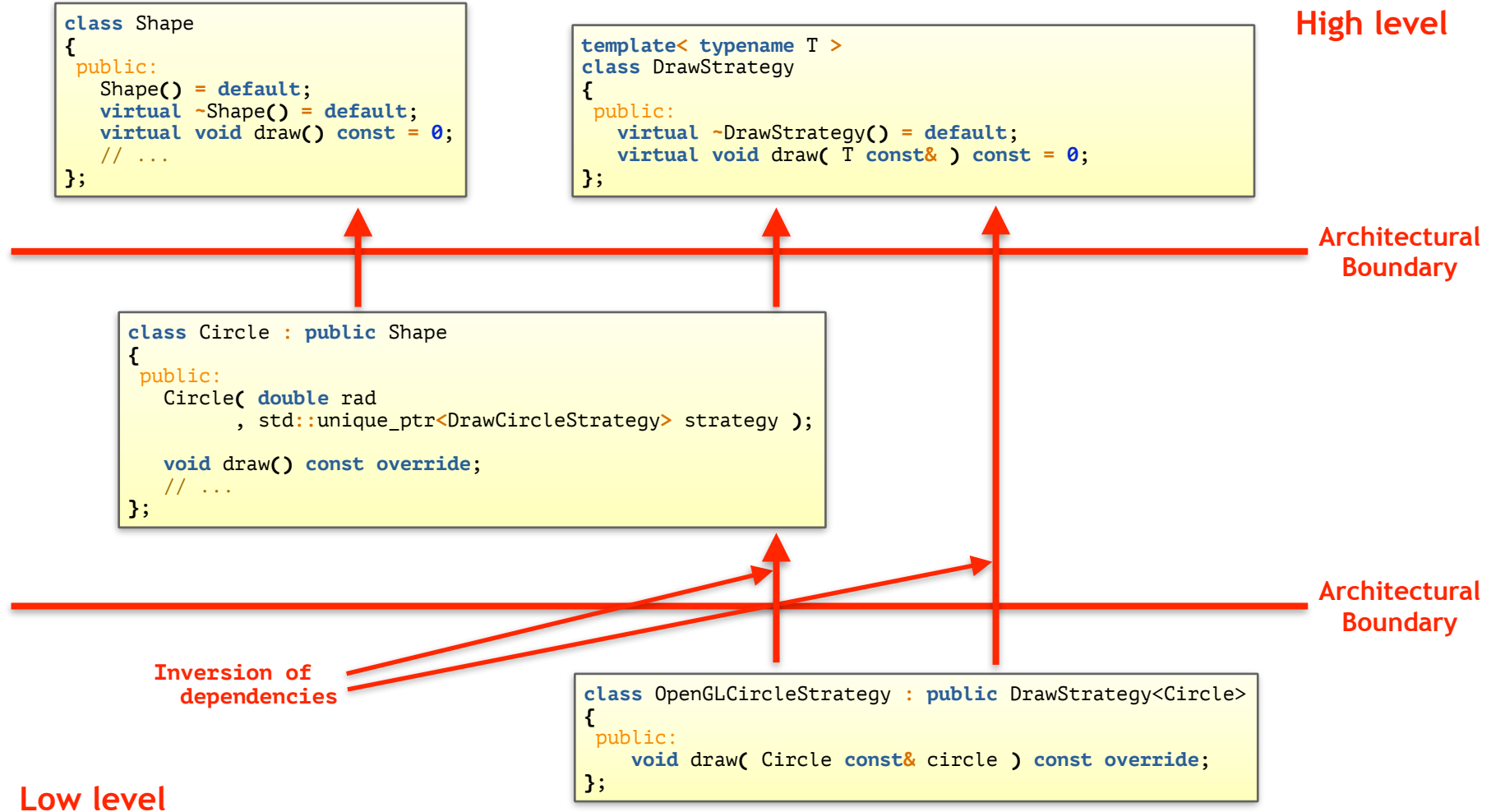
Dependency Structure



Dependency Structure

```
//---- <DrawStrategy.h> -----  
  
template< typename T >  
class DrawStrategy  
{  
public:  
    virtual ~DrawStrategy() = default;  
    virtual void draw( T const& circle ) const = 0;  
};  
  
//---- <OpenGLCircleStrategy.h> -----  
  
#include <Circle.h>  
#include <DrawStrategy.h>  
// ... Potentially some OpenGL-related header  
  
class OpenGLCircleStrategy : public DrawStrategy<Circle>  
{  
public:  
    void draw( Circle const& circle ) const override;  
};
```

Dependency Structure



Dependency Structure

```
class Shape
{
public:
    Shape() = default;
    virtual ~Shape() = default;
    virtual void draw() const = 0;
    // ...
};
```

High level

Architectural
Boundary

```
class Circle : public Shape
{
public:
    Circle( double rad
           , std::function<void(Circle const&> strategy );

    void draw() const override;
    // ...
};
```

Automatic inversion
of dependencies

Architectural
Boundary

```
class OpenGLCircleStrategy
{
public:
    void operator()(Circle const& circle) const;
};
```

Low level

The Classic Strategy Design Pattern

The classic Strategy design pattern ...

- ... requires a base class (dependency);
- ... promotes heap allocation;
- ... requires memory management.

Using `std::function` instead of the classic Strategy design pattern ...

- ... simplifies code;
- ... facilitates comprehension;
- ... reduces dependencies.

std::function — Advantages/Disadvantages

Use `std::function` if ...

- ... you want to abstract and to **decouple a single function**;
- ... performance is not the **primary concern**.

Don't use `std::function` if ...

- ... you need to **decouple several cohesive functions**;
- ... you require **maximum performance**.

The Classic Strategy Design Pattern

Task (C_Cpp_Software_Design/Strategy/MultipleStrategies):

Step 1: Consider the following shape classes with `draw()`, `serialize()` and `print()` functions. How would you externalize the implementation details for all three functions? Think about solutions for both dynamic and static polymorphism and discuss pros and cons.

Step 2: Think about adding a new types (e.g. 'Rectangle', different implementations of either the `draw()`, `serialize()` or `print()` functions). How much effort would this be? How many places would have to be touched and how would this relate to the OCP?

Step 3: Think about adding a new operation. How much effort would this be? How many places would have to be touched and how would this relate to the OCP?

The Classic Strategy Design Pattern

Task (C_Cpp_Software_Design/Strategy/UniquePtr_Strategy): Replace the hard-coded call to `delete()` with a template-based Strategy implementation.

The Classic Strategy Design Pattern

Task (C_Cpp_Software_Design/Strategy/FixedVector_Strategy):

Step 1: Replace the hard-coded logging mechanism by means of a template-based Strategy implementation. Demonstrate how this helps to invert the dependencies.

Step 2: Replace the hard-coded error handling mechanism (i.e. `assert()`) by means of a template-based Strategy implementation.

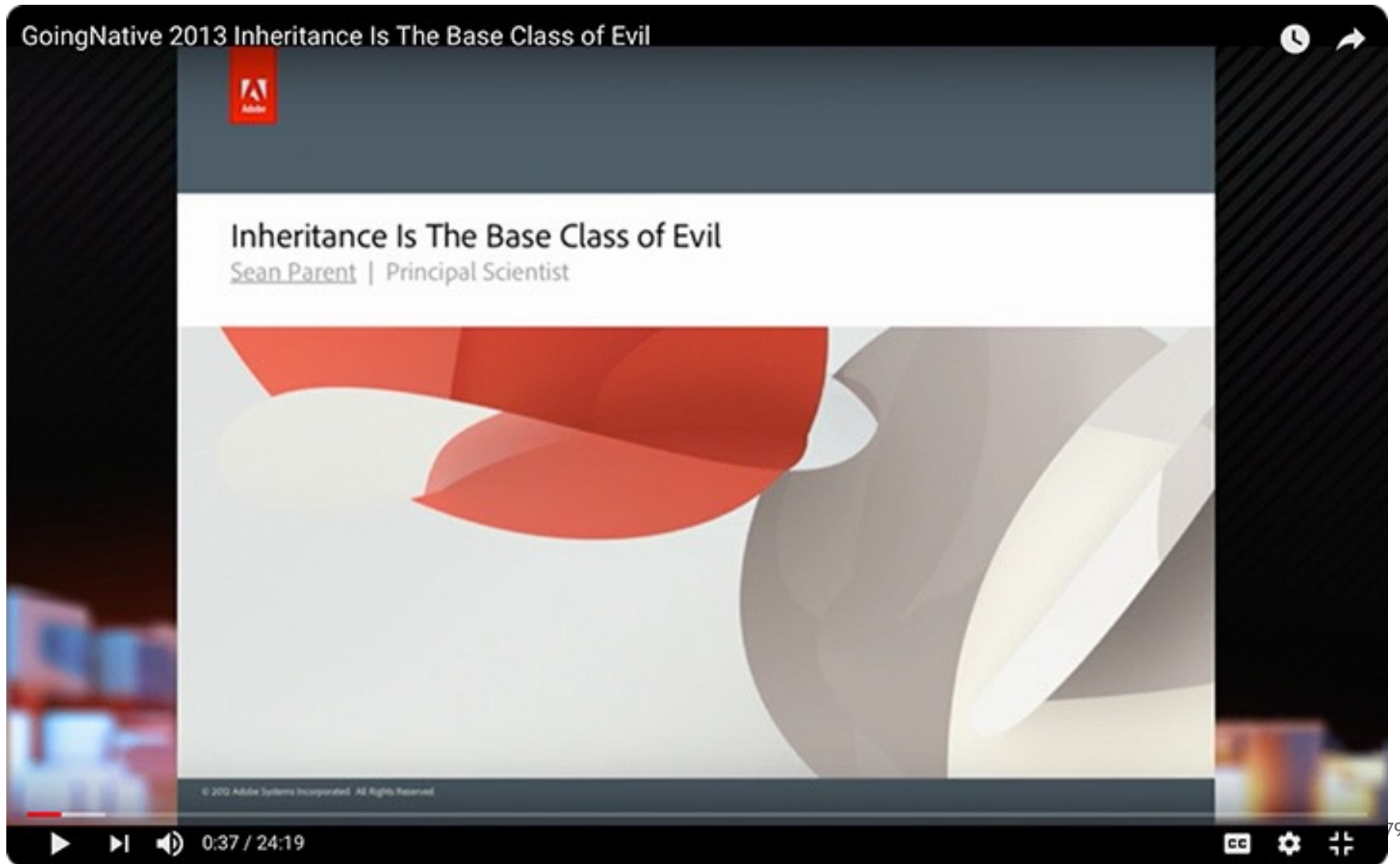
Guidelines

Guideline: Use the Strategy pattern to encapsulate interchangeable behaviours and use delegation to decide which behavior to use.

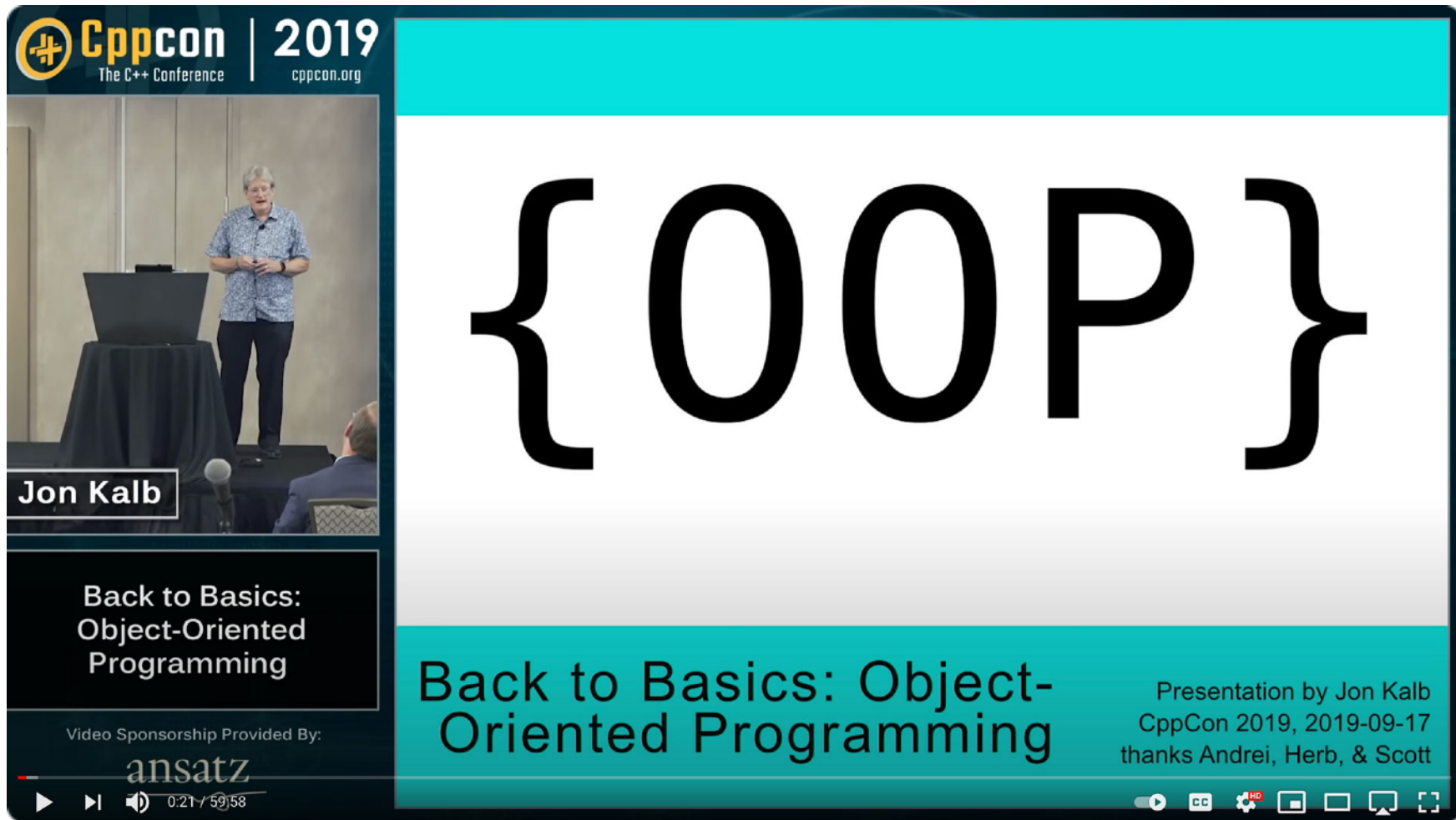
Guideline: Prefer containment (composition/aggregation) to inheritance.

2.6. Interlude 2: Value Semantics

The Expert's Attitude



The Expert's Attitude



The image shows a video player interface for a presentation. The top left corner features the Cppcon 2019 logo with the text "The C++ Conference" and "cppcon.org". The main video area shows a man, Jon Kalb, standing on a stage next to a podium. A large, stylized "{ O O P }" is displayed on the screen behind him. Below the video, the title "Back to Basics: Object-Oriented Programming" is shown. The bottom left corner includes a video player control bar with a play button, a progress bar, and the text "0:21 / 59:58". The bottom right corner contains the text "Presentation by Jon Kalb CppCon 2019, 2019-09-17 thanks Andrei, Herb, & Scott".

Cppcon | 2019
The C++ Conference | cppcon.org

Jon Kalb

Back to Basics:
Object-Oriented
Programming

Video Sponsorship Provided By:
ansatz

Back to Basics: Object-
Oriented Programming

Presentation by Jon Kalb
CppCon 2019, 2019-09-17
thanks Andrei, Herb, & Scott

0:21 / 59:58

The Expert's Attitude

Cppcon | 2019
The C++ Conference | cppcon.org



Jon Kalb

Back to Basics:
Object-Oriented
Programming

Video Sponsorship Provided By:
ansatz

1:27 / 59:58

THE NEW STACK Ebooks Podcasts Events Newsletter

Architecture Development Operations

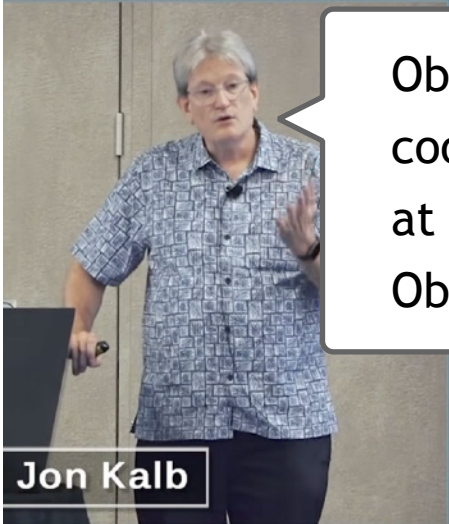
CULTURE / DEVELOPMENT

Why Are So Many Developers Hating on Object-Oriented Programming?

21 Aug 2019 12:00pm, by David Cassel



The Expert's Attitude



Jon Kalb

Cppcon | 2019
The C++ Conference
cppcon.org

Object-oriented programming is not what the cool kids are doing in C++. They are doing things at compile time, functional programming, ...
Object-oriented programming, this is so 90s ...

Back to Basics:
Object-Oriented
Programming

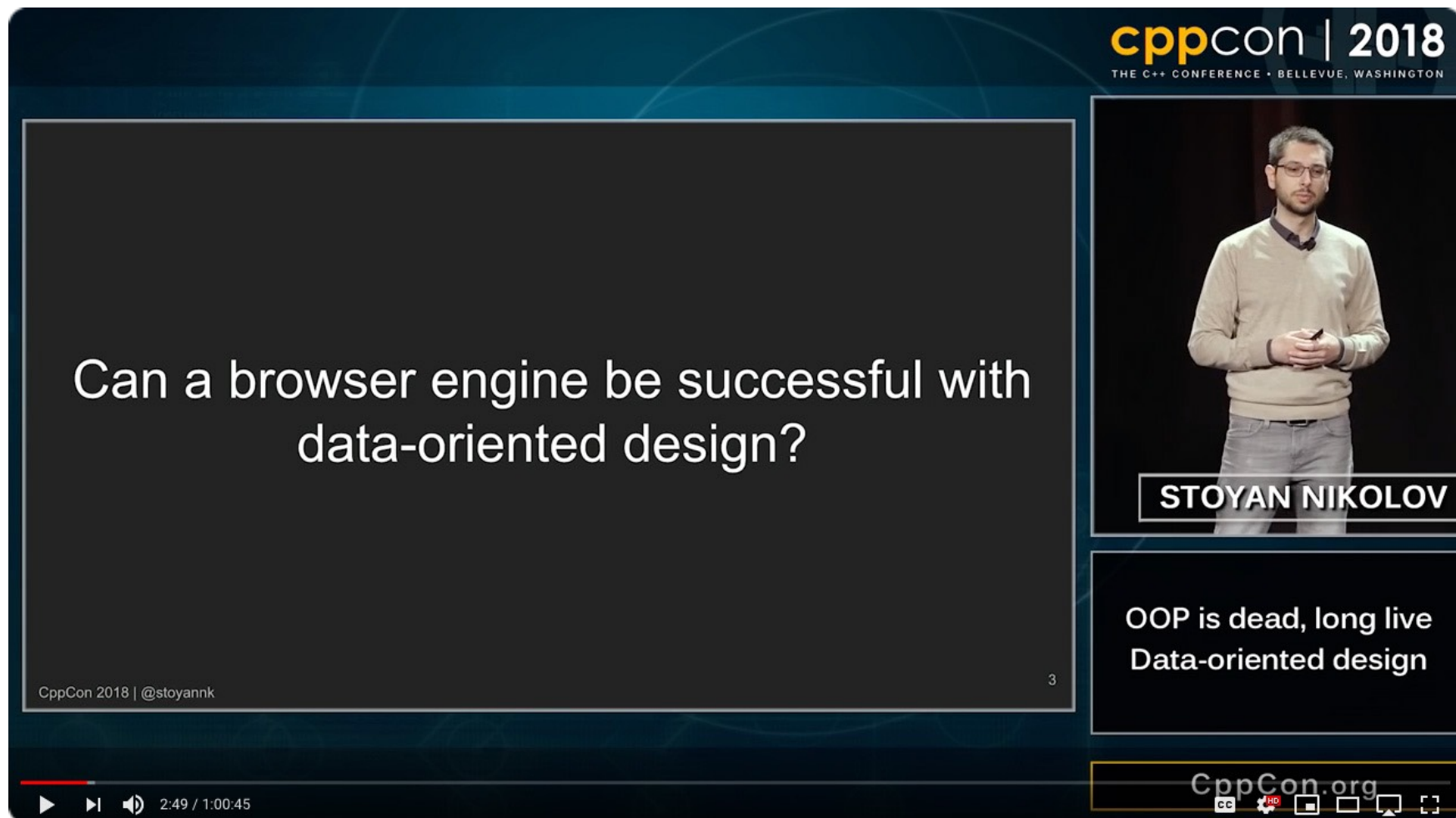
Video Sponsorship Provided By:
ansatz



1:27 / 59:58

2

The Expert's Attitude



The screenshot shows a video player interface for a CppCon 2018 presentation. The main slide area is dark with white text asking "Can a browser engine be successful with data-oriented design?". To the right, a smaller video window shows the speaker, Stoyan Nikolov, a man with glasses wearing a light-colored sweater. Below the speaker's name, a quote reads "OOP is dead, long live Data-oriented design". The video player controls at the bottom show a progress bar at 2:49 / 1:00:45. The CppCon.org logo and social media icons are in the bottom right corner.

cppcon | 2018
THE C++ CONFERENCE • BELLEVUE, WASHINGTON

Can a browser engine be successful with data-oriented design?

STOYAN NIKOLOV

OOP is dead, long live Data-oriented design

CppCon 2018 | @stoyannk 3

2:49 / 1:00:45

CppCon.org

The Expert's Attitude



"... [Programming by difference] fell out of favor in the 1990s when many people in the OO community noticed that inheritance can be rather problematic if it is overused."

(Michael C. Feathers, Working Effectively with Legacy Code)

The Expert's Attitude

- Why Are So Many Developers Hating on Object-Oriented Programming (David Cassel) (<https://thenewstack.io/why-are-so-many-developers-hating-on-object-oriented-programming/>)
- The Forgotten History of OOP (Eric Elliott) (<https://medium.com/javascript-scene/the-forgotten-history-of-oop-88d71b9b2d9f>)
- If everyone hates it, why is OOP still so widely spread? (Medi Madelen Gwosdz) (<https://stackoverflow.blog/2020/09/02/if-everyone-hates-it-why-is-oop-still-so-widely-spread/>)
- Uncle Bob SOLID principles (<https://youtu.be/QHnLmvDxGTY?t=2264>)

Why is it so bad?

- Does not harmonize with the philosophy of the STL (value semantics)
- Fails to model many sub typing relations (i.e. it is hard)

Overuse of Inheritance

Task: Implement the `max()` algorithm by means of inheritance.

```
class Comparable
{
public:
    virtual ~Comparable() = default;
    virtual bool lessThan( const Comparable& other ) = 0;
    // ...
};

// ...

const Comparable& max( const Comparable& a, const Comparable& b );
```

- Types like `int` and `std::string` cannot be compared
- The interface allows to compare different types
 - How should we deal with the comparison of different types?
- For every comparison we have to call a virtual function
- Do we use a `dynamic_cast`?

Overuse of Inheritance

```
template< typename T >  
const T& max( const T& a, const T& b );
```

- Any types can be compared (even `int` and `std::string`)
- The interface allows to compare only values of the same type
 - No error handling for comparing different types
- No virtual function call, no `dynamic_cast`, but inlining

Guidelines

Guideline: Remember that inheritance is about behaviour, not about data!

Why is it so bad?

- Does not harmonize with the philosophy of the STL
- Fails to model many sub typing relations (i.e. it is hard)
- Inheritance creates a very tight coupling (second only to friendship)
 - Intrusive: Forces us to inherit, or ...
 - Verbose: Forces us to wrap perfectly good types to conform to a hierarchy
- Adding functions requires modifications (violation of the OCP)
 - May cause contradictions between SRP and OCP
- Inheritance introduces overhead:
 - Heap allocation (memory management), leads to pointers (nullptr, dangling pointers, ...)

Dynamic Allocation

```
class Animal { virtual ~Animal() = default; };
```

```
class Cat : public Animal { /*...*/ };
```

```
class Dog : public Animal { /*...*/ };
```

```
Animal make_animal();
```

```
std::vector<Animal> v{};
```

Dynamic Allocation

```
class Animal { virtual ~Animal() = default; };
```

```
class Cat : public Animal { /*...*/ };
```

```
class Dog : public Animal { /*...*/ };
```

```
Animal make_animal();
```

```
std::vector<Animal> v{};
```

Dynamic Allocation

```
class Animal { virtual ~Animal() = default; };
```

```
class Cat : public Animal { /*...*/ };
```

```
class Dog : public Animal { /*...*/ };
```

```
std::unique_ptr<Animal> make_animal();
```

```
std::vector<std::unique_ptr<Animal>> v{};
```

Why is it so bad?

- Does not harmonize with the philosophy of the STL
- Fails to model many sub typing relations (i.e. it is hard)
- Inheritance creates a very tight coupling (second only to friendship)
 - Intrusive: Forces us to inherit, or ...
 - Verbose: Forces us to wrap perfectly good types to conform to a hierarchy
- Adding functions requires modifications (violation of the OCP)
 - May cause contradictions between SRP and OCP
- Inheritance introduces overhead:
 - Heap allocation (memory management), leads to pointers (nullptr, dangling pointers)
 - Virtual functions (no inlining)

Virtual Dispatch

```
struct Shape {  
    void translate();  
    virtual void rotate();  
    // ...  
    __VTable__* __vptr__;  
};
```

```
struct Circle : public Shape {  
    void translate();  
    void rotate() override;  
    // ...  
};
```

```
int main()  
{  
    Circle circle{};  
    Shape& shape = derived;  
    shape.rotate();  
    circle.rotate();  
}
```

__vtable__
// ...
void (*rotate)() const
// ...
// ...

Why is it so bad?

- Does not harmonize with the philosophy of the STL
- Fails to model many sub typing relations (i.e. it is hard)
- Inheritance creates a very tight coupling (second only to friendship)
 - Intrusive: Forces us to inherit, or ...
 - Verbose: Forces us to wrap perfectly good types to conform to a hierarchy
- Adding functions requires modifications (violation of the OCP)
 - May cause contradictions between SRP and OCP
- Inheritance introduces overhead:
 - Heap allocation (memory management), leads to pointers (nullptr, dangling pointers)
 - Virtual functions (no inlining)
 - RTTI

Why is it so bad?

- ...
- Forces us to use pointers and references
 - One extra indirection (runtime overhead)
 - Allocations: memory fragmentation, synchronisation, may fail
 - Shallow vs. deep copy
 - Disables local reasoning
 - Incentives sharing, which complicates lifetime management

2. C++ Software Design - Interlude 2: Value Semantics



Value Types

```
class A
{
    public:
        A( int m, int c ) : mult{m}, offset{c} {}

        int foo( int x ) const
        {
            return mult * x + offset;
        }

    private:
        int mult;
        int offset;
};
```

Value Types

- Automatic Memory Management (no garbage collection required)
- Exception-safe
- Does not change
- No side effects
- Can use the same value in multiple threads (lock-free)
- Deterministic
- Pure
- Regular

→ Value Semantics

Pointers and References

```
class A
{
public:
    A( int const& m, int* c ) : mult{m}, offset{c} {}

    int foo( int x ) const
    {
        return mult * x + *offset;
    }

private:
    int const& mult;
    int* offset;
};
```

Pointers and References

- Do I have to delete offset?
- Does someone else have write access to mult and offset?
- Do I need a mutex?
- Is it deterministic?
- It is a bug hive

→ Reference Semantics

Custom Types

```
class A
{
    public:
        A( Mult m, Offset c ) : mult{m}, offset{c} {}

        int foo( int x ) const
        {
            return mult * x + offset;
        }

    private:
        Mult mult;
        Offset offset;
};
```

Custom Types

- Do `Mult` and `Offset` behave like `int`? → Good!
- Do `Mult` and `Offset` behave like `int*` or `int&`? → Bad!

Progressive C++

Meeting C++ 2023

Prog C++ - Ivan Čukić

think-cell 

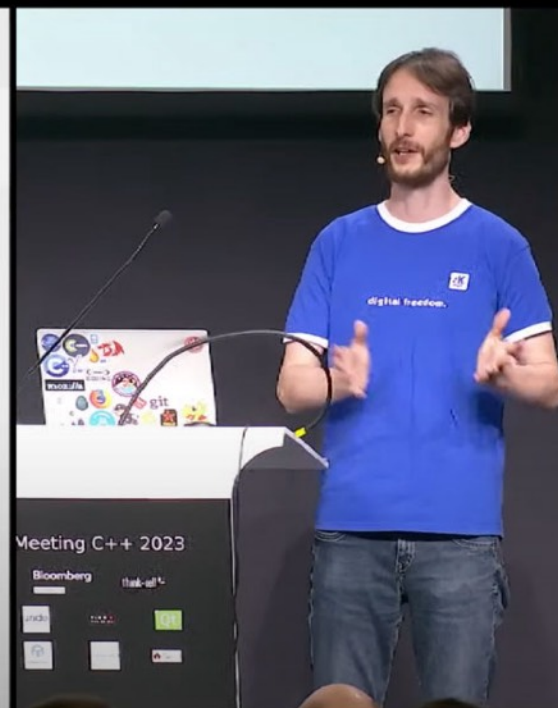
PROG C++

Prog C++ is a broad genre of C++ code. The style was an emergence of psychedelic developers who abandoned standard C with classes traditions in favour of instrumentation and compositional techniques more frequently associated with generic, functional, or value-based object oriented coding practices.

INTRODUCTION
WRAPS
SWAPS
STATES
ERRORS
VALUES
SAFETY

Meeting C++ 2023

Ivan Čukić kdab.com, cukic.co



4:21 / 1:50:35



Guidelines

Guideline: Prefer value semantics over reference semantics!

Guideline: Try to reduce the use of pointers!

2.7. External Polymorphism

External Polymorphism

[External Polymorphism \(3rd Pattern Languages of Programming Conference, September 4-6, 1996\)](#)

External Polymorphism

An Object Structural Pattern for Transparently Extending C++ Concrete Data Types

Chris Cleeland

chris@envision.com

Envision Solutions, St. Louis, MO 63141

Douglas C. Schmidt and Timothy H. Harrison

schmidt@cs.wustl.edu and harrison@cs.wustl.edu

Department of Computer Science

Washington University, St. Louis, Missouri, 63130

This paper appeared in the Proceedings of the 3rd Pattern Languages of Programming Conference, Allerton Park, Illinois, September 4–6, 1996.

1 Intent

Allow C++ classes unrelated by inheritance and/or having no virtual methods to be treated polymorphically. These unrelated classes can be treated in a common manner by software that uses them.

2 Motivation

Working with C++ classes from different sources can be dif-

1. *Space efficiency* – The solution must not constrain the storage layout of existing objects. In particular, classes that have no virtual methods (*i.e.*, concrete data types) must not be forced to add a virtual table pointer.
2. *Polymorphism* – All library objects must be accessed in a uniform, transparent manner. In particular, if new classes are included into the system, we won't want to change existing code.

Consider the following example using classes from the ACE network programming framework [3]:

```
1. SOCK_Acceptor acceptor; // Global storage
2.
```

The Intent

External Polymorphism

An Object Structural Pattern for Transparently Extending C++ Concrete Data Types

Chris Cleeland

chris@envision.com

Envision Solutions, St. Louis, MO 63141

Douglas C. Schmidt and Timothy H. Harrison

schmidt@cs.wustl.edu and harrison@cs.wustl.edu

Department of Computer Science

Washington University, St. Louis, Missouri, 63130

This paper appeared in the Proceedings of the 3rd Pattern Languages of Programming Conference, Allerton Park, Illinois, September 4–6, 1996.

1. *Space efficiency* – The solution must not constrain the storage layout of existing objects. In particular, classes that have no virtual methods (*i.e.*, concrete data types) must not be forced to add a virtual table pointer.

2. *Polymorphism* – All library objects must be accessible to user applications. If existing classes are included into the system, we won't want to change existing code.

Consider the following example using classes from the ACE network programming framework [3]:

```
1. #include <ACE_SOCK_Stream.h>
2. #include <ACE_SOCK_Acceptor.h>
3. int main (void) {
4.     SOCK_Stream stream; // Automatic storage
5.     INET_Addr addr = ...
6.     new INET_Addr(addr, port_);
```

The SOCK_Stream, SOCK_Acceptor, and INET_Addr classes are all concrete data types since they don't all inherit from a common ancestor and/or they don't contain virtual functions. If during a debugging session an application wanted to examine the state of all live ACE objects at line 7, we might get the following output:

```
SOCK_Stream::this = 0x47c393ab, handle_ = {-1}
SOCK_Acceptor::this = 0x2c49a45b, handle_ = {-1}
INET_Addr::this = 0x3c48a432,
port_ = {0}, addr_ = {0.0.0.0}
```

An effective way to project the capability to dump state onto these class without modifying their binary layout is to use the *External Polymorphism pattern*. This pattern constructs a parallel, external inheritance hierarchy that projects polymorphic behavior onto a set of concrete class that need not be related by inheritance. The following OMT diagram illustrates how the External Polymorphism pattern can be used to create the external, parallel hierarchy of classes:

”Allow C++ classes unrelated by inheritance and/or having no virtual methods to be treated polymorphically. These unrelated classes can be treated in a common manner by software that uses them.”
(Cleeland, Schmidt and Harrison, External Polymorphism)

1. Intent

Allow C++ classes unrelated by inheritance and/or having no virtual methods to be treated polymorphically. These unrelated classes can be treated in a common manner by software that uses them.

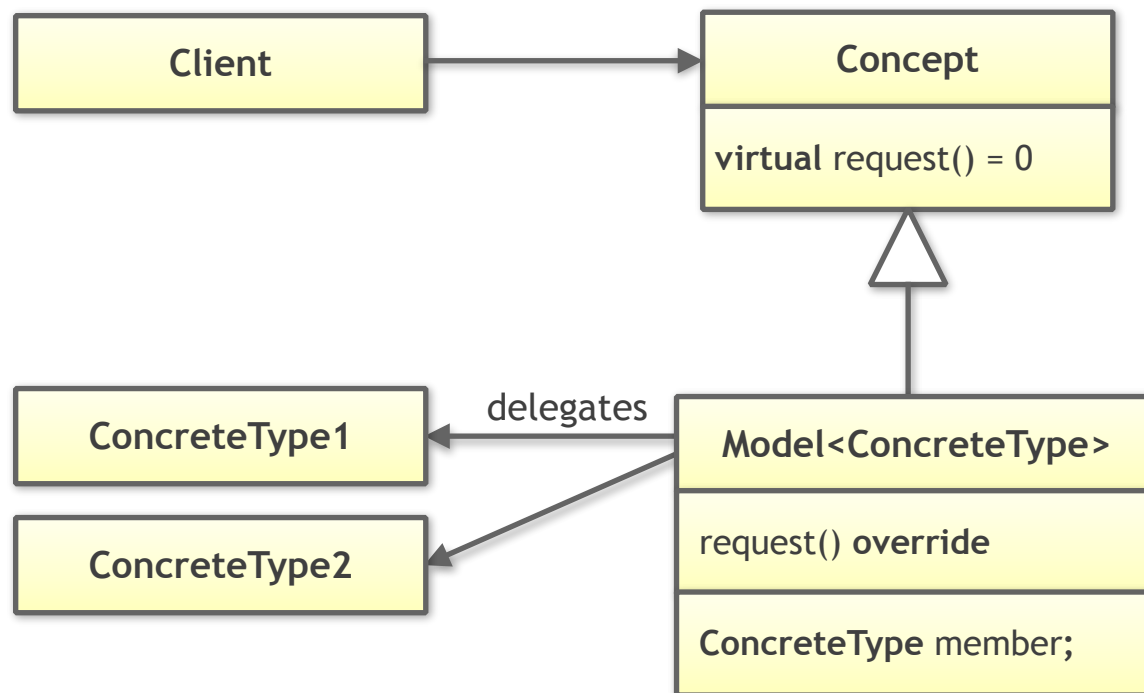
2. Motivation

Working with C++ classes from different sources can be difficult. Often an application may wish to “project” common behavior on such classes, but is restricted by the classes’ behavior. One solution is to implement an object structural pattern such as Adapter or Decorator [1]. Occasionally there are more complex requirements, such as the need to change both underlying interface and implementation. In such cases, classes may need to behave as if they had a common ancestor.

For instance, consider the case where we are debugging an application constructed using classes from various C++ libraries. It would be convenient to be able to ask any instance to “dump” its state in a human-readable format to a file or console display. It would be even more convenient to gather all live class instances into a collection and iterate over that collection asking each instance to dump itself.

Since collections are homogeneous, a common base class must exist to maintain a single collection. Since classes are already designed, implemented and in use, however, modifying the inheritance tree to introduce a common base class is not an option – we may not have access to the source, either! In addition, classes in OO languages like C++ may be *concrete data types* [2], which require strict storage layouts that could be compromised by hidden pointers (such as C++’s virtual table pointer). Re-implementing these classes with a common, polymorphic, base class is not feasible.

The External Polymorphism Design Pattern

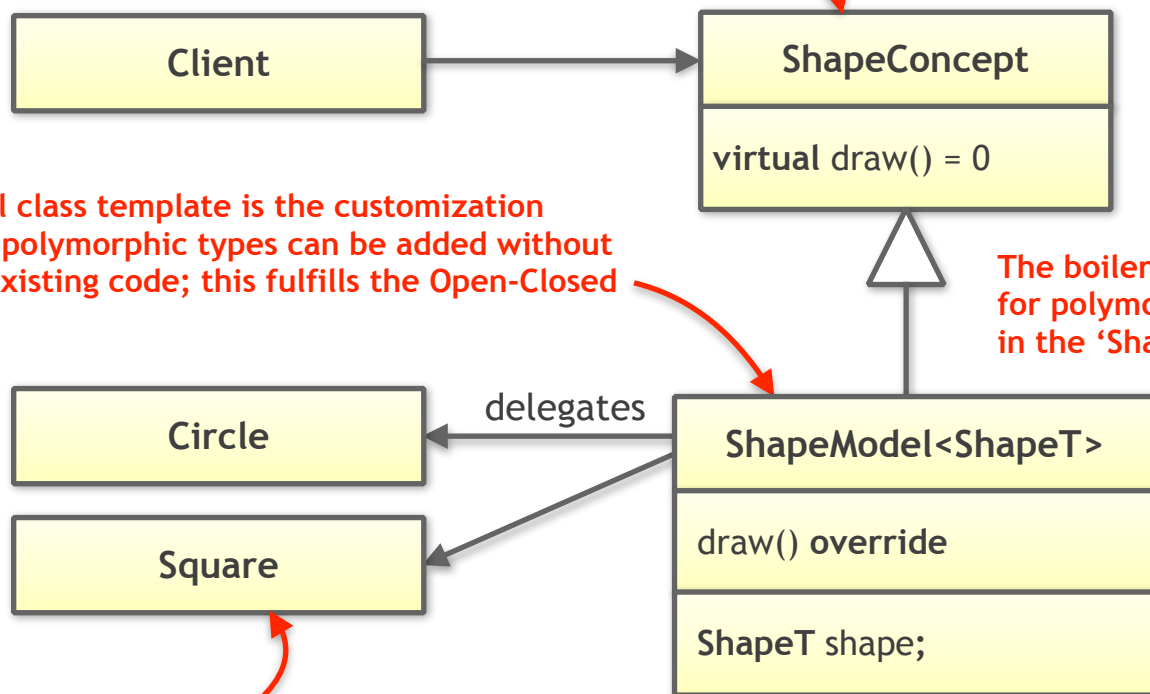


The External Polymorphism Design Pattern

The addition of polymorphic behavior is the variation point: This aspect changes and is extracted and isolated; this fulfills the Single-Responsibility Principle (SRP)

The ShapeModel class template is the customization point: new non-polymorphic types can be added without modifying any existing code; this fulfills the Open-Closed Principle (OCP)

The boilerplate code required for polymorphic behavior is unified in the 'ShapeModel' class (DRY)



All shapes types are non-polymorphic (i.e. neither have a base class nor any virtual function) and don't know about drawer, etc.

External Polymorphism

Task (C_Cpp_Software_Design/External_Polymorphism/Animal_EP):
Create an external hierarchy for animals that represents the polymorphic behavior of animals.

External Polymorphism

Task (C_Cpp_Software_Design/External_Polymorphism/ExternalPolymorphism_1):

Step 1: Refactor the given Shape hierarchy by means of the External Polymorphism design pattern to extract the `draw()` operation from shapes.

Step 2: Switch from one to another graphics library. Discuss the feasibility of the change: how easy is the change? How many pieces of code on which level of the architecture have to be touched?

Step 3: Add the new feature to serialize shapes by means of the `FastSerialization` library. Observe how well the new code can be integrated and how many new dependencies are created.

External Polymorphism

Task (C_Cpp_Software_Design/External_Polymorphism/ExternalPolymorphism_2):

Step 1: Refactor the given Strategy-based solution and extract the polymorphic behavior of all shapes by means of the External Polymorphism design pattern. Note that the general behavior should remain unchanged.

Step 2: Switch from one to another graphics library. Discuss the feasibility of the change: how easy is the change? How many pieces of code on which level of the architecture have to be touched?

Step 3: Add the new feature to serialize shapes by means of the FastSerialization library. Observe how well the new code can be integrated and how many new dependencies are created.

A Solution based on External Polymorphism

```
class Circle
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
        {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

private:
    double radius;
    // ... Remaining data members
};
```

```
class Square
{
public:
    explicit Square( double s )
        : side{ s }
        , // ... Remaining data members
        {}

    double getSide() const noexcept;
    // ... getCenter(), getRotation(), ...
```

A Solution based on External Polymorphism

```
class Circle
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

private:
    double radius;
    // ... Remaining data members
};
```

```
class Square
{
public:
    explicit Square( double s )
        : side{ s }
        , // ... Remaining data members
    {}

    double getSide() const noexcept;
    // ... getCenter(), getRotation(), ...
```

A Solution based on External Polymorphism

```
private:
    double radius;
    // ... Remaining data members
};

class Square
{
public:
    explicit Square( double s )
        : side{ s }
        , // ... Remaining data members
    {}

    double getSide() const noexcept;
    // ... getCenter(), getRotation(), ...

private:
    double side;
    // ... Remaining data members
};

class ShapeConcept
{
public:
    virtual ~ShapeConcept() = default;

    virtual void draw() const = 0;
};
```

A Solution based on External Polymorphism

```
class ShapeConcept
{
public:
    virtual ~ShapeConcept() = default;

    virtual void draw() const = 0;
};

template< typename ShapeT >
class ShapeModel : public ShapeConcept
{
public:
    using DrawStrategy = std::function<void(ShapeT const&)>;

    ShapeModel( ShapeT const& shape, DrawStrategy strategy )
        : shape_{ shape }
        , strategy_{ std::move(strategy) }
    {
        if( !strategy_ ) {
            throw std::invalid_argument( "Invalid draw strategy" );
        }
    }

    void draw() const override { strategy_(shape_); }

private:
    ShapeT shape_;
    DrawStrategy strategy_;
```

A Solution based on External Polymorphism

```
{
    public:
        virtual ~ShapeConcept() = default;

        virtual void draw() const = 0;
};

template< typename ShapeT >
class ShapeModel : public ShapeConcept
{
    public:
        using DrawStrategy = std::function<void(ShapeT const&>>;

        ShapeModel( ShapeT const& shape, DrawStrategy strategy )
            : shape_{ shape }
            , strategy_{ std::move(strategy) }
        {
            if( !strategy_ ) {
                throw std::invalid_argument( "Invalid draw strategy" );
            }
        }

        void draw() const override { strategy_(shape_); }

    private:
        ShapeT shape_;
        DrawStrategy strategy_;
};
```

A Solution based on External Polymorphism

```
void draw() const override { strategy_(shape_); }

private:
    ShapeT shape_;
    DrawStrategy strategy_;
};

class OpenGLStrategy
{
public:
    explicit OpenGLStrategy( /*...*/ ) {}

    void operator()( Circle const& circle ) const;

    void operator()( Square const& square ) const;

private:
    // ... OpenGL specific data members
};

void drawAllShapes( std::vector<ShapeConcept> const& shapes )
{
    for( auto const& shape : shapes )
    {
        shape->draw();
    }
}
```


A Solution based on External Polymorphism

```
public:
    explicit OpenGLStrategy( /*...*/ ) {}

    void operator()( Circle const& circle ) const;

    void operator()( Square const& square ) const;

private:
    // ... OpenGL specific data members
};

void drawAllShapes( std::vector<ShapeConcept> const& shapes )
{
    for( auto const& shape : shapes )
    {
        shape->draw();
    }
}

int main()
{
    using Shapes = std::vector<ShapeConcept>;

    // Creating some shapes
    shapes.emplace_back(
        std::make_unique<ShapeModel<Circle>>(
            Circle{2.3},
            TestDrawStrategy(Color::red) ) );
}
```

A Solution based on External Polymorphism

```
        shape->draw();
    }
}

int main()
{
    using Shapes = std::vector<ShapeConcept>;

    // Creating some shapes
    shapes.emplace_back(
        std::make_unique<ShapeModel<Circle>>(
            Circle{2.3},
            TestDrawStrategy(Color::red) ) );
    shapes.emplace_back(
        std::make_unique<ShapeModel<Square>>(
            Square{1.2},
            TestDrawStrategy(Color::green) ) );
    shapes.emplace_back(
        std::make_unique<ShapeModel<Circle>>(
            Circle{4.1},
            TestDrawStrategy(Color::blue) ) );

    drawAllShapes( shapes );

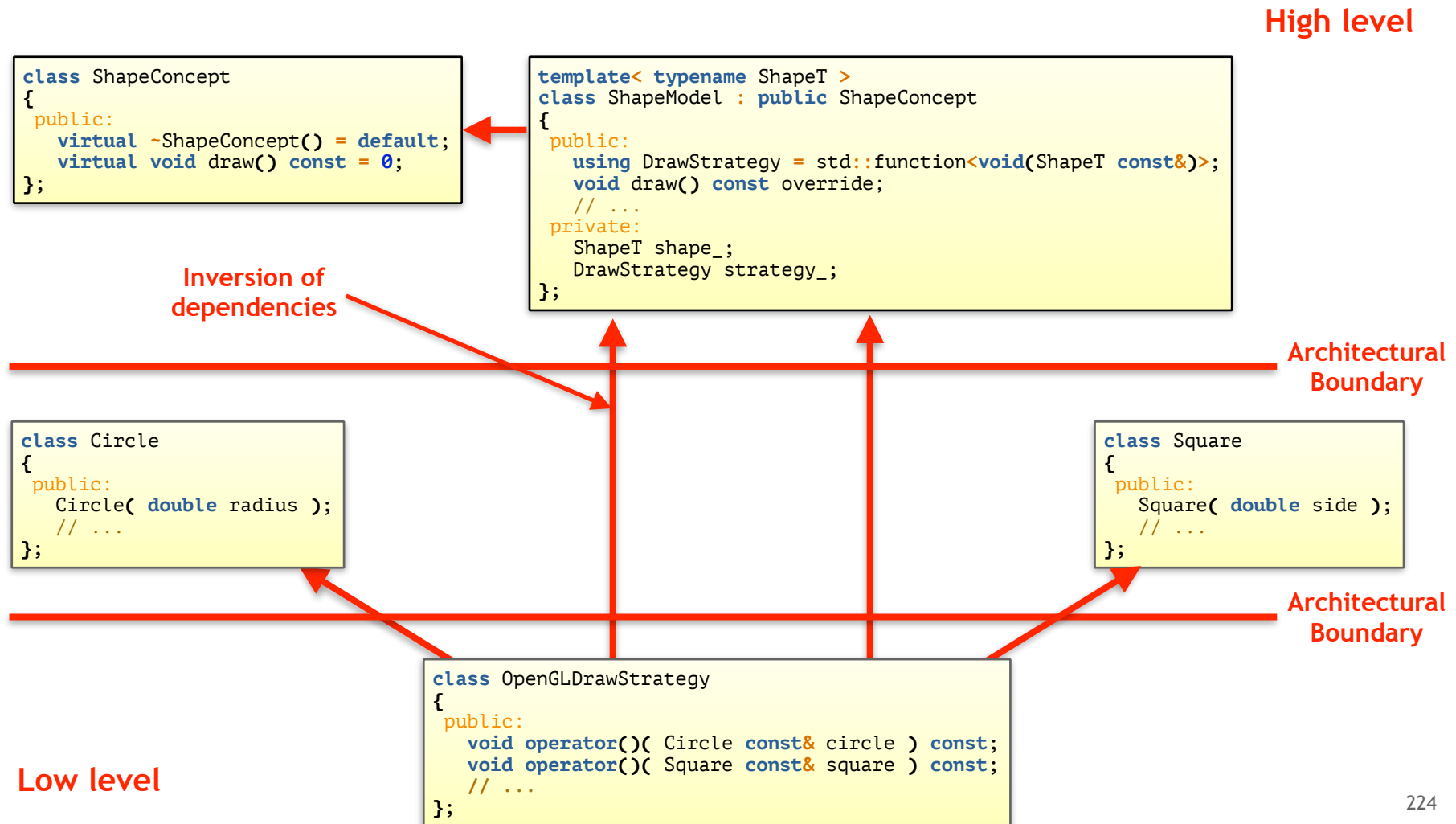
    // Drawing all shapes
    drawAllShapes( shapes );
}
```

External Polymorphism

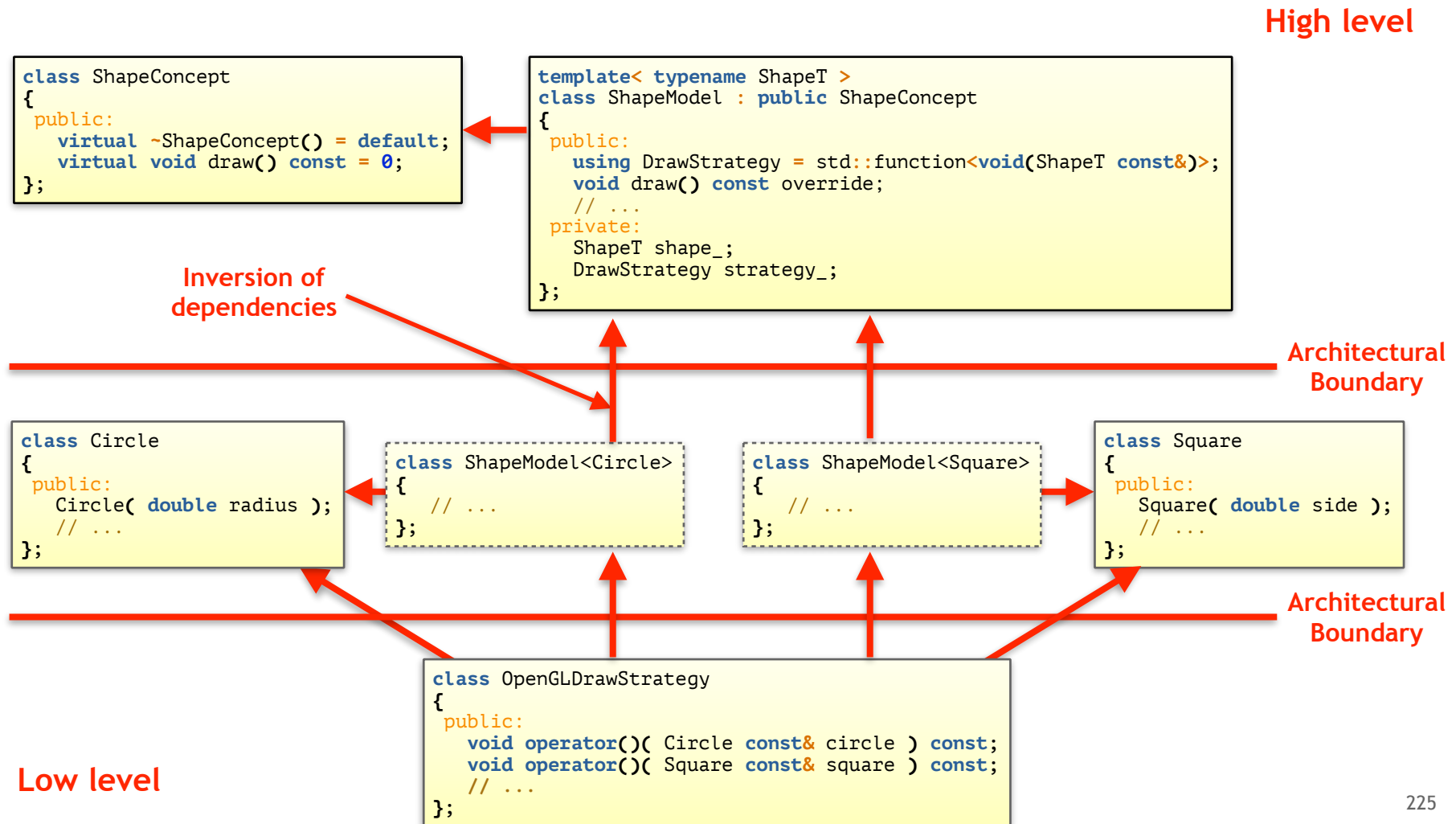
The External Polymorphism design pattern ...

- ... helps to extract polymorphic behavior from a type (SRP);
- ... reduces the boilerplate code for polymorphic behavior (DRY);
- ... allows the addition of non-polymorphic types (OCP);
- ... is non-intrusive;
- ... isolates concrete types (e.g. Circle, Square, ...) from their operations (affordances).

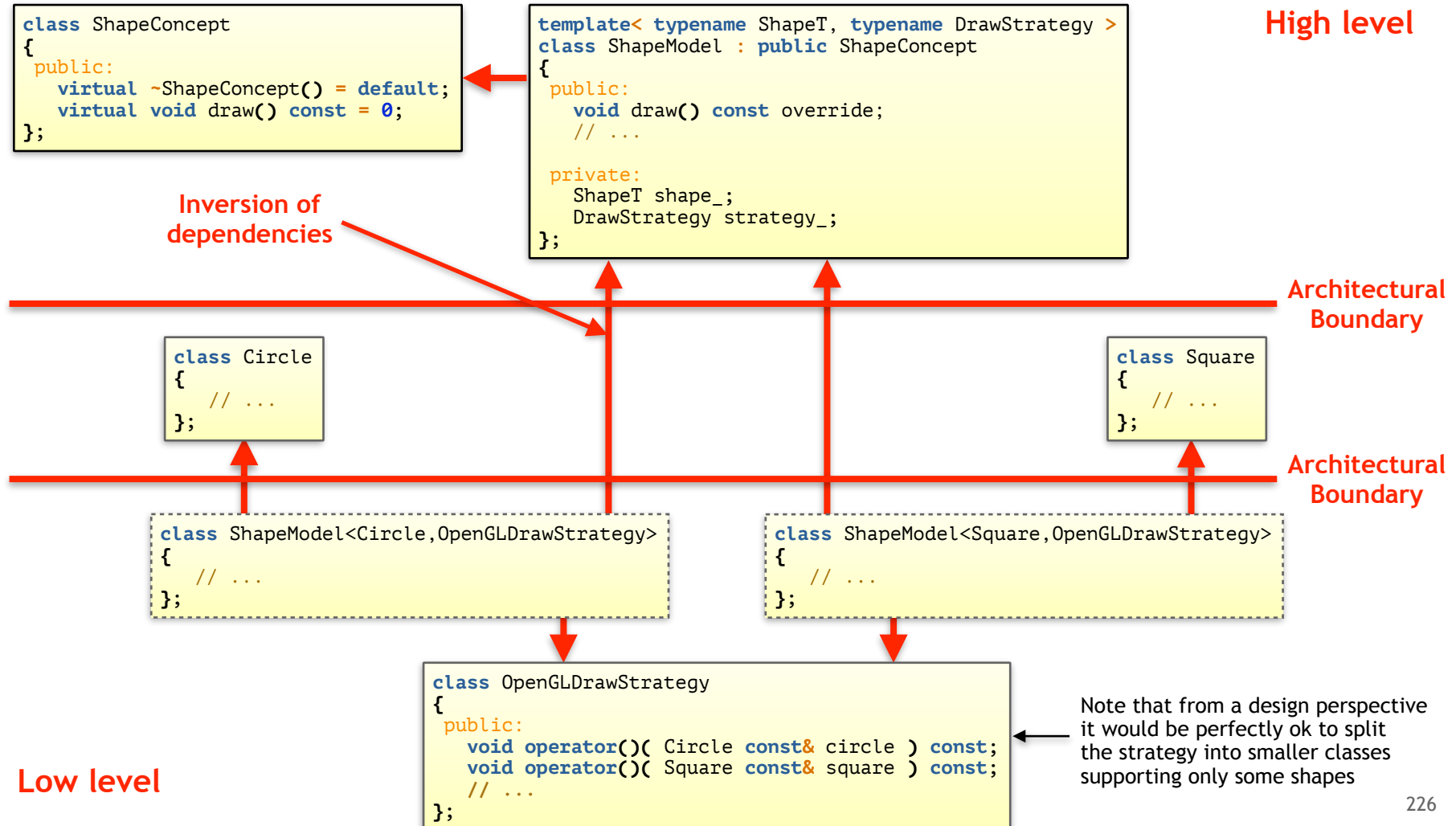
Dependency Structure



Dependency Structure



Dependency Structure



External Polymorphism vs. Strategy

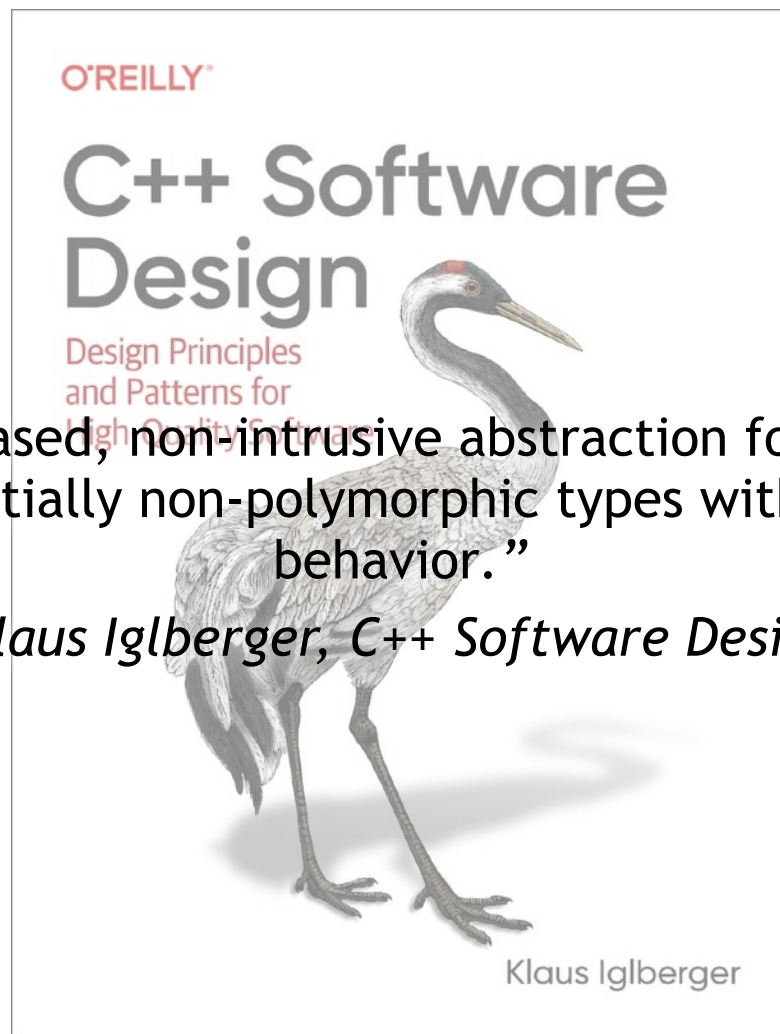
<i>External Polymorphism</i>	<i>Strategy</i>
Extracts the complete polymorphic behaviour	Configures the behavior from outside (dependency injection)
Non-intrusive	Intrusive
Only runtime polymorphism	Runtime or compile time polymorphism
No support by mocking frameworks	Strong support by mocking frameworks

Guidelines

Guideline: Use the External Polymorphism design pattern to extract polymorphic behavior from types or to treat non-polymorphic types polymorphically.

2.8. Type Erasure

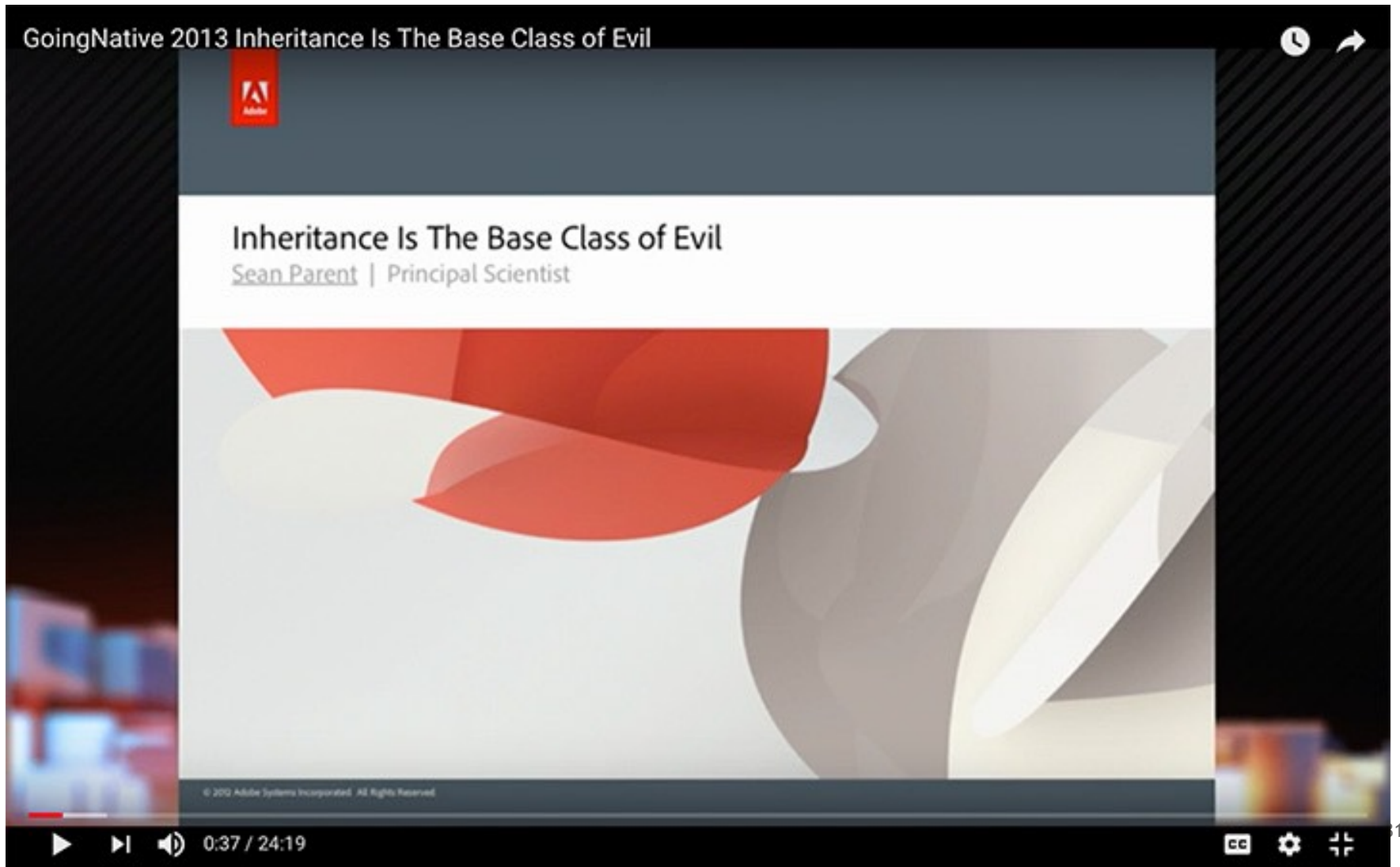
The Intent



”Provide a value-based, non-intrusive abstraction for an extendable set of unrelated, potentially non-polymorphic types with the same semantic behavior.”

(Klaus Iglberger, C++ Software Design)

Type Erasure



Type Erasure

[Valued Conversions \(C++ Report 12\(7\), July-August 2000\)](#)



Kevlin Henney is an independent consultant and trainer based in the UK. He may be contacted at kevin@curbralan.com.

FROM MECHANISM TO METHOD

Valued Conversions

HOW WOULD YOU like to pay for that?” Good question. Digging deep into pockets, wallets, and bags uncovered a wealth of possibilities, a handful of different currencies and mechanisms to choose from: credit cards, debit cards, coins, bills, and a couple of IOUs, each form in some way substitutable for another when realizing monetary value.

Cash is the simplest, least troublesome form for small amounts and quick transactions. However, sifting through the metal and paper, it seemed that my currencies were no good. Well, that’s

to any concrete form of inheritance. Substitutability here is based on values and conversions between values. Sometimes the use is implicit, at other times it must be made explicit. Conversions can be fully value preserving, widening, or narrowing. Widening conversions are always safe and typically acceptable (e.g., tipping), whereas narrowing conversions may not be (e.g., shortchanging tends to lead to exceptional or even undefined behavior).

Rescuing me from further metaphor stretching, the point-of-sale system and the assistant’s smile kicked into life.

Type Erasure

<https://twitter.com/ericniebler/status/1274031123522220033>

**Eric Niebler #BLM**
@ericniebler

If I could go back in time and had the power to change C++, rather than adding virtual functions, I would add language support for type erasure and concepts. Define a single-type concept, automatically generate a type-erasing wrapper for it.

7:27 PM · Jun 19, 2020 · Twitter for Android

6 Retweets 2 Quote Tweets 125 Likes

**Eric Niebler #BLM** @ericniebler · Jun 20

Replying to @ericniebler

This got more likes than I expected, and not one person saying "but but but my elaborate class hierarchies...." Huh.

 4   15 

Type Erasure — Examples from the Standard

```
#include <iostream>
#include <memory>

struct Deleter
{
    template< typename T >
    void operator()( T* ptr ) {
        std::cout << "Deleting ptr " << ptr << "\n";
        delete ptr;
    }
};

int main()
{
    std::shared_ptr<int> sptr1{ new int{42}, Deleter{} };
    std::shared_ptr<int> sptr2{ sptr1 };
}
```

Type Erasure — Examples from the Standard

```
#include <any>
#include <string>

using namespace std;

int main()
{
    any a{};

    a = 7; // Stores the integer 7

    any b{ "Any string"s }; // Creates an any with "Any string"s

    a = b;

    const string s
        = any_cast<string>( a ); // Extracts "Any string"s
}
```

Type Erasure — Examples from the Standard

```
#include <functional>

void foo(int i) {
    std::cout << "foo: " << i << '\n';
}

int main()
{
    std::function<void(int)> f{};

    f = [](int i){ std::cout << "lambda: " << i << '\n'; };

    auto g = f;    // Value semantics: Creates a deep copy

    f = foo;

    f( 1 );
    g( 2 );
}
```


std::function — A Simplified Implementation

Task (C_Cpp_Software_Design/Type_Erasure/Function1): Implement a simplified `std::function` to demonstrate the type erasure design pattern. Use the inheritance-based approach.

std::function – A Simplified Implementation

```
template< typename Fn >  
class function;
```

std::function – A Simplified Implementation

```
template< typename Fn >  
class function;
```

```
template< typename R, typename... Args >  
class function<R(Args...)>  
{  
    // ...  
};
```

std::function – A Simplified Implementation

```
template< typename Fn >
class function;

template< typename R, typename... Args >
class function<R(Args...)>
{
    // ...
private:
    class Concept
    {
    public:
        virtual ~Concept() = default;
        virtual R operator()( Args... ) const = 0;
        virtual Concept* clone() const = 0;
    };

    // ...
};
```

std::function – A Simplified Implementation

```
template< typename Fn >  
class function;
```

```
template< typename R, typename... Args >  
class function<R(Args...)>  
{  
    // ...  
    class Concept { ... };  

```

```
    // ...
```

```
    Concept* pimpl;  
};
```

std::function – A Simplified Implementation

```
template< typename Fn >
class function;
```

```
template< typename R, typename... Args >
class function<R(Args...)>
{
```

```
    // ...
```

```
    class Concept { ... };
```

```
    template< typename Fn >
```

```
    class Model : public Concept {
```

```
        explicit Model( Fn fn ) : fn_( fn ) {}
```

```
        R operator()( Args... args ) const override
```

```
        { return fn_( std::forward<Args>( args )... ); }
```

```
        Concept* clone() const override
```

```
        { return new Model( fn_ ); }
```

```
        Fn fn_;
```

```
    };
```

```
    // ...
```

```
};
```

std::function – A Simplified Implementation

```
template< typename Fn >  
class function;
```

```
template< typename R, typename... Args >  
class function<R(Args...)>  
{  
    public:
```

```
        template< typename Fn >  
        function( Fn fn ) : pimpl_( new Model<Fn>( fn ) ) {}
```

```
    private:  
        // ...  
};
```

std::function – A Simplified Implementation

```
template< typename Fn >
class function;
```

```
template< typename R, typename... Args >
class function<R(Args...)>
```

```
{
    public:
        template< typename Fn > function( Fn fn );

        function( function const& f )
            : pimpl_( f.pimpl_->clone() ) {}
        function& operator=( function f )
            { std::swap( pimpl_, f.pimpl_ ); return *this; }

```

```
    private:
        // ...
};
```


std::function – A Simplified Implementation

```
template< typename Fn >
class function;

template< typename R, typename... Args >
class function<R(Args...)>
{
public:
    template< typename Fn > function( Fn fn );
    function( function const& f );
    function& operator=( function f );
    function( function&& f ) : pimpl_( f.pimpl_ )
        { f.pimpl_ = nullptr; }
    function& operator=( function&& f )
        { delete pimpl_; pimpl_ = f.pimpl_;
          f.pimpl_ = nullptr; return *this; }

private:
    // ...
};
```

std::function – A Simplified Implementation

```
template< typename Fn >
class function;

template< typename R, typename... Args >
class function<R(Args...)>
{
public:
    template< typename Fn > function( Fn fn );
    function( function const& f );
    function& operator=( function f );
    function( function&& f );
    Function& operator=( Function&& f );
    ~function() { delete pimpl_; }

private:
    // ...
};
```

std::function – A Simplified Implementation

```
template< typename Fn >  
class function;
```

```
template< typename R, typename... Args >  
class function<R(Args...)>  
{  
    public:  
        // ...
```

```
    R operator()( Args&&... args )  
        { return (*pimpl_)( std::forward<Args>( args )... ); }
```

```
    private:  
        // ...  
};
```

std::function — A Simplified Implementation

Task (C_Cpp_Software_Design/Type_Erasure/Function2): Implement a simplified `std::function` to demonstrate the type erasure design pattern. Use the `void*`-based approach.

Applied Type Erasure

Task (C_Cpp_Software_Design/Type_Erasure/Animal_TE):

Step 1: Encapsulate the external `AnimalConcept` hierarchy in an owning Type Erasure wrapper and refactor the given solution into a value-based solution.

Step 2: Provide a non-owning Type Erasure wrapper that does not allocate dynamically and that does not create a copy of the given animal.

Applied Type Erasure

Task (C_Cpp_Software_Design/Type_Erasure/TypeErasure_1):

Step 1: Refactor the given External Polymorphism solution into Type Erasure. The Shape class may require all types to provide a `free_draw()` function that draws them to the screen.

Step 2: Switch from one to another graphics library. Discuss the feasibility of the change: how easy is the change? How many pieces of code on which level of the architecture have to be touched?

Applied Type Erasure

Task (C_Cpp_Software_Design/Type_Erasure/TypeErasure_2):

Step 1: Implement the Shape class by means of Type Erasure. Shape may require all types to provide a `free_draw()` function that draws them to the screen.

Step 2: Switch from one to another graphics library. Discuss the feasibility of the change: how easy is the change? How many pieces of code on which level of the architecture have to be touched?

std::function — A Simplified Implementation

Task (C_Cpp_Software_Design/Type_Erasure/TypeErasure_SBO_1/2): Upgrade the given Type Erasure implementation by means of the Small Buffer Optimization (SBO) technique to avoid any dynamic allocation. Shape may require all types to provide a `free_draw()` function that draws them to the screen.

A Type-Erased Solution

```
class Circle
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

private:
    double radius;
    // ... Remaining data members
};

void translate( Circle const&, Vector2D const& );
void rotate( Circle const&, double const& );
void draw( Circle const& );
```

```
class Square
{
public:
    explicit Square( double s )
        : side{ s }
        , // ... Remaining data members
    {}

    double getSide() const noexcept;
```

A Type-Erased Solution

```
class Circle
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
        {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

private:
    double radius;
    // ... Remaining data members
};

void translate( Circle const&, Vector2D const& );
void rotate( Circle const&, double const& );
void draw( Circle const& );
```

```
class Square
{
public:
    explicit Square( double s )
        : side{ s }
        , // ... Remaining data members
        {}

    double getSide() const noexcept;
```

A Type-Erased Solution

```
void draw( Circle const& );
```

```
class Square
{
public:
    explicit Square( double s )
        : side{ s }
        , // ... Remaining data members
    {}

    double getSide() const noexcept;
    // ... getCenter(), getRotation(), ...

private:
    double side;
    // ... Remaining data members
};
```

```
void translate( Square const&, Vector2D const& );
void rotate( Square const&, double const& );
void draw( Square const& );
```

```
class Shape
{
private:
    struct Concept
    {
        virtual ~Concept() = default;
        virtual void do_translate( Vector2D const& v ) const = 0;
    };
};
```

A Type-Erased Solution

```
void rotate( Square const&, double const& );  
void draw( Square const& );
```

```
class Shape  
{  
private:  
    struct Concept  
    {  
        virtual ~Concept() = default;  
        virtual void do_translate( Vector2D const& v ) const = 0;  
        virtual void do_rotate( double const& q ) const = 0;  
        virtual void do_draw() const = 0;  
        // ...  
    };  
  
    template< typename ShapeT >  
    struct Model : public Concept  
    {  
        Model( ShapeT const& value )  
            : object{ value }  
        {}  
  
        void do_translate( Vector2D const& v ) const override  
        {  
            translate( object, v );  
        }  
  
        void do_rotate( double const& q ) const override  
        {  
            rotate( object, q );  
        }  
    };  
};
```

A Type-Erased Solution

```
template< typename ShapeT >
struct Model : public Concept
{
    Model( ShapeT const& value )
        : object{ value }
    {}

    void do_translate( Vector2D const& v ) const override
    {
        translate( object, v );
    }

    void do_rotate( double const& q ) const override
    {
        rotate( object, q );
    }

    void do_draw() const override
    {
        draw( object );
    }

    // ...

    ShapeT object;
};

std::unique_ptr<Concept> pimpl;

friend void translate( Shape& shape, Vector2D const& v )
```

A Type-Erased Solution

```
void rotate( Square const&, double const& );
void draw( Square const& );

class Shape
{
private:
    struct Concept
    {
        virtual ~Concept() = default;
        virtual void do_translate( Vector2D const& v ) const = 0;
        virtual void do_rotate( double const& q ) const = 0;
        virtual void do_draw() const = 0;
        // ...
    };

    template< typename ShapeT >
    struct Model : public Concept
    {
        Model( ShapeT const& value )
            : object{ value }
        {}

        void do_translate( Vector2D const& v ) const override
        {
            translate( object, v );
        }

        void do_rotate( double const& q ) const override
        {
            rotate( object, q );
        }
    };
};
```

A Type-Erased Solution

```
    ShapeT object;
};

std::unique_ptr<Concept> pimpl;

friend void translate( Shape& shape, Vector2D const& v )
{
    shape.pimpl->do_translate( v );
}

friend void rotate( Shape& shape, double const& q )
{
    shape.pimpl->do_rotate( q );
}

friend void draw( Shape const& shape )
{
    shape.pimpl->do_draw();
}

public:
    template< typename ShapeT >
    Shape( ShapeT const& shape )
        : pimpl{ new Model<ShapeT>( shape ) }
    {}

    // Special member functions
    Shape( Shape const& s );
    Shape( Shape&& s );
    Shape& operator=( Shape const& s );
    Shape& operator=( Shape&& s );
```

A Type-Erased Solution

```
    ShapeT object;
};

std::unique_ptr<Concept> pimpl;

friend void translate( Shape& shape, Vector2D const& v )
{
    shape.pimpl->do_translate( v );
}

friend void rotate( Shape& shape, double const& q )
{
    shape.pimpl->do_rotate( q );
}

friend void draw( Shape const& shape )
{
    shape.pimpl->do_draw();
}

public:
    template< typename ShapeT >
    Shape( ShapeT const& shape )
        : pimpl{ new Model<ShapeT>( shape ) }
    {}

    // Special member functions
    Shape( Shape const& s );
    Shape( Shape&& s );
    Shape& operator=( Shape const& s );
    Shape& operator=( Shape&& s );
```


A Type-Erased Solution

```
    {
        shape.pimpl->do_draw();
    }

public:
    template< typename ShapeT >
    Shape( ShapeT const& shape )
        : pimpl{ new Model<ShapeT>( shape ) }
    {}

    // Special member functions
    Shape( Shape const& s );
    Shape( Shape&& s );
    Shape& operator=( Shape const& s );
    Shape& operator=( Shape&& s );

    // ...
};

void drawAllShapes( std::vector<Shape> const& shapes )
{
    for( auto const& shape : shapes )
    {
        draw( shape );
    }
}

int main()
{
```

A Type-Erased Solution

```
Shape( Shape const& s );
Shape( Shape&& s );
Shape& operator=( Shape const& s );
Shape& operator=( Shape&& s );

// ...
};

void drawAllShapes( std::vector<Shape> const& shapes )
{
    for( auto const& shape : shapes )
    {
        draw( shape );
    }
}

int main()
{
    using Shapes = std::vector<Shape>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( Circle{ 2.0 } );
    shapes.emplace_back( Square{ 1.5 } );
    shapes.emplace_back( Circle{ 4.2 } );

    // Drawing all shapes
    drawAllShapes( shapes );
}
```

A Type-Erased Solution

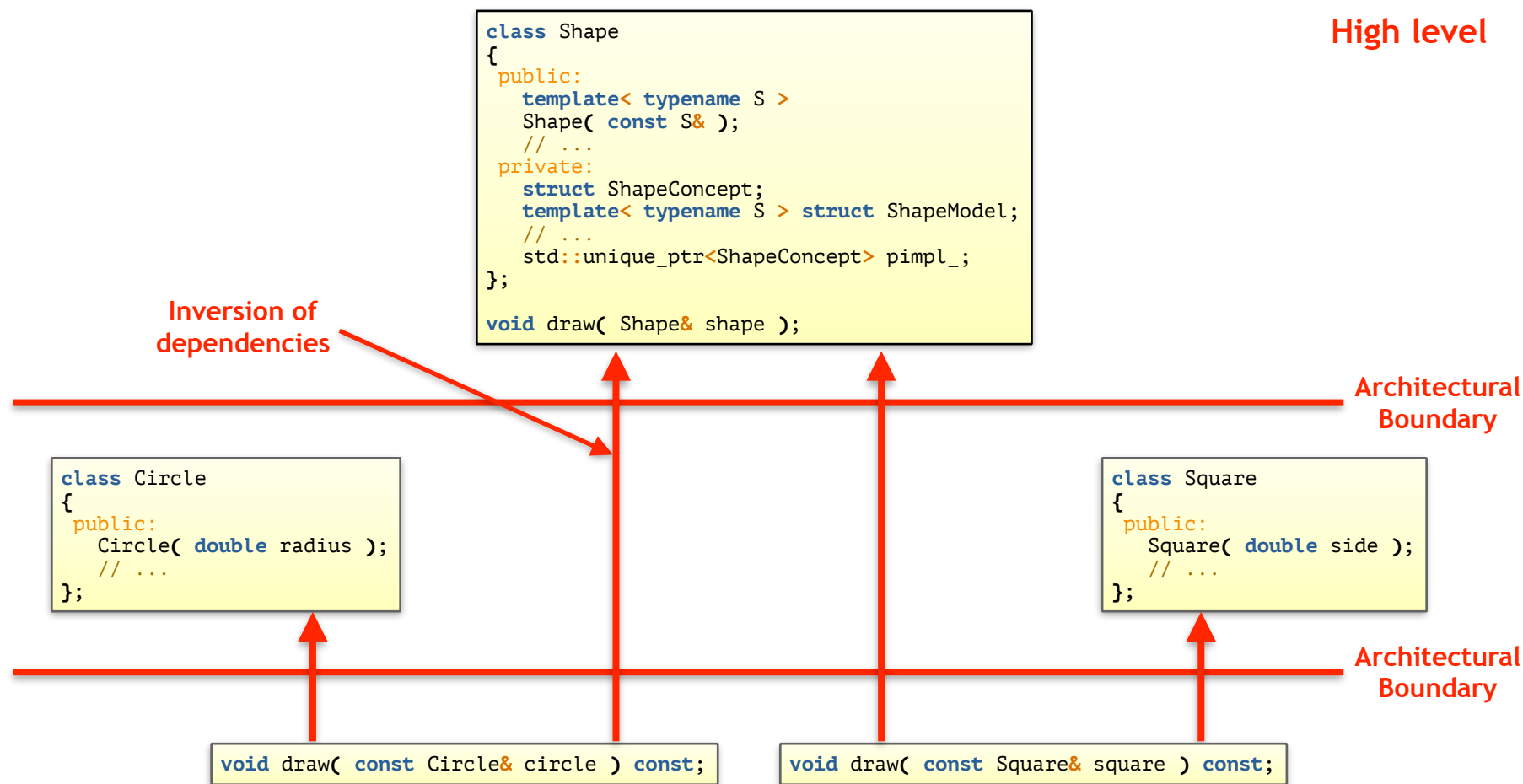
```
for( auto const& shape : shapes )
{
    draw( shape );
}

int main()
{
    using Shapes = std::vector<Shape>;

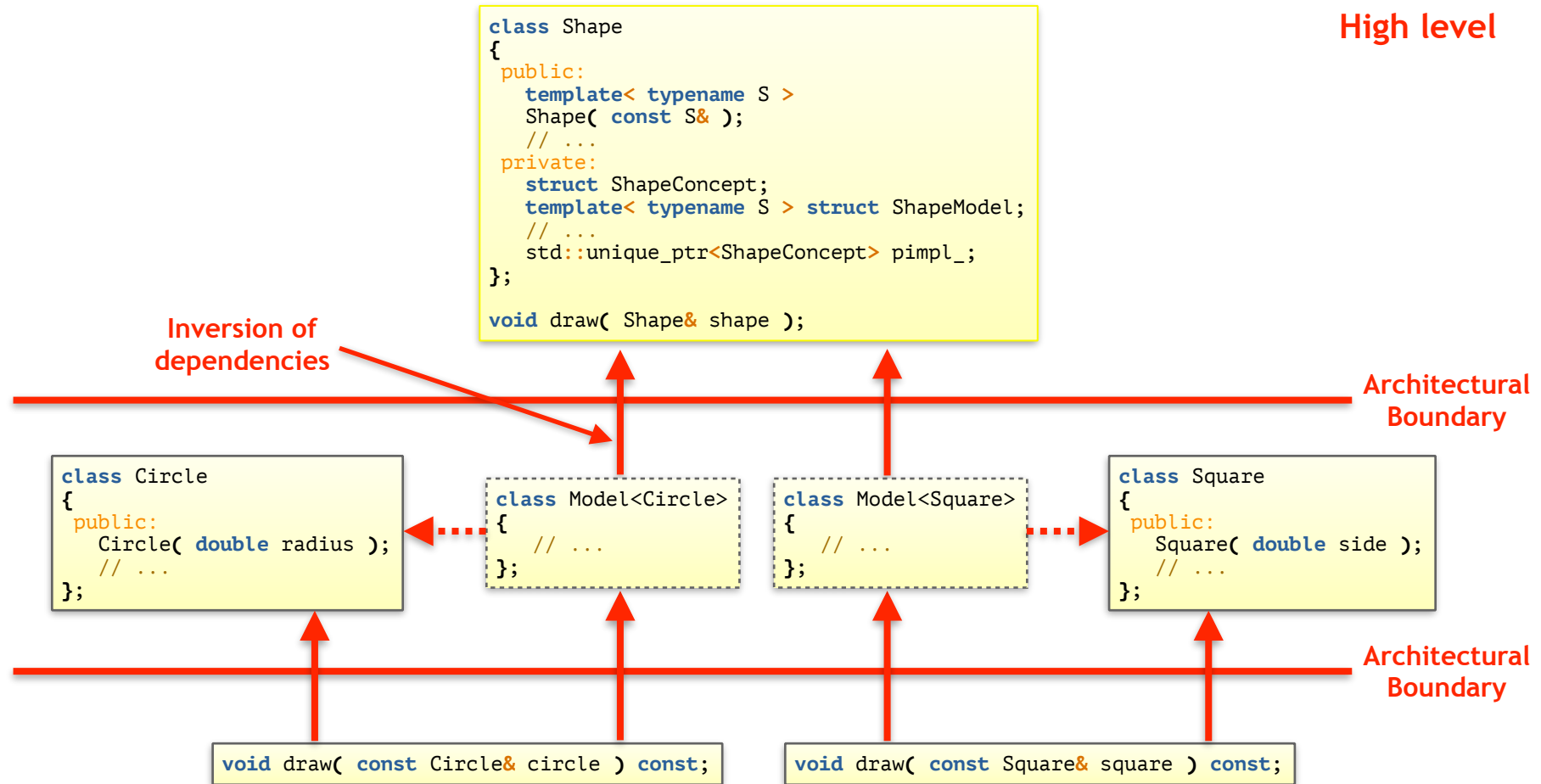
    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( Circle{ 2.0 } );
    shapes.emplace_back( Square{ 1.5 } );
    shapes.emplace_back( Circle{ 4.2 } );

    // Drawing all shapes
    drawAllShapes( shapes );
}
```

Dependency Structure



Dependency Structure



Type Erasure with Small Buffer Optimization

```
class Circle
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

private:
    double radius;
    // ... Remaining data members
};

void translate( Circle const&, Vector2D const& );
void rotate( Circle const&, double const& );
void draw( Circle const& );
```

```
class Square
{
public:
    explicit Square( double s )
        : side{ s }
        , // ... Remaining data members
    {}

    double getSide() const noexcept;
```

Type Erasure with Small Buffer Optimization

```
class Circle
{
public:
    explicit Circle( double rad )
        : radius{ rad }
        , // ... Remaining data members
    {}

    double getRadius() const noexcept;
    // ... getCenter(), getRotation(), ...

private:
    double radius;
    // ... Remaining data members
};

void translate( Circle const&, Vector2D const& );
void rotate( Circle const&, double const& );
void draw( Circle const& );
```

```
class Square
{
public:
    explicit Square( double s )
        : side{ s }
        , // ... Remaining data members
    {}

    double getSide() const noexcept;
```

Type Erasure with Small Buffer Optimization

```
void draw( Circle const& );
```

```
class Square
{
public:
    explicit Square( double s )
        : side{ s }
        , // ... Remaining data members
    {}

    double getSide() const noexcept;
    // ... getCenter(), getRotation(), ...

private:
    double side;
    // ... Remaining data members
};
```

```
void translate( Square const&, Vector2D const& );
void rotate( Square const&, double const& );
void draw( Square const& );
```

```
class Shape
{
private:
    struct Concept
    {
        virtual ~Concept() = default;
        virtual void do_translate( Vector2D const& v ) const = 0;
```


Type Erasure with Small Buffer Optimization

```
class Shape
{
private:
    struct Concept
    {
        virtual ~Concept() = default;
        virtual void do_translate( Vector2D const& v ) const = 0;
        virtual void do_rotate( double const& q ) const = 0;
        virtual void do_draw() const = 0;
        // ...
    };

    template< typename ShapeT >
    struct Model final : public Concept
    {
        Model( ShapeT const& value )
            : object{ value }
        {}

        void do_translate( Vector2D const& v ) const override
        {
            translate( object, v );
        }

        void do_rotate( double const& q ) const override
        {
            rotate( object, q );
        }

        void do_draw() const override
```

Type Erasure with Small Buffer Optimization

```
class Shape
{
private:
    struct Concept
    {
        virtual ~Concept() = default;
        virtual void do_translate( Vector2D const& v ) const = 0;
        virtual void do_rotate( double const& q ) const = 0;
        virtual void do_draw() const = 0;
        // ...
    };

    template< typename ShapeT >
    struct Model final : public Concept
    {
        Model( ShapeT const& value )
            : object{ value }
        {}

        void do_translate( Vector2D const& v ) const override
        {
            translate( object, v );
        }

        void do_rotate( double const& q ) const override
        {
            rotate( object, q );
        }

        void do_draw() const override
```

Type Erasure with Small Buffer Optimization

```
};

template< typename ShapeT >
struct Model final : public Concept
{
    Model( ShapeT const& value )
        : object{ value }
    {}

    void do_translate( Vector2D const& v ) const override
    {
        translate( object, v );
    }

    void do_rotate( double const& q ) const override
    {
        rotate( object, q );
    }

    void do_draw() const override
    {
        draw( object );
    }

    // ...

    ShapeT object;
};
```

```
static constexpr size_t buffersize = 128UL;
static constexpr size_t alignment = 16UL;
```

Type Erasure with Small Buffer Optimization

```
void do_draw() const override
{
    draw( object );
}

// ...

ShapeT object;
};

static constexpr size_t buffersize = 128UL;
static constexpr size_t alignment = 16UL;

alignas(alignment) std::array<std::byte, buffersize> buffer;

Concept* pimpl() noexcept
{
    return reinterpret_cast<Concept*>( buffer.data() );
}
const Concept* pimpl() const noexcept
{
    return reinterpret_cast<const Concept*>( buffer.data() );
}

friend void translate( Shape& shape, Vector2D const& v )
{
    shape.pimpl()->do_translate( v );
}

friend void rotate( Shape& shape, double const& q )
{
    shape.pimpl()->do_rotate( q );
}
```

Type Erasure with Small Buffer Optimization

```

{
    return reinterpret_cast<Concept*>( buffer.data() );
}
const Concept* pimpl() const noexcept
{
    return reinterpret_cast<const Concept*>( buffer.data() );
}

friend void translate( Shape& shape, Vector2D const& v )
{
    shape.pimpl()->do_translate( v );
}

friend void rotate( Shape& shape, double const& q )
{
    shape.pimpl()->do_rotate( q );
}

friend void draw( Shape const& shape )
{
    shape.pimpl()->do_draw();
}

public:
template< typename ShapeT >
Shape( ShapeT const& shape )
{
    using M = Model<ShapeT>;
    static_assert( sizeof(M) <= buffersize, "Given type is too large" );
    static_assert( alignof(M) <= alignment, "Given type is overaligned" );
    ::new (pimpl()) M( shape );
}

```

Type Erasure with Small Buffer Optimization

```
friend void draw( Shape const& shape )
{
    shape.pimpl()->do_draw();
}

public:
    template< typename ShapeT >
    Shape( ShapeT const& shape )
    {
        using M = Model<ShapeT>;
        static_assert( sizeof(M) <= buffersize, "Given type is too large" );
        static_assert( alignof(M) <= alignment, "Given type is overaligned" );
        ::new (pimpl()) M( shape );
    }

    // Special member functions
    ~Shape();
    Shape( Shape const& other );
    Shape& operator=( Shape const& other );

    // Move operations intentionally ignored!

    // ...
};

void drawAllShapes( std::vector<Shape> const& shapes )
{
    for( auto const& shape : shapes )
    {

```

Type Erasure with Small Buffer Optimization

```
// Special member functions
~Shape();
Shape( Shape const& other );
Shape& operator=( Shape const& other );

// Move operations intentionally ignored!

// ...
};

void drawAllShapes( std::vector<Shape> const& shapes )
{
    for( auto const& shape : shapes )
    {
        draw( shape );
    }
}

int main()
{
    using Shapes = std::vector<Shape>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( Circle{ 2.0 } );
    shapes.emplace_back( Square{ 1.5 } );
    shapes.emplace_back( Circle{ 4.2 } );

    // Drawing all shapes
```

Type Erasure with Small Buffer Optimization

```
void drawAllShapes( std::vector<Shape> const& shapes )
{
    for( auto const& shape : shapes )
    {
        draw( shape );
    }
}
```

```
int main()
{
    using Shapes = std::vector<Shape>;

    // Creating some shapes
    Shapes shapes;
    shapes.emplace_back( Circle{ 2.0 } );
    shapes.emplace_back( Square{ 1.5 } );
    shapes.emplace_back( Circle{ 4.2 } );

    // Drawing all shapes
    drawAllShapes( shapes );
}
```


Parameter Conventions: Concrete Types

Passing by reference-to-const to a concrete type we can express that we can only work with a specific kind of shape.

```
Square square{ 1.5 };  
Shape shape{ Square{ 1.5 } };
```

```
void f( const Square& );    // Reference to a concrete type:  
                           // Works only for Square, not for  
                           // other shapes
```

```
f( square );    // Compiles  
f( shape );    // Does NOT compile
```

Parameter Conventions: Abstract Types (I)

Passing by reference-to-const to the type erased type we can express that we can only work with any kind of shape.

```
Square square{ 1.5 };  
Shape  shape{ Square{ 1.5 } };
```

```
void f( const Shape& );           // Reference to abstract type:  
                                   // Works for all kinds of  
                                   // shapes, but might allocate
```

```
f( square );    // Compiles  
f( shape );     // Compiles, but creates a temporary 'Shape'
```

Parameter Conventions: Abstract Types (II)

Passing by value to a non-owning kind of type erasure we can express that we can only work with any kind of shape, but don't need to own.

```
Square square{ 1.5 };  
Shape shape{ Square{ 1.5 } };
```

```
void f( ShapeView );           // Pass via non-owning type erasure:  
                               // Works for all kinds of  
                               // shapes, no allocation
```

```
f( square ); // Compiles  
f( shape );  // Compiles, no allocation
```

Non-Intrusive API Design: Type Erasure

Task (C_Cpp_Software_Design/Type_Erasure/TypeErasure_Ref): Implement the `ShapeView` class, representing a reference to a constant shape, by means of Type Erasure. `ShapeView` may require all types to provide a free `draw()` function that draws them to the screen.

Non-Intrusive API Design: Type Erasure

```
class ShapeView
{
public:
    template< typename ShapeT >
    ShapeView( ShapeT const& shape )
        : shape_{ std::addressof(shape) }
        , draw_{ []( void const* shape ){
                    draw( *static_cast<ShapeT const*>(shape) );
                } }
    {}

private:
    friend void draw( ShapeView const& shape )
    {
        shape.draw_( shape.shape_ );
    }

    using DrawOperation = void(void const*);

    void const* shape_{ nullptr };
    DrawOperation* draw_{ nullptr };
};
```

Non-Intrusive API Design: Type Erasure

Task (C_Cpp_Software_Design/Type_Erasure/Function_Ref):
Implement a simplified `std::function_ref` to represent a non-owning abstraction for any type of callable.

Non-Intrusive API Design: Type Erasure

```
template< typename Fn >
class Function_Ref;

template< typename R, typename... Args >
class Function_Ref<R(Args...)>
{
public:
    template< typename Fn >
    Function_Ref( Fn const& fn )
        : invoke_( []( void const* c, Args... args ) -> R {
            auto const* const fn( static_cast<Fn const*>(c) );
            return (*fn)( std::forward<Args>(args)... );
        } )
        , callable_( std::addressof(fn) )
    {}

    R operator()( Args... args ) const
    {
        return invoke_( callable_, std::forward<Args>(args)... );
    }

private:
    using InvokeOperation = R(void const*, Args...);
```

Non-Intrusive API Design: Type Erasure

```
template< typename R, typename... Args >
class Function_Ref<R(Args...)>
{
public:
    template< typename Fn >
    Function_Ref( Fn const& fn )
        : invoke_( []( void const* c, Args... args ) -> R {
            auto const* const fn( static_cast<Fn const*>(c) );
            return (*fn)( std::forward<Args>(args)... );
        } )
        , callable_( std::addressof(fn) )
    {}

    R operator()( Args... args ) const
    {
        return invoke_( callable_, std::forward<Args>(args)... );
    }

private:
    using InvokeOperation = R(void const*, Args...);

    InvokeOperation* invoke_{ nullptr };
    void const* callable_;
};
```


Type Erasure

In its basic form, type erasure is a clever combination of several classic design patterns:

- Prototype: Enables virtual copying;
- Bridge: Provides a compilation firewall;
- External Polymorphism: Adds a new inheritance hierarchy to enable the storage of any requirement fulfilling type;
- Proxy: Enables a seamless integration into existing code.

Also, it nicely combines the strengths of static and dynamic polymorphism by means of ...

- ... an encapsulated inheritance hierarchy;
- ... a class template.

Type Erasure

Classic design patterns ...

- ... require a base class (dependency);
- ... promote heap allocation;
- ... require memory management.

Using type erasure instead ...

- ... simplifies code by encapsulating all responsibilities;
- ... facilitates comprehension;
- ... reduces dependencies;
- ... allows performance optimizations (e.g. SBO).

Type Erasure — Advantages/Disadvantages

Use type erasure if ...

- ... you have a **closed set** of functions;
- ... you want to **extend types**, not functions;
- ... you want to promote **polymorphic usage of types**.

Don't use type erasure if ...

- ... you have an **open set** of functions;
- ... you want to **extend functions**, not types.

Type Erasure vs. `std::variant`

<i>Type Erasure</i>	<i><code>std::variant</code></i>
Open set of types / Closed set of operations	Open set of operations / Closed set of types
No dynamic allocation with SBO	Never uses dynamic allocation
Manual optimization required (SBO and MVF)	No manual optimization required

A wrong choice particularly hurts in a large code base and if other people would experience a change.

Applied Type Erasure

Task (C_Cpp_Software_Design/Type_Erasure/PolymorphicAllocator): Implement the `PolymorphicAllocator` class by means of Type Erasure. `PolymorphicAllocator` may require all types to provide a free `allocate()` and a `deallocate()` member function.

Guidelines

Guideline: Use Type Erasure to enable the non-intrusive and value-based extension of an open set of types.

Guideline: Don't use Type Erasure when you need to frequently extend the closed set of operations.

Guideline: Choose between Type Erasure and Visitor (aka `std::variant`) depending on whether you have an open set of types or an open set of operations.

Core Guideline T.49: Where possible, avoid type-erasure

There Is No Silver Bullet



*"It's a general mistake to search for a silver bullet."
(Neal Ford)*

2.9. Prototype

The Motivation



Motivation

```
//----- <Animal.h> -----  
  
class Animal  
{  
public:  
    virtual ~Animal() = default;  
    virtual void make_sound() const = 0;  
  
    // ... more animal-specific functions  
};  
  
//----- <Sheep.h> -----  
  
#include <Animal.h>  
  
class Sheep : public Animal  
{  
public:  
    void make_sound() const override { std::cout << "baa\n"; };  
  
    // ... more animal-specific functions  
};  
  
//----- <Main.cpp> -----  
  
#include <Sheep.h>  
#include <cstdlib>  
#include <memory>
```

Motivation

```
class Sheep : public Animal
{
    public:
        void make_sound() const override { std::cout << "baa\n"; };

        // ... more animal-specific functions
};

//----- <Main.cpp> -----

#include <Sheep.h>
#include <cstdlib>
#include <memory>

int main()
{
    // Creating the one and only Dolly
    std::unique_ptr<Animal> const sheep{ std::make_unique<Sheep>( "Dolly" ) };
    sheep->make_sound();

    // Cloning Dolly
    // ???

    return EXIT_SUCCESS;
}
```

Motivation

```
//----- <Animal.h> -----  
  
class Animal  
{  
public:  
    virtual ~Animal() = default;  
    virtual void make_sound() const = 0;  
  
    // ... more animal-specific functions  
};  
  
//----- <Sheep.h> -----  
  
#include <Animal.h>  
  
class Sheep : public Animal  
{  
public:  
    void make_sound() const override { std::cout << "baa\n"; };  
  
    // ... more animal-specific functions  
};  
  
//----- <Main.cpp> -----  
  
#include <Sheep.h>  
#include <cstdlib>  
#include <memory>
```

Motivation

```
//----- <Animal.h> -----  
  
class Animal  
{  
public:  
    virtual ~Animal() = default;  
    virtual void make_sound() const = 0;  
    virtual std::unique_ptr<Animal> clone() const = 0;  
    // ... more animal-specific functions  
};  
  
//----- <Sheep.h> -----  
  
#include <Animal.h>  
  
class Sheep : public Animal  
{  
public:  
    void make_sound() const override { std::cout << "baa\n"; };  
    std::unique_ptr<Animal> clone() const override  
    {  
        return std::make_unique<Sheep>(*this);  
    }  
    // ... more animal-specific functions  
};  
  
//----- <Main.cpp> -----
```

Motivation

```
public:
    void make_sound() const override { std::cout << "baa\n"; };
    std::unique_ptr<Animal> clone() const override
    {
        return std::make_unique<Sheep>(*this);
    }
    // ... more animal-specific functions
};

//----- <Main.cpp> -----

#include <Sheep.h>
#include <cstdlib>
#include <memory>

int main()
{
    // Creating the one and only Dolly
    std::unique_ptr<Animal> const sheep{ std::make_unique<Sheep>( "Dolly" ) };
    sheep->make_sound();

    // Cloning Dolly
    auto cloned_sheep{ sheep->clone() };
    cloned_sheep->make_sound();

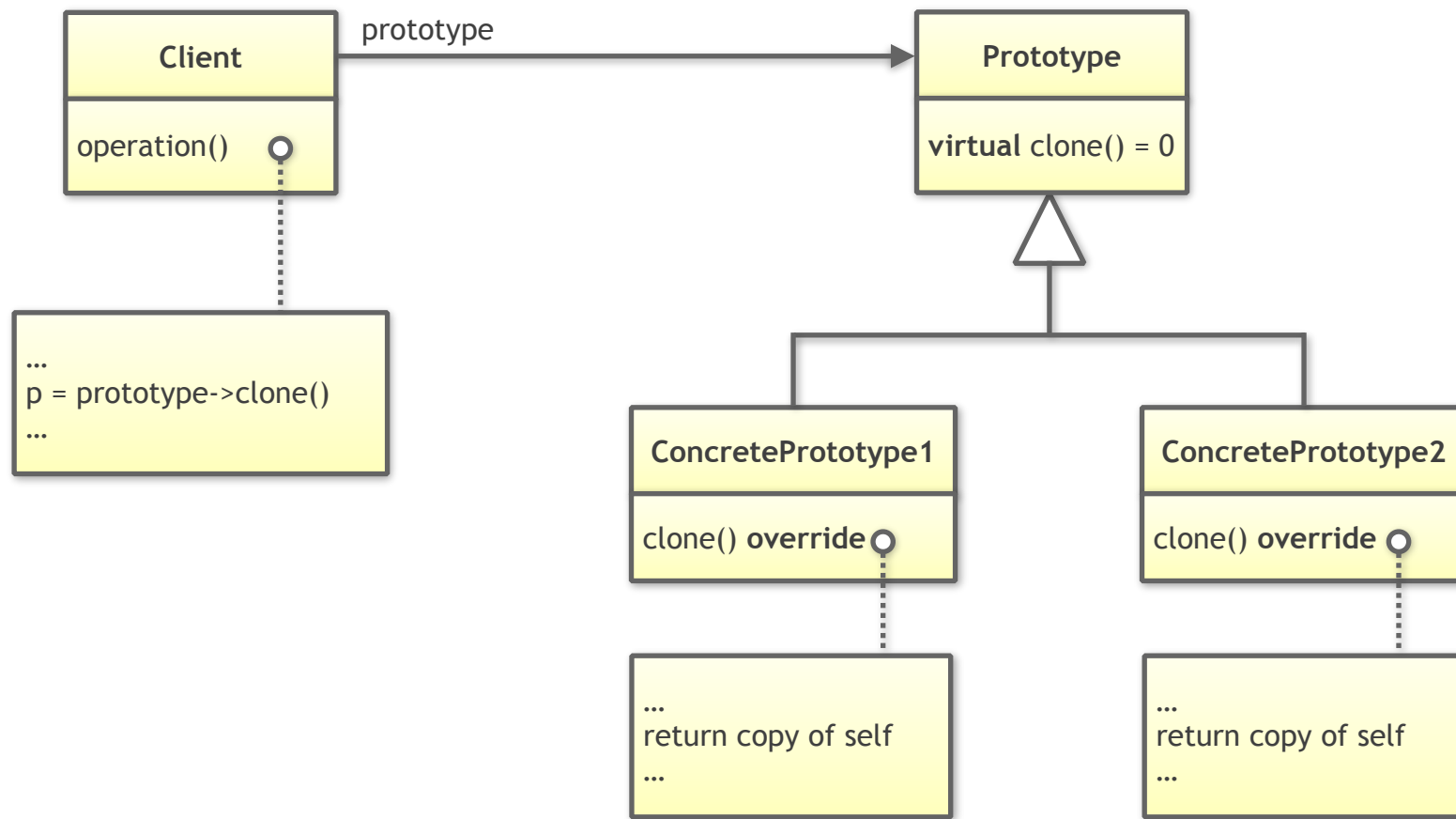
    return EXIT_SUCCESS;
}
```

The Intent

”Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.”
(GoF, *Design Patterns: Elements of Reusable Object-Oriented Software*)



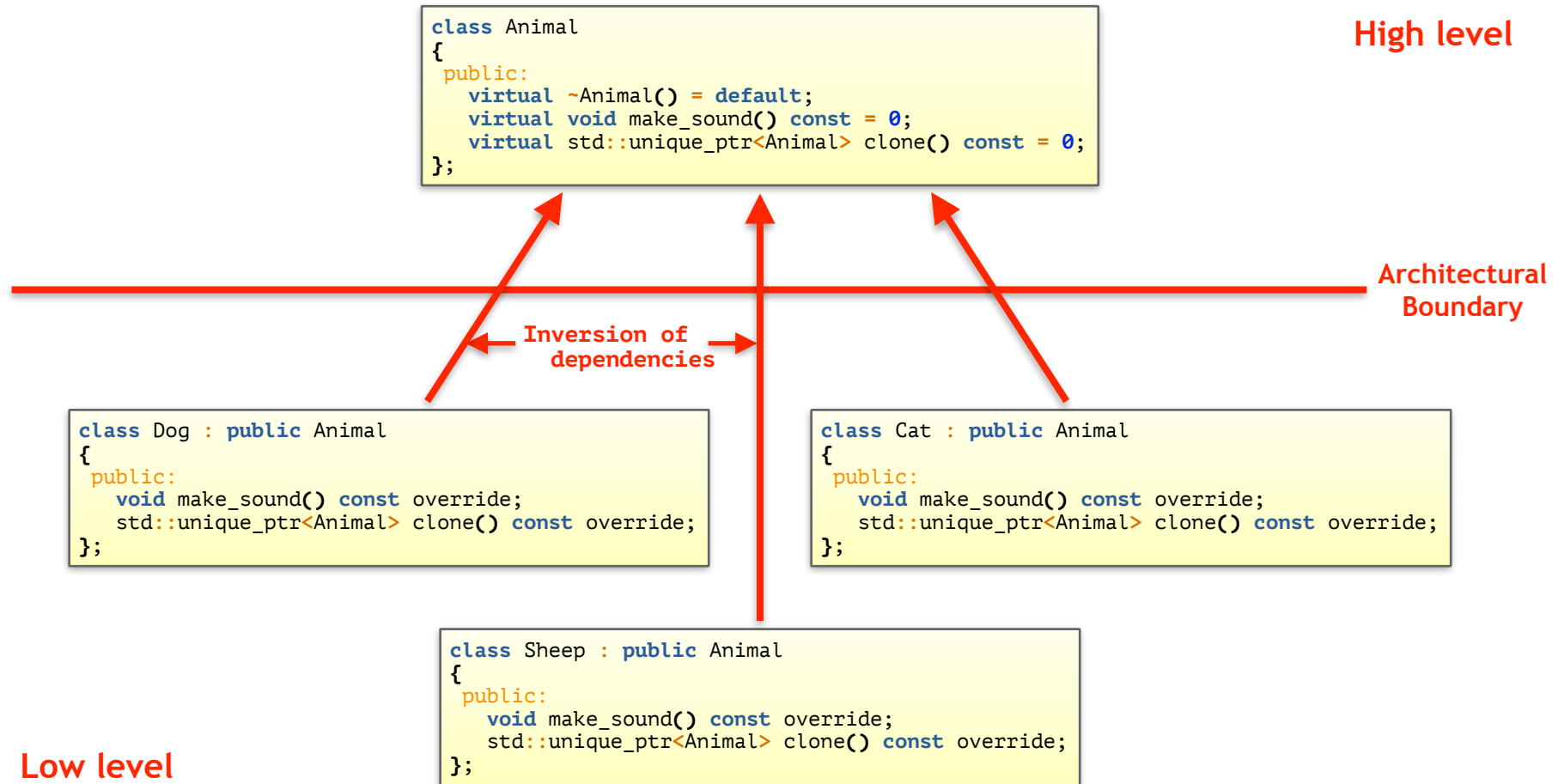
The Prototype Design Pattern



The Classic Command Pattern

Task (C_Cpp_Software_Design/Prototype/Prototype): Discuss the advantages and disadvantages of the given implementation of the classic Prototype design pattern.

Dependency Structure



Guidelines

Guideline: Use the Prototype design pattern to create exact copies of an abstraction.

Core Guideline C.130: For making deep copies of polymorphic classes prefer a virtual `clone` function instead of copy construction/assignment.

Core Guideline I.11: Never transfer ownership by a raw pointer (`T*`) or reference (`T&`).

Core Guideline F.26: Use a `unique_ptr<T>` to transfer ownership where a pointer is needed.

Guidelines

Guideline: Remember that the Prototype design patterns encapsulates and distributes object creation (i.e. potentially violates SRP).

2.10. Bridge

Motivation

Change
of implementation details

Motivation



Electric Hero: The Ultimate Electric Cars



Electric Hero: The Ultimate Electric Cars



```
//----- <Model10.h> -----
```

```
#include <BatteryGen1.h>
#include <ElectricEngineGen1.h>
// ...
```

```
namespace eh {
```

```
class Model10
{
```

```
public:
    void drive();
    // ...
```

```
private:
    BatteryGen1 battery_;
    ElectricEngineGen1 engine_;
```

```
    // ... more car-specific data members (wheels, drivetrain, ...)
};
```

```
} // namespace eh
```

This creates a direct dependency on the **Battery** and **ElectricEngine** classes: Changes to either of them will be visible to all classes including the **<ElectricHero.h>** header.

Electric Hero: The Ultimate Electric Cars



```
//----- <Model10.h> -----  
  
#include <memory>  
// ...  
  
namespace eh {  
  
    struct BatteryGen1;           // Forward declaration  
    struct ElectricEngineGen1;    // Forward declaration  
  
    class Model10  
    {  
    public:  
        void drive();  
        // ...  
    private:  
        std::unique_ptr<BatteryGen1> battery_;  
        std::unique_ptr<ElectricEngineGen1> engine_;  
        // ...  
    };  
} // namespace eh
```

Relying on a forward declarations instead removes dependencies to the implementation details of **Battery** and **ElectricEngine**, but still reveals that a **Battery** and **ElectricEngine** are used. Switching to another classes will affect all classes including the **<ElectricHero.h>** header.

```
//----- <Model10.cpp> -----  
  
#include <BatteryGen1.h>  
#include <ElectricEngineGen1.h>
```

Electric Hero: The Ultimate Electric Cars

```
#include <memory>
// ...

namespace eh {

struct BatteryGen1;           // Forward declaration
struct ElectricEngineGen1;    // Forward declaration

class Model10
{
public:
    void drive();
    // ...
private:
    std::unique_ptr<BatteryGen1> battery_;
    std::unique_ptr<ElectricEngineGen1> engine_;
    // ...
};

} // namespace eh

//----- <Model10.cpp> -----

#include <BatteryGen1.h>
#include <ElectricEngineGen1.h>

// ...
```



Electric Hero: The Ultimate Electric Cars



```
//----- <ElectricCarImpl.h> -----  
  
namespace eh {  
  
class ElectricCarImpl  
{  
public:  
    virtual ~ElectricCarImpl() = default;  
  
    virtual void start() = 0;  
    virtual void stop() = 0;  
  
    virtual void drawPower() = 0;  
    virtual void charge() = 0;  
  
    // ... more car-specific functions  
  
private:  
    // ... potentially some car-specific data members (wheels, drivetrain, ...)  
};  
} // namespace eh  
  
//----- <Model10.h> -----  
  
#include <ElectricCarImpl.h>  
#include <memory>  
  
namespace eh {
```

Electric Hero: The Ultimate Electric Cars

```
//---- <ElectricCarImpl.h> -----
```

```
namespace eh {
```

```
class ElectricCarImpl  
{
```

```
public:
```

```
    virtual ~ElectricCarImpl() = default;
```

```
    virtual void start() = 0;
```

```
    virtual void stop() = 0;
```

```
    virtual void drawPower() = 0;
```

```
    virtual void charge() = 0;
```

```
    // ... more car-specific functions
```

```
private:
```

```
    // ... potentially some car-specific data members (wheels, drivetrain, ...)
```

```
};
```

```
} // namespace eh
```

```
//---- <Model10.h> -----
```

```
#include <ElectricCarImpl.h>
```

```
#include <memory>
```

```
namespace eh {
```



Electric Hero: The Ultimate Electric Cars



```
//  
  
} // namespace eh  
  
//----- <Model10.h> -----  
  
#include <ElectricCarImpl.h>  
#include <memory>  
  
namespace eh {  
  
class Model10  
{  
public:  
    void drive();  
    // ...  
  
private:  
    std::unique_ptr<ElectricCarImpl> pimpl_;  
  
    // ... potentially some non-coupling car-specific data members  
};  
  
} // namespace eh  
  
//----- <Model10Impl.h> -----  
  
#include <ElectricCarImpl.h>  
#include <BatteryGen1.h>  
#include <TheElectricHeroGen1.h>
```

Introducing the **ElectricCarImpl** abstraction enables the **ElectricHero** class to switch to different implementations without anyone noticing. Thus we have build a Bridge to the implementation details.



Electric Hero: The Ultimate Electric Cars

```
//----- <Model10Impl.h> -----

#include <ElectricCarImpl.h>
#include <BatteryGen1.h>
#include <ElectricEngineGen1.h>

namespace eh {

class Model10Impl : public ElectricCarImpl
{
public:
    explicit ElectricHeroImpl( double charge )
        : battery_{charge}
        , engine_{}
    {}

    void start() override { engine_.start(); }
    void stop() override { engine_.stop(); }

    void drawPower() override { battery_.drawPower(); }
    void charge() override { battery_.charge(); }

    // ... more car or EH-specific functions

private:
    BatteryGen1 battery_;
    ElectricEngineGen1 engine_;
    // ... more EH-specific data members (wheels, drivetrain, ...)
};
```



Electric Hero: The Ultimate Electric Cars

```
// ... more car or EH-specific functions

private:
    BatteryGen1 battery_;
    ElectricEngineGen1 engine_;
    // ... more EH-specific data members (wheels, drivetrain, ...)
};

} // namespace eh

//----- <Model10.cpp> -----

#include <Model10.h>
#include <Model10Impl.h>

namespace eh {

Model10::Model10( /*maybe some battery and/or engine arguments*/ )
    : pimpl_{ std::make_unique<Model10Impl>( 100.0 ) }
{}

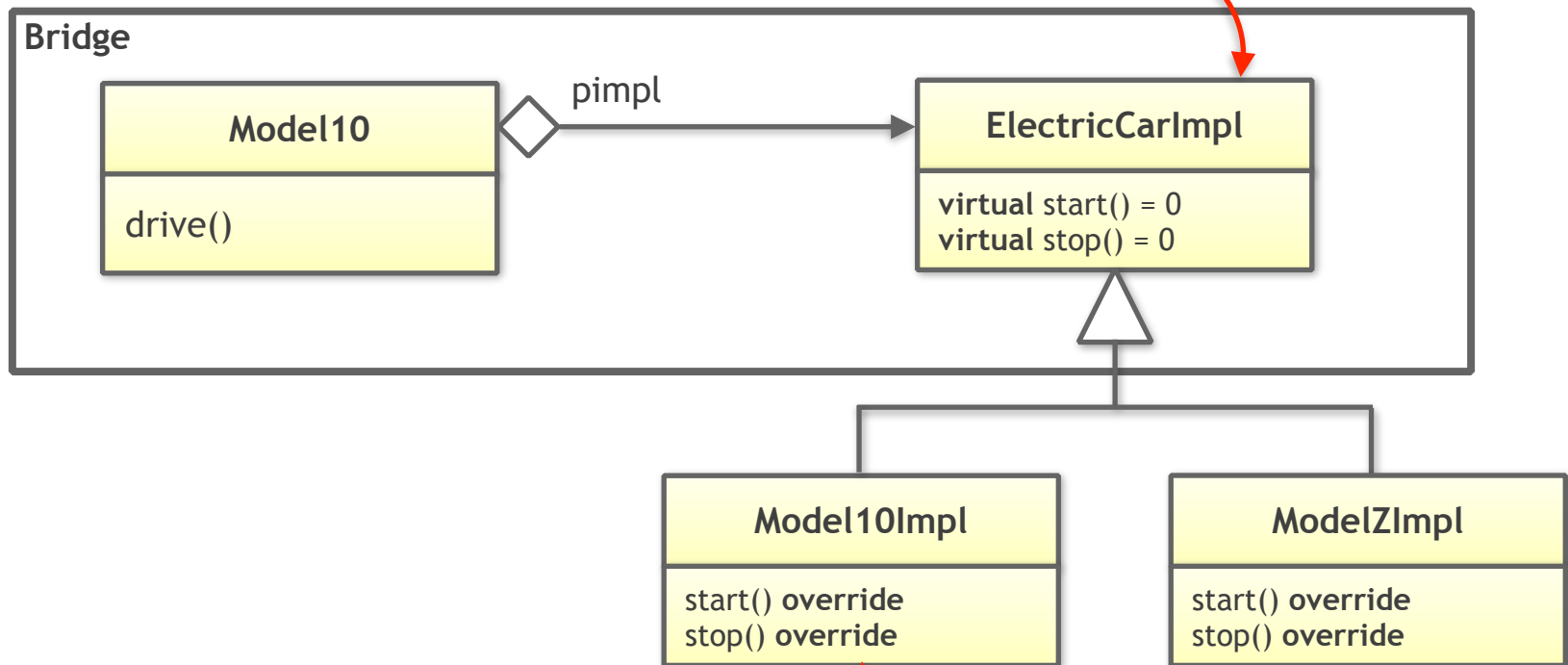
// ... Other 'ElectricHero' member functions, primarily using the battery
//      and engine.

} // namespace eh
```



The Classic Bridge Design Pattern

The implementation details of electric hero cars is the variation point. This aspect changes and is extracted and isolated; this fulfills the Single-Responsibility Principle (SRP)



Adding new implementations is the customization point: new implementations can be added without modifying any existing code; this fulfills the Open-Closed Principle (OCP)

Motivation

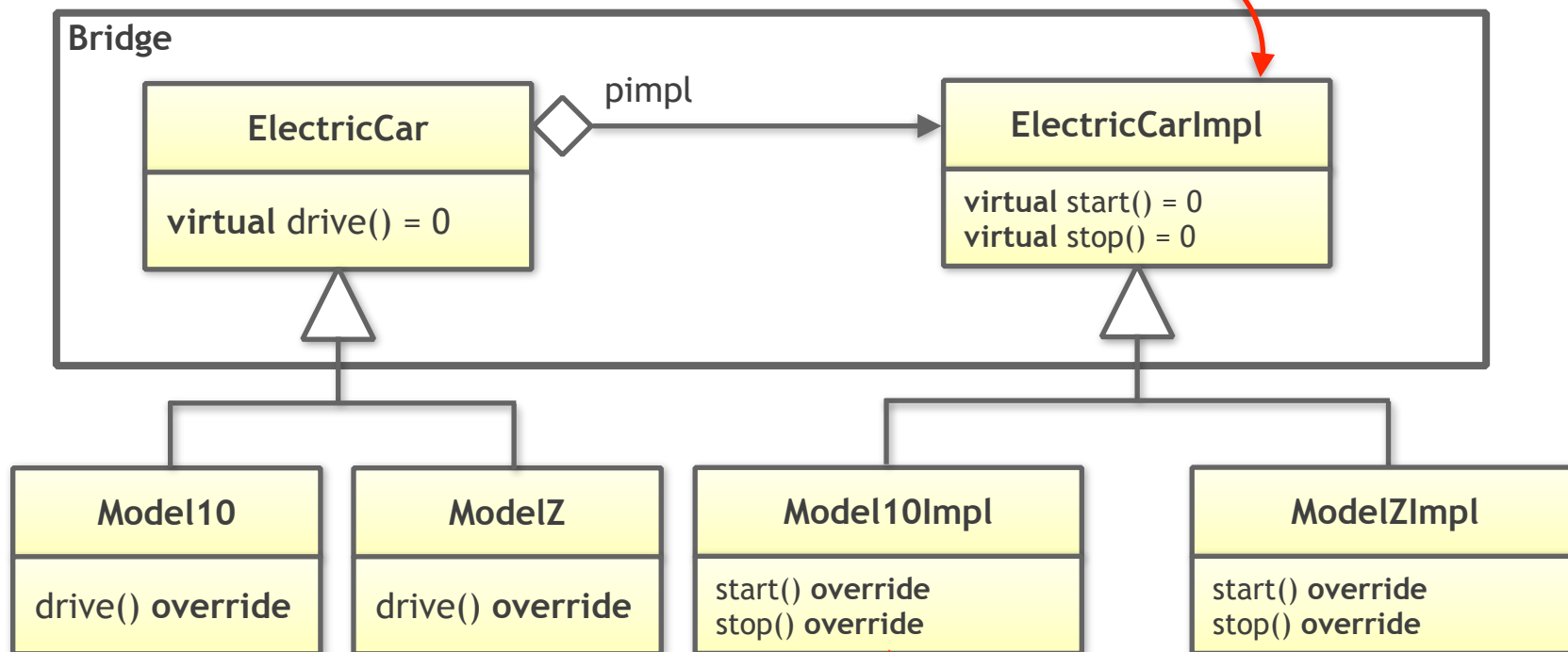


```
//----- <Car.h> -----  
  
#include <ElectricCarImpl.h>  
#include <memory>  
#include <utility>  
  
namespace eh {  
  
class ElectricCar  
{  
    protected:  
        explicit ElectricCar( std::unique_ptr<ElectricCarImpl> impl )  
            : pimpl_{ std::move(impl) }  
        {}  
  
    public:  
        virtual ~ElectricCar() = default;  
        virtual void drive() = 0;  
        // ... more car-specific functions  
  
    protected:  
        ElectricCarImpl* getImpl() { return pimpl_.get(); }  
  
    private:  
        std::unique_ptr<ElectricCarImpl> pimpl_; // Pointer-to-implementation  
};  
  
} // namespace eh
```

Introducing the **ElectricCar** abstraction enables the reuse of the Bridge for many different kinds of car.

The Classic Bridge Design Pattern

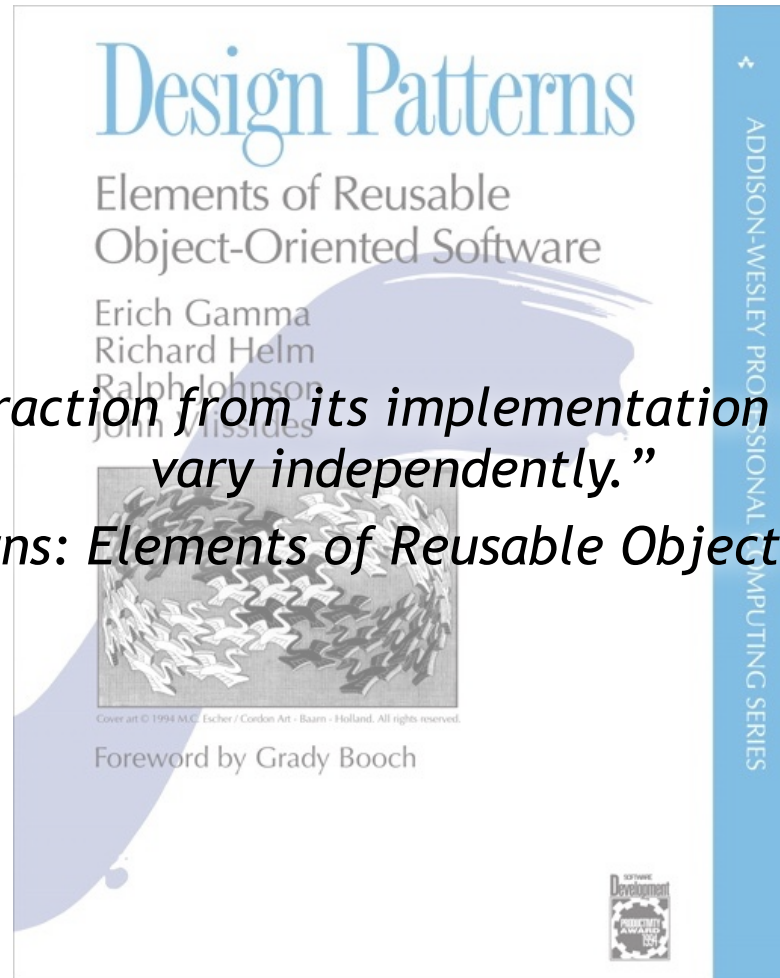
The implementation details of electric hero cars is the variation point. This aspect changes and is extracted and isolated; this fulfills the Single-Responsibility Principle (SRP)



Adding new implementations is the customization point: new implementations can be added without modifying any existing code; this fulfills the Open-Closed Principle (OCP)

The Intent

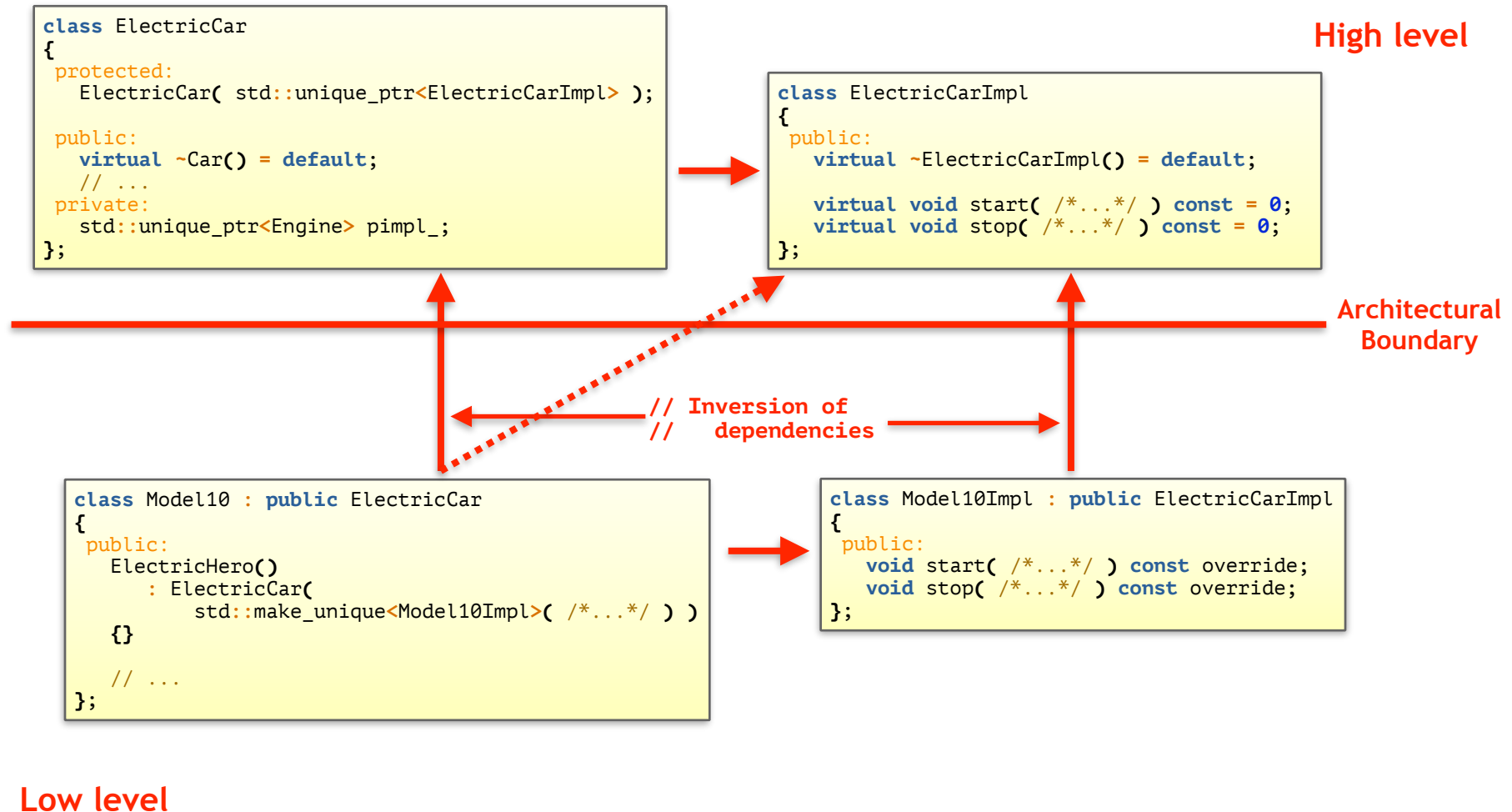
"Decouple an abstraction from its implementation so that the two can vary independently."
(GoF, *Design Patterns: Elements of Reusable Object-Oriented Software*)



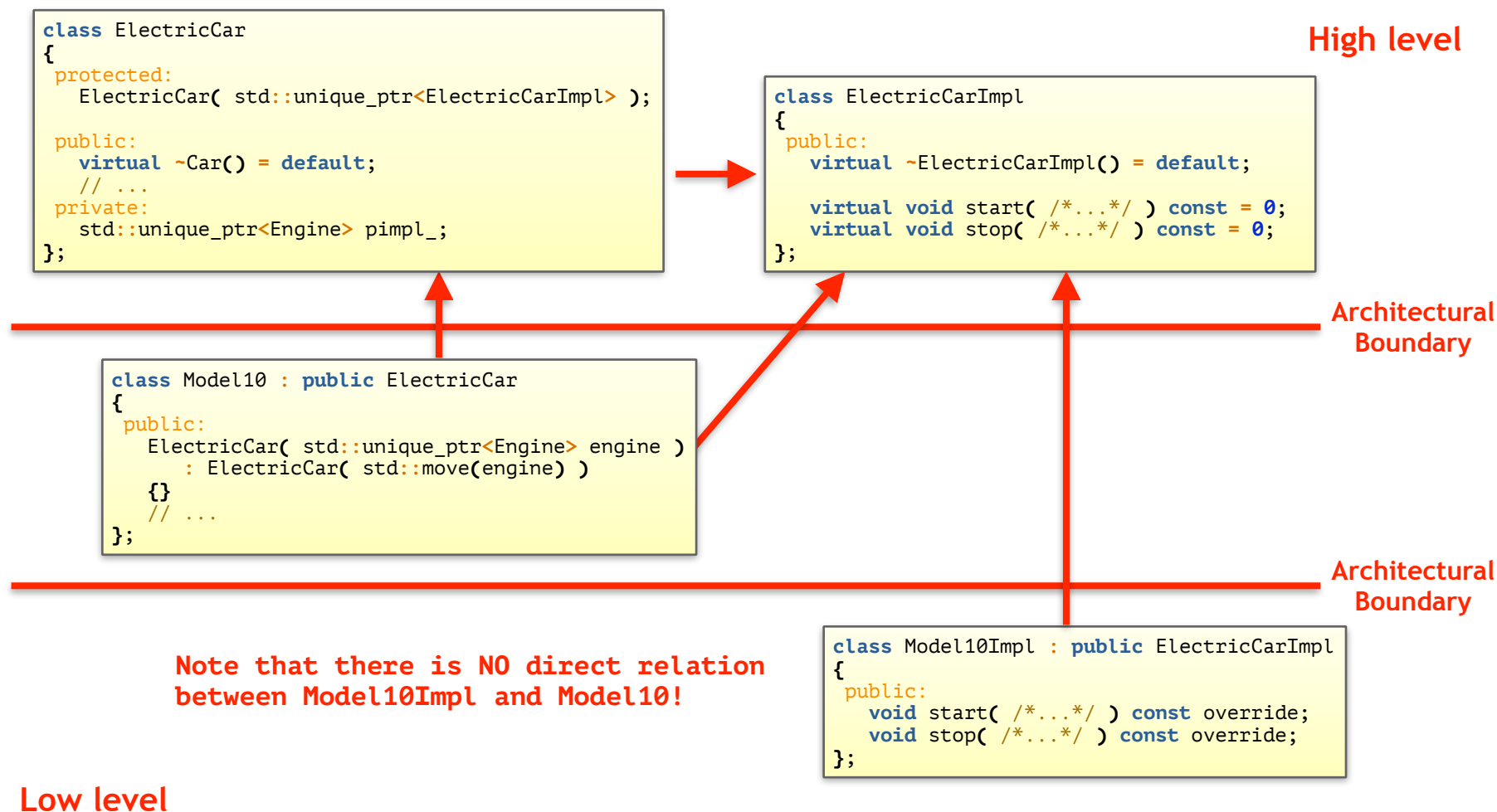
Programming Task

Task (C_Cpp_Software_Design/Bridge/Car_Bridge): Consider the given Car example, which demonstrates the Bridge design pattern. What would have to be changed to implement it in terms of the Strategy design pattern?

Dependency Structure of Bridge



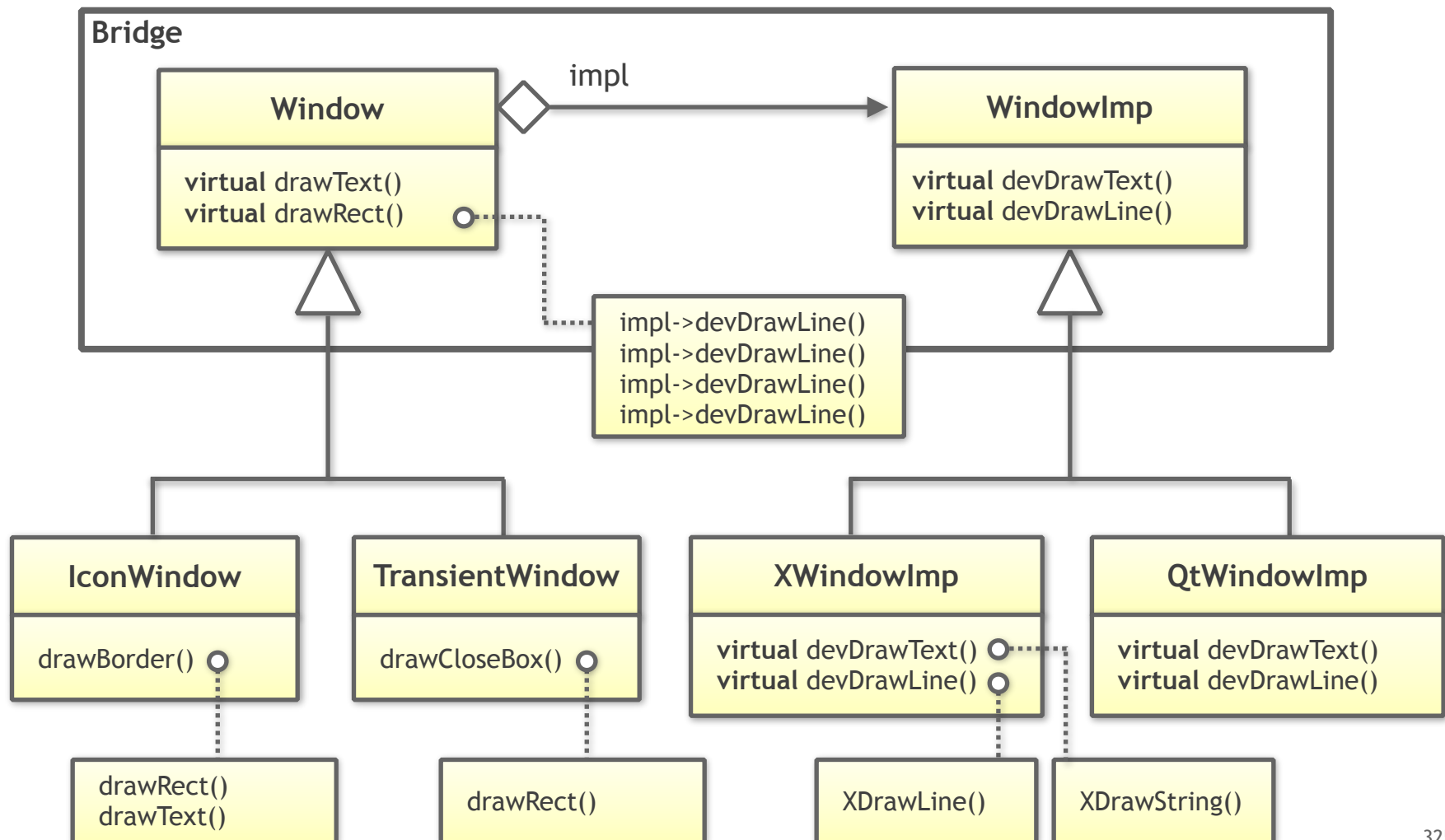
Dependency Structure of Strategy



Bridge vs. Strategy – Telltale Signs

<i>Bridge</i>	<i>Strategy</i>
Reduces physical dependencies	Reduces logical dependencies
No dependency injection	Always dependency injection
Internal customization point	External customization point
Does not improve testability	Improves testability

No Dependency Injection – Really?



No Dependency Injection – Really?

```
class WindowImp {  
public:  
    virtual void ImpTop() = 0;  
    virtual void ImpBottom() = 0;  
    virtual void SetExtent(const Point&) = 0;  
    virtual void SetOrigin(const Point&) = 0;  
  
    // ...  
    // lots more functions for drawer on windows  
};
```

```
class Window {  
public:  
    Window(View* contents);  
  
    virtual void DrawContents();  
    // ...  
  
private:  
    View* _contents; // the window's contents  
    WindowImp* _imp;  
};
```

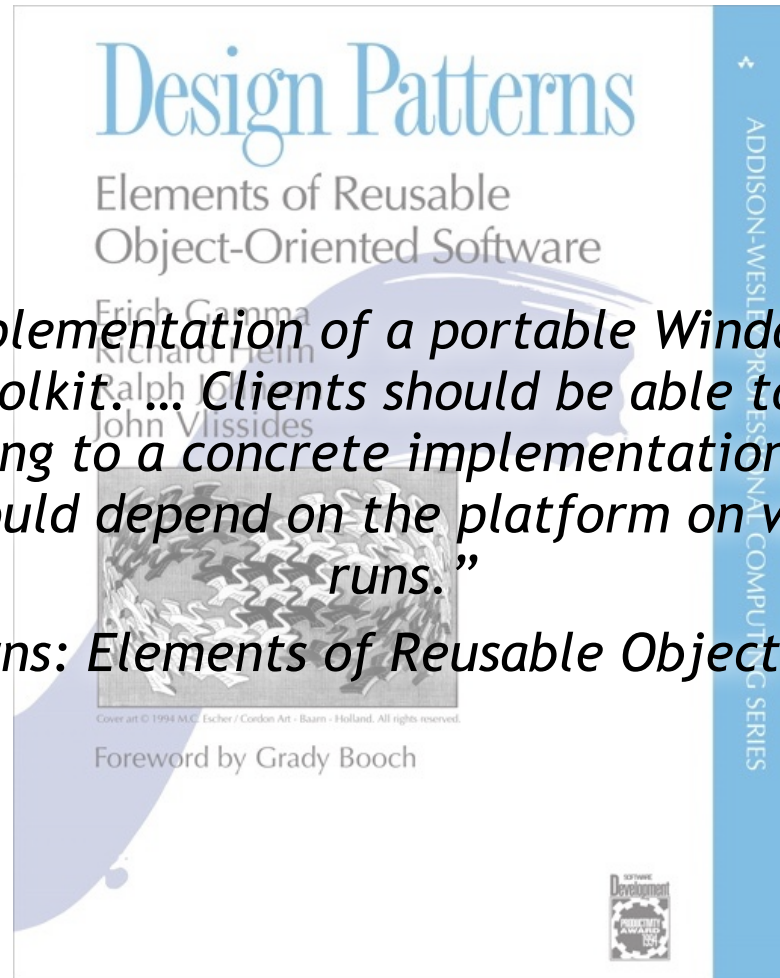
This is a Strategy (dependency injection, external customization point), ...

... this is a Bridge (no dependency injection, internal customization point).

No Dependency Injection — Really?

”Consider the implementation of a portable Window abstraction in a user interface toolkit. ... Clients should be able to create a window without committing to a concrete implementation. Only the window implementation should depend on the platform on which the application runs.”

(GoF, Design Patterns: Elements of Reusable Object-Oriented Software)



Programming Task

Task (C_Cpp_Software_Design/Bridge/Bridge)

Step 1: In the conceptual header file `<X.h>`, which `#include` directives could be immediately removed without ill effect? You may not make any changes other than removing or rewriting `#include` directives.

Step 2: What further `#includes` could be removed if we made some suitable changes, and how? This time, you may make changes to `X` as long as `X`'s base classes and its public interface remain unchanged; and current code that already uses `X` should not be affected beyond requiring a simple recompilation.

Step 3: Now you may make any changes to `X` as long as they don't change its public interface so that existing code that uses `X` is unaffected beyond requiring a simple recompilation. Again, note that the comments are important.

Bridge and Performance

The Pimpl idiom (simplest form of the Bridge Design Pattern) can be “used” to store rarely used data members and to reduce the size of a class in order to speed up performance:

```
struct Person
{
    std::string forename{ "Homer" };
    std::string surname{ "Simpson" };
    int age{ 42 };

    struct Pimpl {
        std::string address{ "712 Red Bark Lane" };
        std::string zip{ "89011" };
        std::string city{ "Henderson" };
        std::string state{ "Nevada" };
    };

    std::unique_ptr<Pimpl> pimpl{ new Pimpl{} };
};
```

Programming Task

Task (C_Cpp_Software_Design/Bridge/BridgedMembers): Copy-and-paste the following code into quick-bench.com. Benchmark the time to compute the total age of all persons contained in a `std::vector`.

Static Memory Instead of Dynamic Memory

```
//----- <X.h> -----  
  
#include <type_traits>  
// ...  
  
class X : public A  
{  
public:  
    X( C const& c );  
    // ...  
  
private:  
    struct Impl;  
  
    Impl* pimpl() { return reinterpret_cast<Impl*>(&buffer); }  
  
    static constexpr size_t buffersize = 100;  
    static constexpr size_t bufferalign = 16;  
  
    using Buffer = std::aligned_storage<buffersize,bufferalign>::type;  
    Buffer buffer_;  
};
```

```
//----- <X.cpp> -----
```

```
#include <X.h>
```

Static Memory Instead of Dynamic Memory

```
//----- <X.h> -----  
  
#include <type_traits>  
// ...  
  
class X : public A  
{  
public:  
    X( C const& c );  
    // ...  
  
private:  
    struct Impl;  
  
    Impl* pimpl() { return reinterpret_cast<Impl*>(&buffer); }  
  
    static constexpr size_t buffersize = 100;  
    static constexpr size_t bufferalign = 16;  
  
    using Buffer = std::aligned_storage<buffersize,bufferalign>::type;  
    Buffer buffer_;  
};
```

```
//----- <X.cpp> -----
```

```
#include <X.h>
```


Static Memory Instead of Dynamic Memory

```
//----- <X.h> -----  
  
#include <type_traits>  
// ...  
  
class X : public A  
{  
public:  
    X( C const& c );  
    // ...  
  
private:  
    struct Impl;  
  
    Impl* pimpl() { return reinterpret_cast<Impl*>(&buffer); }  
  
    static constexpr size_t buffersize = 100;  
    static constexpr size_t bufferalign = 16;  
  
    using Buffer = std::aligned_storage<buffersize,bufferalign>::type;  
    Buffer buffer_;  
};
```

```
//----- <X.cpp> -----
```

```
#include <X.h>
```

Static Memory Instead of Dynamic Memory

```
//----- <X.h> -----  
  
#include <type_traits>  
// ...  
  
class X : public A  
{  
public:  
    X( C const& c );  
    // ...  
  
private:  
    struct Impl;  
  
    Impl* pimpl() { return reinterpret_cast<Impl*>(&buffer); }  
  
    static constexpr size_t buffersize = 100;  
    static constexpr size_t bufferalign = 16;  
  
    using Buffer = std::aligned_storage<buffersize,bufferalign>::type;  
    Buffer buffer_;  
};
```

```
//----- <X.cpp> -----
```

```
#include <X.h>
```

Static Memory Instead of Dynamic Memory

```

static constexpr size_t bufferalign = 16;

using Buffer = std::aligned_storage<buffersize,bufferalign>::type;
Buffer buffer_;
};

//----- <X.cpp> -----

#include <X.h>
#include <memory>
// ...

struct Impl { /*...*/ };

X::X( C const& c )
: A{}
{
    static_assert( sizeof(Impl) <= sizeof(buffer_) );
    static_assert( alignof(Impl) <= alignof(Buffer) );

    std::construct_at( pimpl(), c );
    //::new (&buffer_) Impl{ c };
}

// ...

```

Static Memory Instead of Dynamic Memory

```
//----- <X.h> -----
```

```
#include <type_traits>
// ...
```

```
class X : public A
{
public:
    X( C const& c );
    // ...
```

```
private:
    struct Impl;
```

```
Impl* pimpl() { return reinterpret_cast<Impl*>(&buffer); }
```

```
static constexpr size_t buffersize = 100;
static constexpr size_t bufferalign = 16;
```

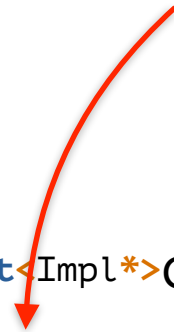
```
using Buffer = std::aligned_storage<buffersize,bufferalign>::type;
Buffer buffer_;
```

```
};
```

```
//----- <X.cpp> -----
```

```
#include <X.h>
```

Note that these two values can never change without breaking the ABI of class X!



Programming Task

Task (C_Cpp_Software_Design/Bridge/Pimpl): Refactor the given `eh::Model10` class by means of the "Pimpl Idiom":

- Replace the `ElectricEngineGen1` and `BatteryGen1` data members with an `Impl` pointer
- Remove all physical dependencies from the `<Model10.h>` header file
- Rework the `Model10` member functions in the source file
- Rethink/Refactor the special member functions of the `Model10` class

Programming Task

Task (C_Cpp_Software_Design/Bridge/FastPimpl): Refactor the given `eh::Model10` class by means of the "Fast Pimpl Idiom":

- Evaluate the required size and alignment of the 'Impl' class
- Store an instance of `Impl` in a byte buffer
- Introduce the necessary functionality to work with `Impl`
- Rework the `Model10` member functions in the source file
- Refactor the special member functions of the `Model10` class

Guidelines

Guideline: Use the Bridge design pattern to create a compilation firewall, i.e. to move dependencies from the header to the source file.

Guideline: Use the Bridge design pattern to make client code independent of platform specific code (no dependency injection).

Guideline: Use the pimpl idiom to store rarely used members and to reduce the size of a class in order to speed up performance.

Core Guideline I.27: For stable library ABI, consider the Pimpl idiom

2.11. Decorator

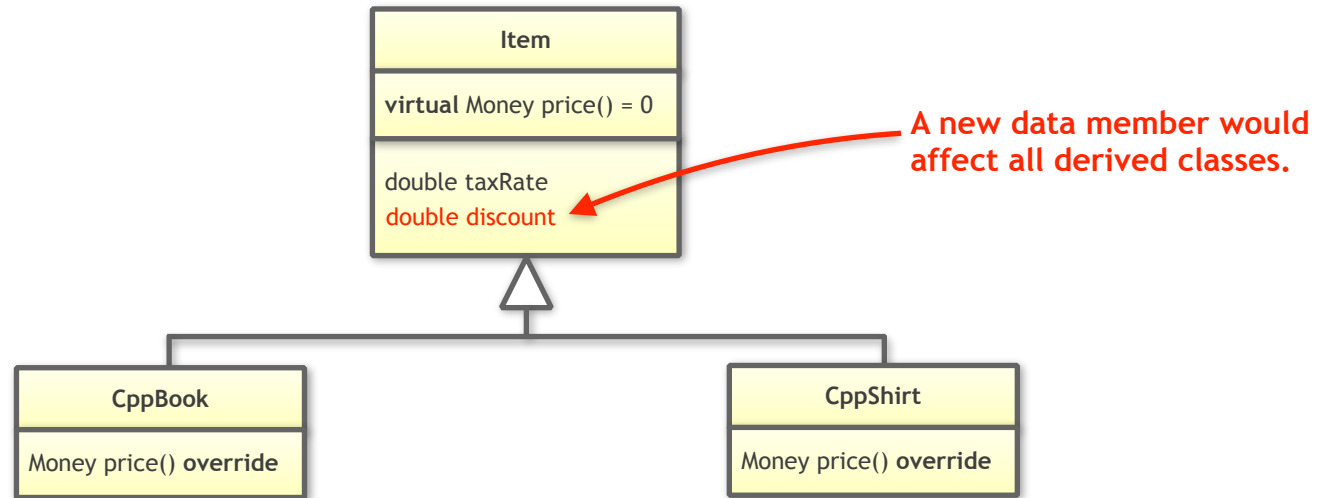
A New Programming Challenge



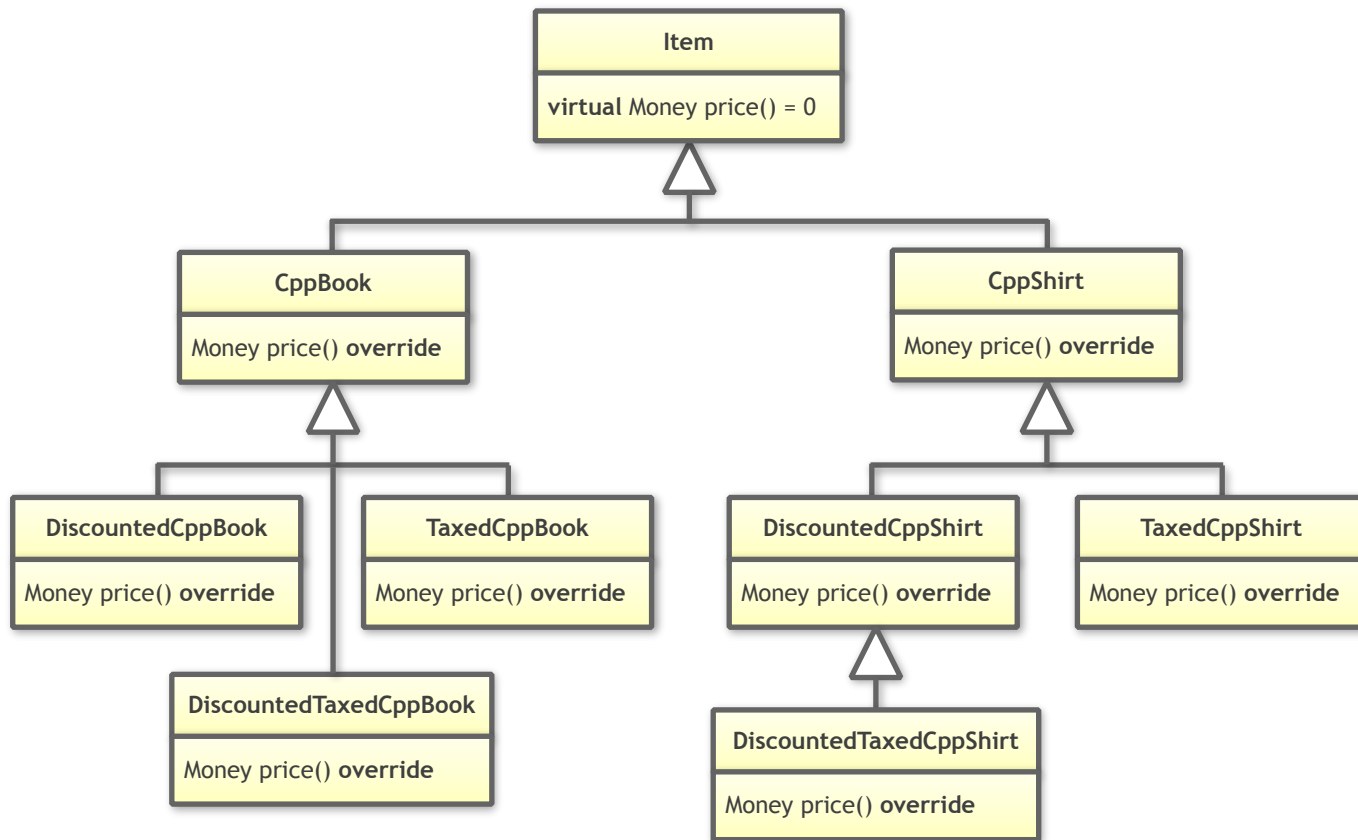
A New Programming Challenge



A Design Challenge



A Design Challenge



How do we deal with items that are both discounted and taxed?

How do we deal with the possible logic duplication?

A Design Challenge



*"Inheritance is Rarely the Answer."
(Andrew Hunt, David Thomas, The Pragmatic
Programmer)*



Guidelines

Guideline: Prefer containment (composition/aggregation) to inheritance.

A Word About Technical Debt

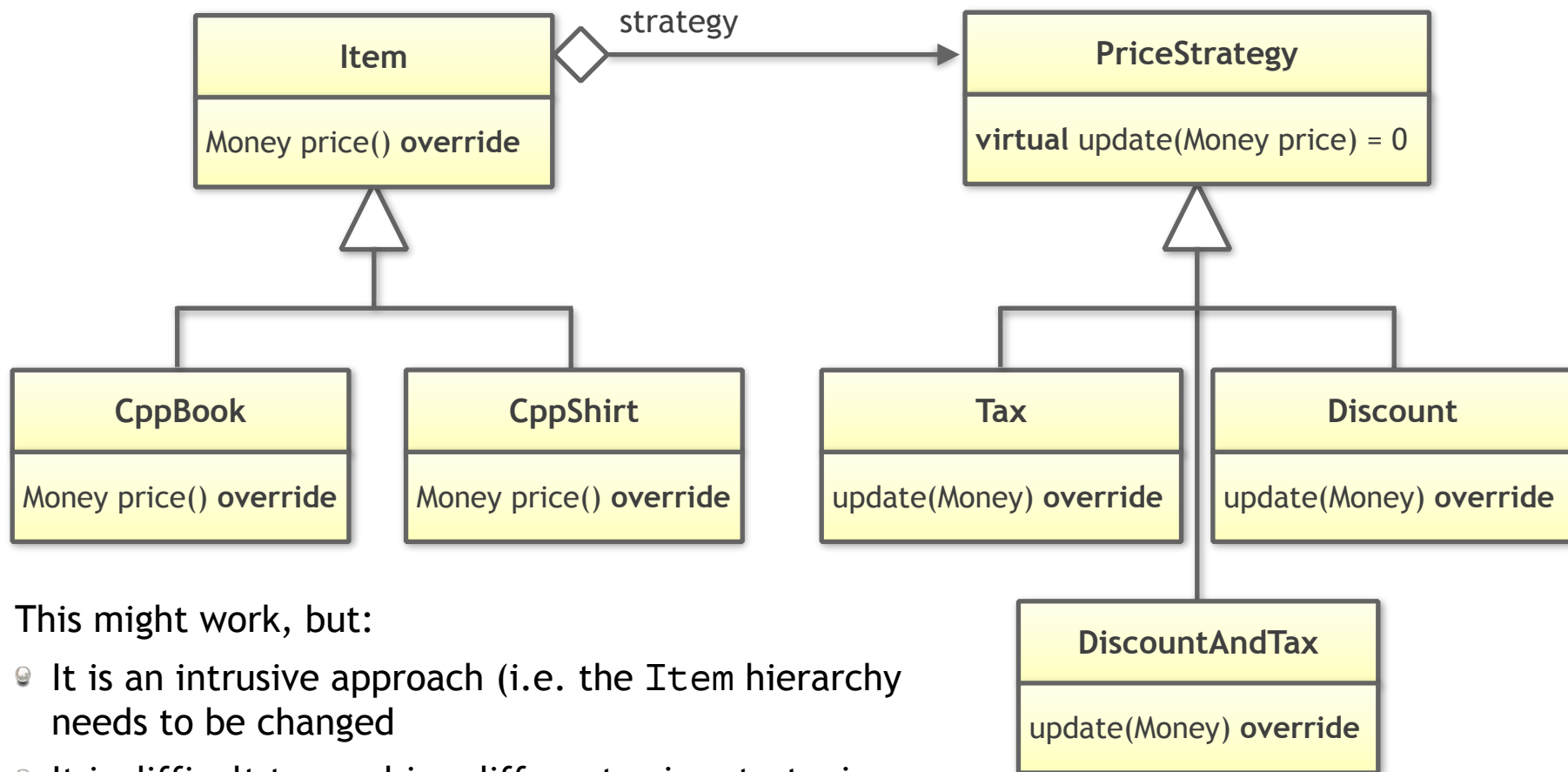
Despite the fact that we know that something isn't appropriate, we still sometimes do it because ...

- ... there is no better solution;
- ... there is no time (due to deadlines, etc.);
- ... we sometimes don't care.

This results in technical debt. Technical debt may sometimes help (for instance to get things done faster), but proves to be a burden in the long run. Therefore make sure that ...

- ... technical debt is documented;
- ... reducing technical debt is made part of regular maintenance.

A Better Solution: The Strategy Design Pattern

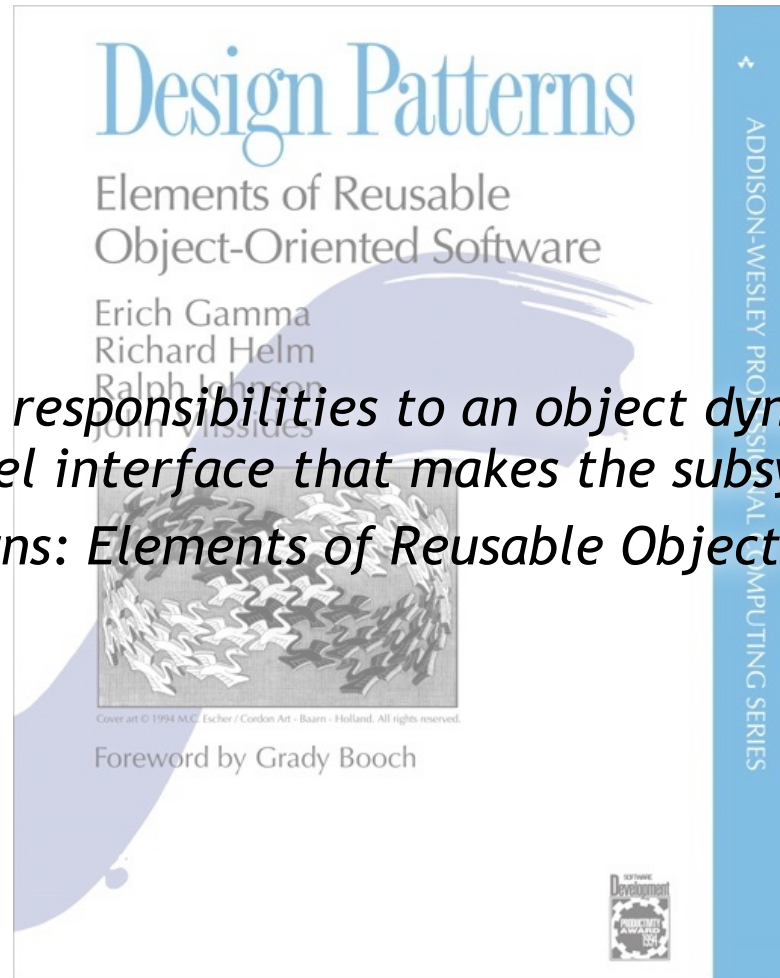


This might work, but:

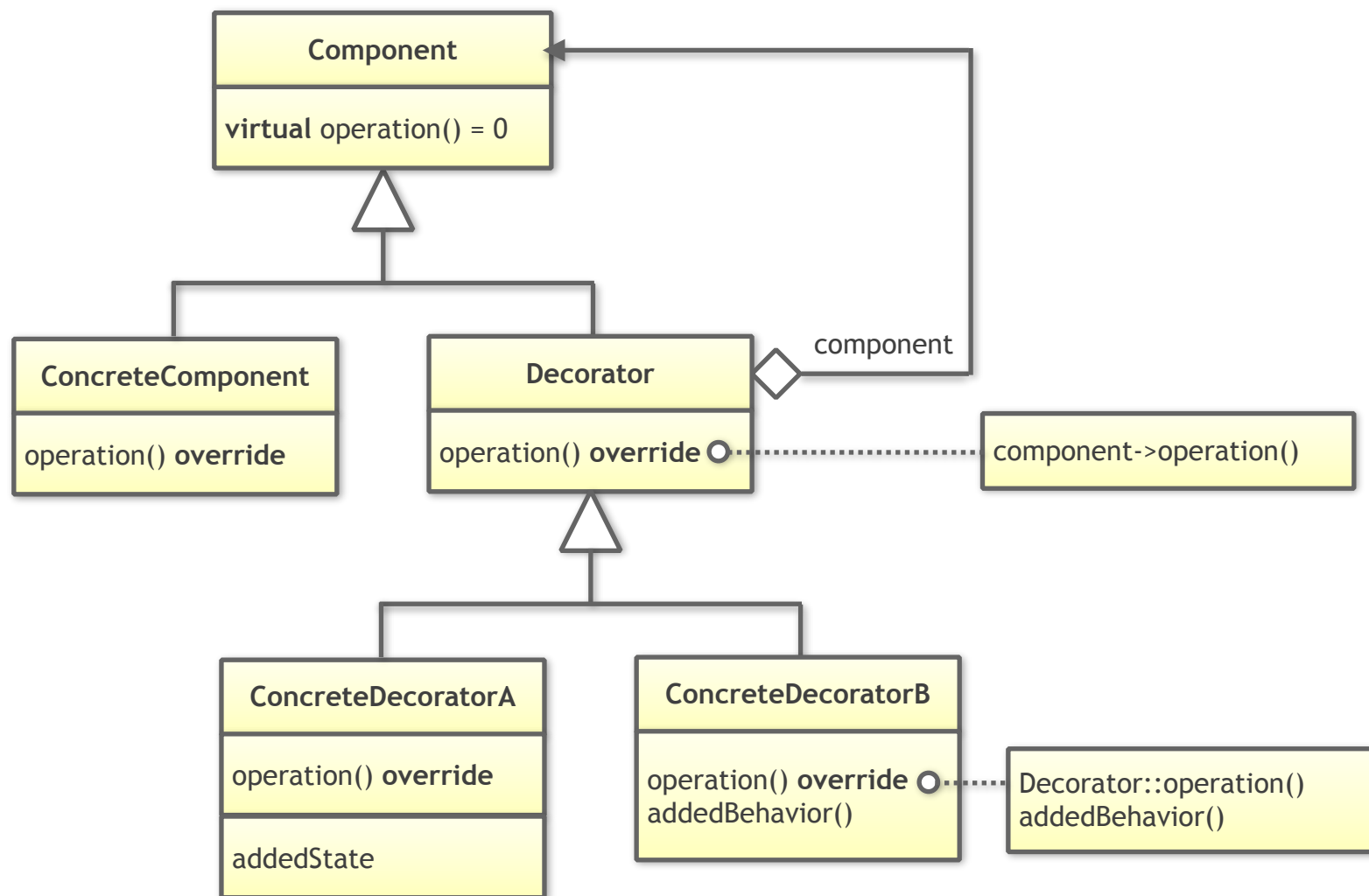
- It is an intrusive approach (i.e. the `Item` hierarchy needs to be changed)
- It is difficult to combine different price strategies

The Intent

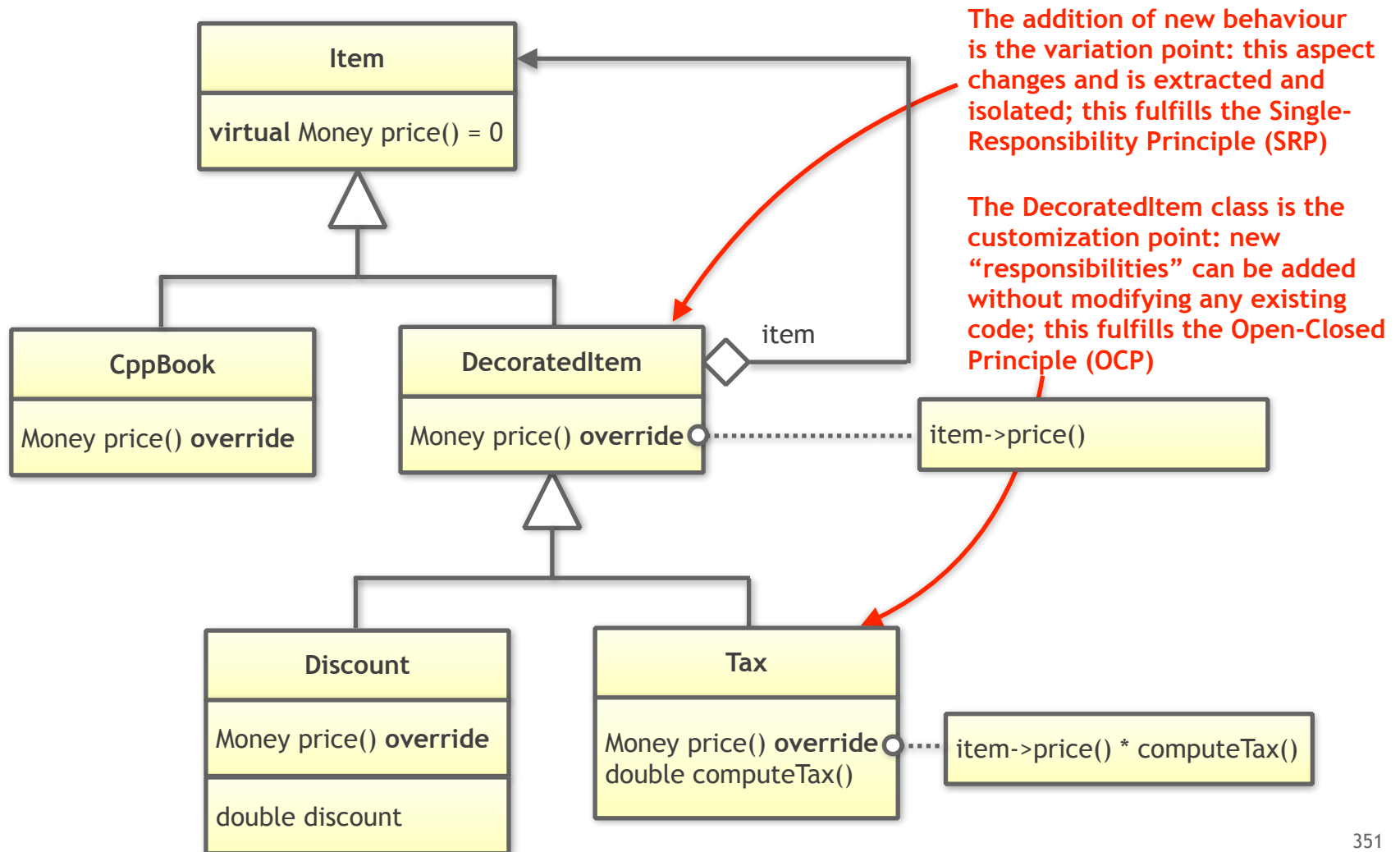
*“Attach additional responsibilities to an object dynamically. Decorator defines a higher-level interface that makes the subsystem easier to use.”
(GoF, Design Patterns: Elements of Reusable Object-Oriented Software)*



The Classic Decorator Design Pattern



The Classic Decorator Design Pattern



Decorator vs. Strategy - Telltale Signs

<i>Decorator</i>	<i>Strategy</i>
Wraps functionality around a class or function (no dependency injection)	Injects functionality into a class or function (always dependency injection)
Non-intrusive (can always be added)	Intrusive (requires refactoring)
Usually in the form of classes	Both in the form of functions/ classes

Programming Exercise: The C++ Fan Shop

Task (C_Cpp_Software_Design/Decorator/Decorator_CppFanShop): You are hired to rework the pricing system of the "C++ Fan Shop":

Step 1: Implement a `DecoratedItem`, which enables the Decorator design pattern for the items of the "C++ Fan Shop". Based on this, implement a `Discount` decorator, a `Coupon` decorator, and a `Tax` decorator.

Step 2: Introduce a quantity discount (2 items: 10%, 3 items: 15%, 4 items: 20%, more than four items: 25%)

Step 3: Refactor the classic Decorator solution by a value semantics based solution. Note that the general behavior should remain unchanged.

A Decorator-Based Solution

```
class Money;

constexpr Money operator*( Money money, T factor );
constexpr Money operator+( Money lhs, Money rhs );
constexpr Money operator-( Money lhs, Money rhs );
std::ostream& operator<<( std::ostream& os, Money money );
```

```
class Item
{
public:
    virtual ~Item() = default;
    virtual Money price() const = 0;
};
```

```
class ConferenceTicket
    : public Item
{
public:
    ConferenceTicket( std::string name, Money price )
        : name_{ std::move(name) }
        , price_{ price }
    {}

    std::string const& name() const noexcept {
        return name_;
    }

    Money price() const noexcept override {
```

A Decorator-Based Solution

```
class Money;
```

```
constexpr Money operator*( Money money, T factor );  
constexpr Money operator+( Money lhs, Money rhs );  
constexpr Money operator-( Money lhs, Money rhs );  
std::ostream& operator<<( std::ostream& os, Money money );
```

```
class Item  
{  
public:  
    virtual ~Item() = default;  
    virtual Money price() const = 0;  
};
```

```
class ConferenceTicket  
    : public Item  
{  
public:  
    ConferenceTicket( std::string name, Money price )  
        : name_{ std::move(name) }  
        , price_{ price }  
    {}  
  
    std::string const& name() const noexcept {  
        return name_;  
    }  
  
    Money price() const noexcept override {
```

A Decorator-Based Solution

```
class Money;

constexpr Money operator*( Money money, T factor );
constexpr Money operator+( Money lhs, Money rhs );
constexpr Money operator-( Money lhs, Money rhs );
std::ostream& operator<<( std::ostream& os, Money money );
```

```
class Item
{
public:
    virtual ~Item() = default;
    virtual Money price() const = 0;
};
```

```
class ConferenceTicket
    : public Item
{
public:
    ConferenceTicket( std::string name, Money price )
        : name_{ std::move(name) }
        , price_{ price }
    {}

    std::string const& name() const noexcept {
        return name_;
    }

    Money price() const noexcept override {
```


A Decorator-Based Solution

```
class ConferenceTicket
    : public Item
{
    public:
        ConferenceTicket( std::string name, Money price )
            : name_{ std::move(name) }
            , price_{ price }
        {}

        std::string const& name() const noexcept {
            return name_;
        }

        Money price() const noexcept override {
            return price_;
        }

    private:
        std::string name_{};
        Money price_{};
};

class CppBook : public Item { /*...*/ };
class CppShirt : public Item { /*...*/ };
class CppStandard : public Item { /*...*/ };
```

A Decorator-Based Solution

```
class CppBook : public Item { /*...*/ };  
class CppShirt : public Item { /*...*/ };  
class CppStandard : public Item { /*...*/ };
```

```
class DecoratedItem  
: public Item  
{  
public:  
    explicit DecoratedItem( std::unique_ptr<Item>&& item )  
        : item_( std::move(item) )  
    {}  
  
protected:  
    std::unique_ptr<Item> item_;  
};
```

```
class Discount  
: public DecoratedItem  
{  
public:  
    Discount( double discount, std::unique_ptr<Item>&& item )  
        : DecoratedItem( std::move(item) )  
        , factor_( 1.0 - discount )  
    {}  
  
    Money price() const override {
```

A Decorator-Based Solution

```
protected:
    std::unique_ptr<Item> item_;
};

class Discount
    : public DecoratedItem
{
public:
    Discount( double discount, std::unique_ptr<Item>&& item )
        : DecoratedItem( std::move(item) )
        , factor_( 1.0 - discount )
    {}

    Money price() const override {
        return item_->price() * factor_;
    }

private:
    double factor_;
};

class Coupon
    : public DecoratedItem
{
public:
    Coupon( Money value, std::unique_ptr<Item>&& item )
        : DecoratedItem( std::move(item) )
```

A Decorator-Based Solution

```
private:
    double factor_;
};

class Coupon
    : public DecoratedItem
{
public:
    Coupon( Money value, std::unique_ptr<Item>&& item )
        : DecoratedItem( std::move(item) )
        , value_( value )
    {}

    Money price() const override {
        if( item().price() < value_ )
            return Money{};
        else
            return item().price() - value_;
    }

private:
    Money value_;
};

class Tax
    : public DecoratedItem
{

```

A Decorator-Based Solution

```
private:
    Money value_;
};

class Tax
    : public DecoratedItem
{
public:
    Tax( double tax, std::unique_ptr<Item>&& item )
        : DecoratedItem( std::move(item) )
        , factor_( 1.0 + tax )
    {}

    Money price() const override {
        return item_->price() * factor_;
    }

private:
    double factor_;
};

class QuantityDiscount
    : public DecoratedItem
{
public:
    QuantityDiscount( std::unique_ptr<Item>&& item, size_t count )
        : DecoratedItem{ std::move(item) }
```

A Decorator-Based Solution

```
private:
    double factor_;
};

class QuantityDiscount
    : public DecoratedItem
{
public:
    QuantityDiscount( std::unique_ptr<Item>&& item, size_t count )
        : DecoratedItem{ std::move(item) }
        , count_{ count }
    {}

    Money price() const override {
        return item().price() * count_ * ( 1.0-discount() );
    }

private:
    double discount() const
    {
        // ...
    }

    size_t count_{};
};
```

```
class ShoppingCart
    : public Item
```

A Decorator-Based Solution

```
class ShoppingCart
    : public Item
{
public:
    template< typename... Items >
    ShoppingCart( Items&&... items )
        : items_{}
    {
        ( items_.push_back( std::forward<Items>(items) ), ... );
    }

    Money price() const override {
        // ...
    }

private:
    double discount() const
    {
        // ...
    }

    std::vector<std::unique_ptr<Item>> items_;
};
```

```
int main()
{
    std::unique_ptr<Item> item1(
```

A Decorator-Based Solution

```
int main()
{
    std::unique_ptr<Item> item1(
        std::make_unique<Tax>( 0.19,
            std::make_unique<Discount>( 0.2,
                std::make_unique<ConferenceTicket>( "CppCon", Money{999.0F} ) ) ) );

    std::unique_ptr<Item> item2(
        std::make_unique<Tax>( 0.07,
            std::make_unique<QuantityDiscount>( 3,
                std::make_unique<CppBook>( "Effective C++", Money{19.0F} ) ) ) );

    std::unique_ptr<Item> item3(
        std::make_unique<Tax>( 0.19,
            std::make_unique<CppStandard>( "ISO/IEC 14882:2020", Money{200.0F} ) ) );

    std::unique_ptr<Item> item4(
        std::make_unique<Tax>( 0.19,
            std::make_unique<Coupon>( Money{5.0F},
                std::make_unique<CppShirt>( Color::red, Money{15.0F} ) ) ) );

    std::unique_ptr<Item> item5(
        std::make_unique<ShoppingCart>(
            std::move(item1), std::move(item3), std::move(item4) ) );

    std::cout << "\n"
        << " Total price of item2 = " << item2->price() << "\n"
        << " Total price of item5 = " << item5->price() << "\n"
        << " ";
```


A Decorator-Based Solution

```
std::make_unique<Tax>( 0.19,  
    std::make_unique<Discount>( 0.2,  
        std::make_unique<ConferenceTicket>( "CppCon", Money{999.0F} ) ) ) );  
  
std::unique_ptr<Item> item2(  
    std::make_unique<Tax>( 0.07,  
        std::make_unique<QuantityDiscount>( 3,  
            std::make_unique<CppBook>( "Effective C++", Money{19.0F} ) ) ) );  
  
std::unique_ptr<Item> item3(  
    std::make_unique<Tax>( 0.19,  
        std::make_unique<CppStandard>( "ISO/IEC 14882:2020", Money{200.0F} ) ) );  
  
std::unique_ptr<Item> item4(  
    std::make_unique<Tax>( 0.19,  
        std::make_unique<Coupon>( Money{5.0F},  
            std::make_unique<CppShirt>( Color::red, Money{15.0F} ) ) ) );  
  
std::unique_ptr<Item> item5(  
    std::make_unique<ShoppingCart>(  
        std::move(item1), std::move(item3), std::move(item4) ) );  
  
std::cout << "\n"  
    << " Total price of item2 = " << item2->price() << "\n"  
    << " Total price of item5 = " << item5->price() << "\n"  
    << "\n";  
  
return EXIT_SUCCESS;  
}
```

The Classic Decorator Design Pattern

- We have to use virtual functions (no inlining)
- We have to use the heap for each single object
- We have to manage the lifetime via an `std::unique_ptr`
- In every decorator we have to deal with object lifetimes
- Due to pointers we have to deal with potential `nullptr`

A “Modern C++” Solution

```
class Money;
```

```
constexpr Money operator*( Money money, T factor );  
constexpr Money operator+( Money lhs, Money rhs );  
constexpr Money operator-( Money lhs, Money rhs );  
std::ostream& operator<<( std::ostream& os, Money money );
```

```
class Item
```

```
{  
    public:  
        template< typename T >  
        Item( T&& item )  
            : pimpl_( std::make_unique<Model<std::remove_reference_t<T>>>>(  
                    std::forward<T>(item) ) )  
        {}  
}
```

```
Item( const Item& item ) : pimpl_( item.pimpl_->clone() ) {}  
Item( Item&& ) = default;
```

```
~Item() = default;
```

```
Item& operator=( Item const& item ) {  
    pimpl_ = item.pimpl_->clone(); return *this; }  
Item& operator=( Item&& item ) = default;
```

```
Money price() const { return pimpl_->price(); }
```

```
private:
```

A “Modern C++” Solution

```
class Money;
```

```
constexpr Money operator*( Money money, T factor );  
constexpr Money operator+( Money lhs, Money rhs );  
constexpr Money operator-( Money lhs, Money rhs );  
std::ostream& operator<<( std::ostream& os, Money money );
```

```
class Item  
{  
public:  
    template< typename T >  
    Item( T&& item )  
        : pimpl_( std::make_unique<Model<std::remove_reference_t<T>>>>(  
                    std::forward<T>(item) ) )  
    {}  
  
    Item( const Item& item ) : pimpl_( item.pimpl_->clone() ) {}  
    Item( Item&& ) = default;  
  
    ~Item() = default;  
  
    Item& operator=( Item const& item ) {  
        pimpl_ = item.pimpl_->clone(); return *this; }  
    Item& operator=( Item&& item ) = default;  
  
    Money price() const { return pimpl_->price(); }  
  
private:
```

A “Modern C++” Solution

```
class Item
{
public:
    template< typename T >
    Item( T&& item )
        : pimpl_( std::make_unique<Model<std::remove_reference_t<T>>>>(
                    std::forward<T>(item) ) )
    {}

    Item( const Item& item ) : pimpl_( item.pimpl_->clone() ) {}
    Item( Item&& ) = default;

    ~Item() = default;

    Item& operator=( Item const& item ) {
        pimpl_ = item.pimpl_->clone(); return *this; }
    Item& operator=( Item&& item ) = default;

    Money price() const { return pimpl_->price(); }

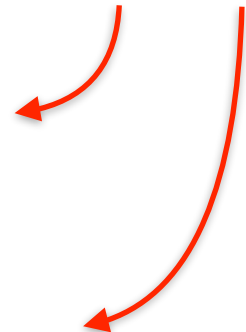
private:
    struct Concept
    {
        virtual ~Concept() = default;
        virtual Money price() const = 0;
        virtual std::unique_ptr<Concept> clone() const = 0;
    };

    template< typename T >
    struct Model final : public Concept
```

A “Modern C++” Solution

```
Item& operator=( Item const& item ) {  
    pimpl_ = item.pimpl_->clone(); return *this; }  
Item& operator=( Item&& item ) = default;  
  
Money price() const { return pimpl_->price(); }  
  
private:  
    struct Concept  
    {  
        virtual ~Concept() = default;  
        virtual Money price() const = 0;  
        virtual std::unique_ptr<Concept> clone() const = 0;  
    };  
  
    template< typename T >  
    struct Model final : public Concept  
    {  
        explicit Model( T const& item ) : item_( item ) {}  
        explicit Model( T&& item ) : item_( std::move(item) ) {}  
        Money price() const final { return item_.price(); }  
        std::unique_ptr<Concept> clone() const final {  
            return std::make_unique<Model<T>>(*this);  
        }  
        T item_;  
    };  
  
    std::unique_ptr<Concept> pimpl_;  
};
```

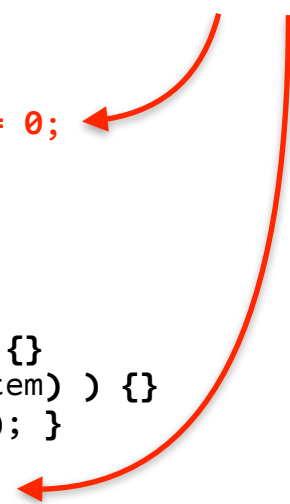
The External Polymorphism
design pattern



A “Modern C++” Solution

```
Item& operator=( Item const& item ) {  
    pimpl_ = item.pimpl_->clone(); return *this; }  
Item& operator=( Item&& item ) = default;  
  
Money price() const { return pimpl_->price(); }  
  
private:  
    struct Concept  
    {  
        virtual ~Concept() = default;  
        virtual Money price() const = 0;  
        virtual std::unique_ptr<Concept> clone() const = 0;  
    };  
  
    template< typename T >  
    struct Model final : public Concept  
    {  
        explicit Model( T const& item ) : item_( item ) {}  
        explicit Model( T&& item ) : item_( std::move(item) ) {}  
        Money price() const final { return item_.price(); }  
        std::unique_ptr<Concept> clone() const final {  
            return std::make_unique<Model<T>>(*this);  
        }  
        T item_;  
    };  
  
    std::unique_ptr<Concept> pimpl_;  
};
```

The Prototype
design pattern



A “Modern C++” Solution

```
Item& operator=( Item const& item ) {
    pimpl_ = item.pimpl_->clone(); return *this; }
Item& operator=( Item&& item ) = default;

Money price() const { return pimpl_->price(); }

private:
struct Concept
{
    virtual ~Concept() = default;
    virtual Money price() const = 0;
    virtual std::unique_ptr<Concept> clone() const = 0;
};

template< typename T >
struct Model final : public Concept
{
    explicit Model( T const& item ) : item_( item ) {}
    explicit Model( T&& item ) : item_( std::move(item) ) {}
    Money price() const final { return item_.price(); }
    std::unique_ptr<Concept> clone() const final {
        return std::make_unique<Model<T>>(*this);
    }
    T item_;
};

std::unique_ptr<Concept> pimpl_;
```

 The Bridge design pattern

A “Modern C++” Solution

```
class ConferenceTicket
    : public Item
{
public:
    ConferenceTicket( Money price, std::string name )
        : name_{ std::move(name) }
        , price_{ price }
    {}

    std::string const& name() const noexcept {
        return name_;
    }

    Money price() const noexcept override {
        return price_;
    }

private:
    std::string name_{};
    Money price_{};
};

class CppBook : public Item { /* ... */ };
class CppStandard : public Item { /* ... */ };
class CppShirt : public Item { /* ... */ };
```

A “Modern C++” Solution

```
class CppBook : public Item { /* ... */ };  
class CppStandard : public Item { /* ... */ };  
class CppShirt : public Item { /* ... */ };
```

```
class DecoratedItem  
{  
public:  
    explicit DecoratedItem( Item item )  
        : item_{ std::move(item) }  
    {}  
  
protected:  
    Item&      item()      { return item_; }  
    Item const& item() const { return item_; }  
  
private:  
    Item item_;  
};
```

```
class Discount  
    : public DecoratedItem  
{  
public:  
    Discount( double discount, Item item )  
        : Decorated( std::move(item) )  
        , factor_( 1.0 - discount )
```

A “Modern C++” Solution

```
private:
    Item item_;
};

class Discount
    : public DecoratedItem
{
public:
    Discount( double discount, Item item )
        : Decorated( std::move(item) )
        , factor_( 1.0 - discount )
    {}

    Money price() const {
        return item().price() * factor_;
    }

private:
    double factor_;
};
```

```
class Tax
    : public DecoratedItem
{
public:
    Tax( double tax, Item item )
        : Decorated( std::move(item) )
```

A “Modern C++” Solution

```
private:
    double factor_;
};

class Tax
    : public DecoratedItem
{
public:
    Tax( double tax, Item item )
        : Decorated( std::move(item) )
        , factor_( 1.0 + tax )
    {}

    Money price() const {
        return item_.price() * factor_;
    }

private:
    double factor_;
};

class ShoppingCart
{
public:
    template< typename... Items >
    ShoppingCart( Items&&... items )
        : items_{}

```

A “Modern C++” Solution

```
class ShoppingCart
{
public:
    template< typename... Items >
    ShoppingCart( Items&&... items )
        : items_{}
    {
        ( items_.push_back( std::forward<Items>(items) ), ... );
    }

    Money price() const {
        // ...
    }

private:
    double discount() const
    {
        // ...
    }

    std::vector<Item> items_;
};
```

```
int main()
{
    Item item1(
```

A “Modern C++” Solution

```
int main()
{
    Item item1(
        Tax( 0.19,
            Discount( 0.2,
                ConferenceTicket{ "CppCon", Money{999.0F} } ) ) );
    Item item2(
        Tax( 0.07,
            QuantityDiscount( 3,
                CppBook{ "Effective C++", Money{19.0F} } ) ) );
    Item item3(
        Tax( 0.19,
            CppStandard{ "ISO/IEC 14882:2020", Money{200.0F} } ) );
    Item item4(
        Tax( 0.19,
            Coupon( Money{5.0F},
                CppShirt( Color::red, Money{15.0F} ) ) ) );

    Item item5( ShoppingCart( item1, item3, item4 ) );

    std::cout << "\n"
        << " Total price of item1 = " << item1.price() << "\n"
        << " Total price of item2 = " << item2.price() << "\n"
        << " Total price of item3 = " << item3.price() << "\n"
        << " Total price of item4 = " << item4.price() << "\n"
        << " Total price of item5 = " << item5.price() << "\n"
        << "\n";

    return EXIT_SUCCESS;
}
```

The Classic Decorator Design Pattern

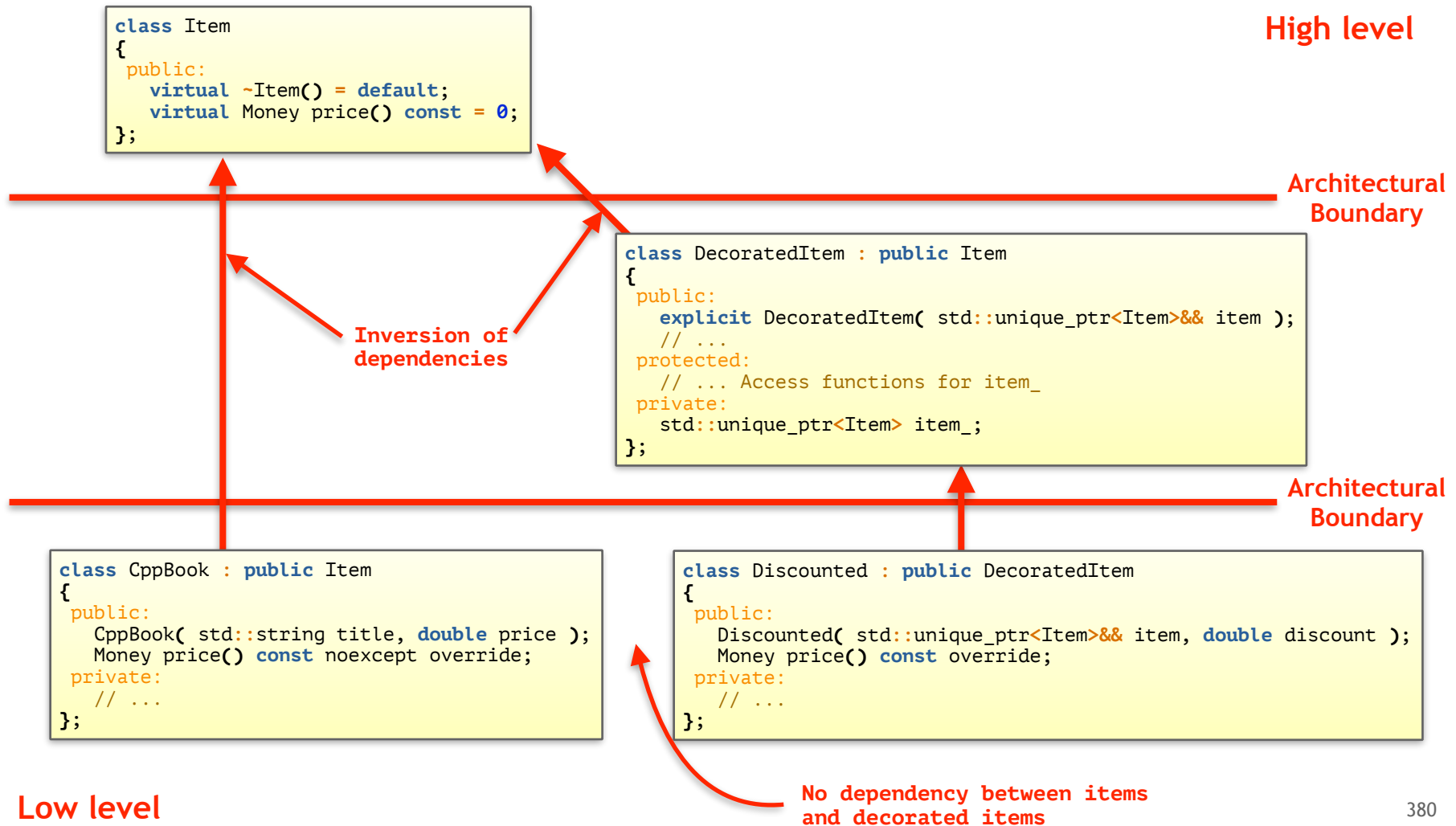
The classic Decorator design pattern ...

- ... requires a base class (dependency);
- ... promotes heap allocation;
- ... requires memory management;
- ... uses many pointers.

Using type erasure instead of the classic Decorator design pattern ...

- ... simplifies code;
- ... facilitates comprehension;
- ... reduces dependencies;
- ... removes all pointers.

Dependency Structure



Alternative 1: A Policy-Based Decorator

```
template< typename StoragePolicy >
class Item;

using DynamicItem = Item<DynamicStorage>;
using StaticItem  = Item<StaticStorage<40UL,8UL>>;
using SmallItem   = Item<SmallStorage<40UL,8UL>>;

// ...
```

Alternative 1: A Policy-Based Decorator

```
// ...  
  
struct DynamicStorage  
{  
    template< typename T, typename... Args >  
    T* create( Args&&... args ) const  
    {  
        return new T( std::forward<Args>( args )... );  
    }  
  
    template< typename T >  
    void destroy( T* ptr ) const noexcept  
    {  
        delete ptr;  
    }  
};  
  
// ...
```

Alternative 1: A Policy-Based Decorator

```
// ...
```

```
template< size_t Capacity, size_t Alignment >
struct StaticStorage
{
    template< typename T, typename... Args >
    T* create( Args&&... args ) const
    {
        return new (static_cast<void*>(buffer_.data()))
            T( std::forward<Args>( args )... );
    }

    template< typename T >
    void destroy( T* ptr ) const noexcept
    {
        ptr->~T();
    }

    alignas(Alignment) std::array<std::byte,Capacity> buffer_;
};
```

```
// ...
```

Alternative 2: A Template-Based Decorator

```
template< typename T >
concept PricedItem =
    requires ( const T item ) {
        { item.price() } -> std::same_as<Money>;
    };

template< double discount, PricedItem Item >
class Discount
{
public:
    template< typename... Args >
        std::constructible_from<Item,Args...>
    explicit Discount( Args&&... args )
        : item_{ std::forward<Args>(args)... }
    {}

    Money price() const {
        return item_.price() * ( 1.0 - discount );
    }

private:
    Item item_;
};
```

Alternative 2: A Template-Based Decorator

```
template< typename T >
concept PricedItem =
    // ...

template< double discount, PricedItem Item >
class Discount { /*...*/ };

template< double tax, PricedItem Item >
class Tax : private Item
{
public:
    template< typename... Args >
        std::constructible_from<Item,Args...>
    explicit Tax( Args&&... args )
        : Item{ std::forward<Args>(args)... }
    {}

    Money price() const {
        return Item::price() * ( 1.0 + tax );
    }
};
```

Alternative 2: A Template-Based Decorator

```
class ConferenceTicket
{
public:
    ConferenceTicket( std::string name, Money price )
        : name_{ std::move(name) }
        , price_{ price }
    {}

    const std::string& name() const noexcept { return name_; }
    Money price() const noexcept { return price_; }

private:
    std::string name_{};
    Money price_{};
};

int main()
{
    // ...
    Tax<0.15,Discount<0.2,ConferenceTicket>> item{ "CppCon", Money{999.0F} };

    item.price();
    // ...
}
```

Alternative 2: A Template-Based Decorator

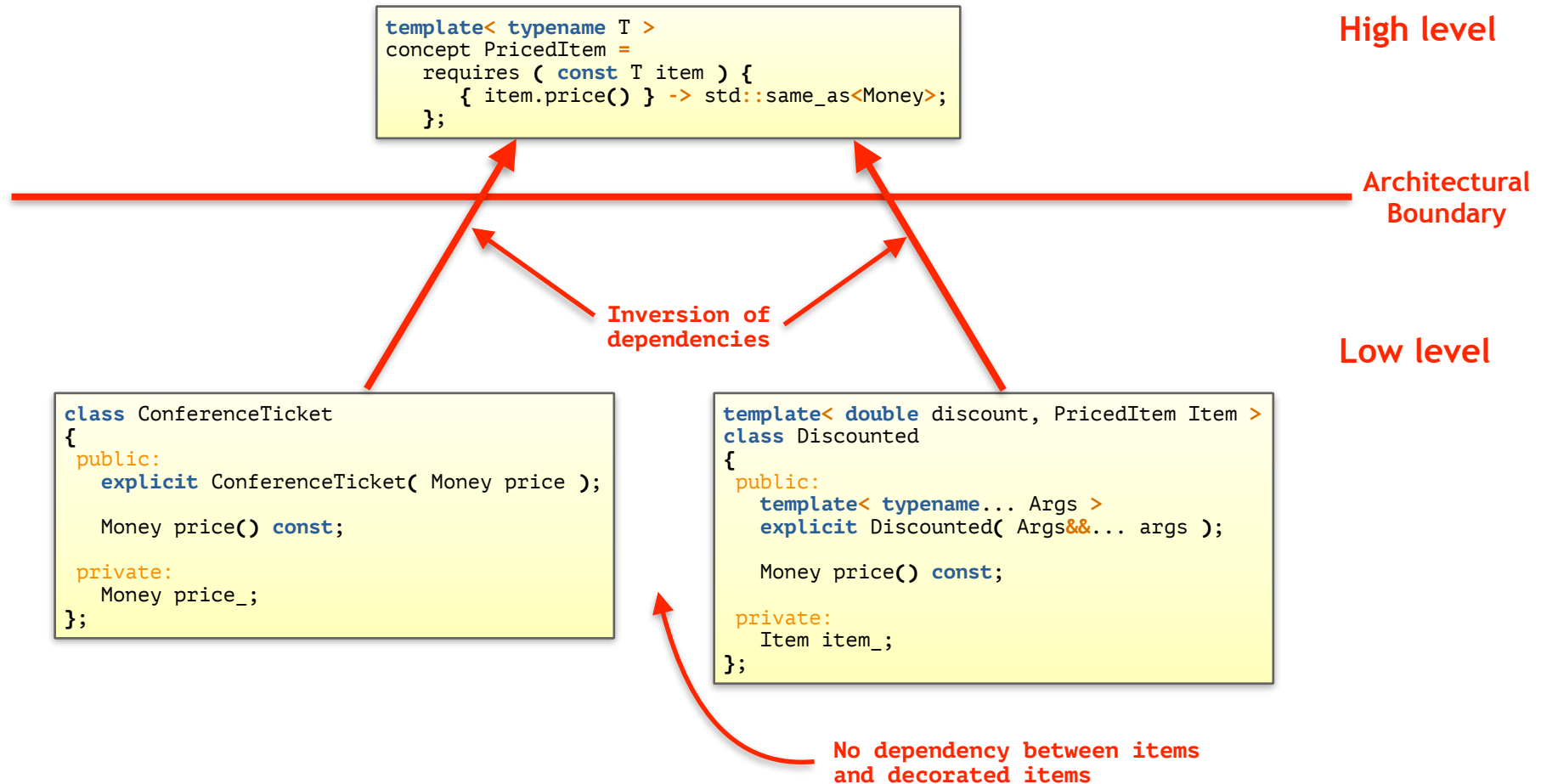
```
template< double discount, PricedItem Item >
Discount<discount,Item> applyDiscount( Item item )
{
    return Discount<discount,Item>{ item };
}
```

```
template< double tax, PricedItem Item >
Tax<tax,Item> applyTax( Item item )
{
    return Tax<tax,Item>{ item };
}
```

```
int main()
{
    // ...
    auto item2 =
        applyTax<0.15>(
            applyDiscount<0.2>( ConferenceTicket{ "CppCon", Money{999.0F} } ) );

    item2.price();
    // ...
}
```

Alternative 2: A Template-Based Decorator



Alternative 2: A Template-Based Decorator

Task (C_Cpp_Software_Design/Decorator/Decorator_Static): Discuss the advantages and disadvantages of the given compile-time decorator.

Alternative 3: A CRTP-Based Decorator

```
template< typename Derived >
class Item
{
protected:
    ~Item() = default;

public:
    Money price() const { return derived().price(); }

    Derived& derived() {
        return static_cast<Derived&>(*this);
    }
    Derived const& derived() const {
        return static_cast<Derived const&>(*this);
    }
};
```

Alternative 3: A CRTP-Based Decorator

```
class ConferenceTicket
: public Item<ConferenceTicket>
{
public:
    ConferenceTicket( std::string name, Money price )
        : name_{ std::move(name) }
        , price_{ price }
    {}

    std::string const& name() const noexcept { return name_; }

    Money price() const noexcept { return price_; }

private:
    std::string name_{};
    Money price_{};
};
```

Alternative 3: A CRTP-Based Decorator

```
template< typename ItemT >
class Discount
    : public Item<Discount<ItemT>>
{
public:
    template< typename... Args >
    Discount( double discount, Args&&... args )
        : item_( std::forward<Args>(args)... )
        , factor_( 1.0 - discount )
    {
        if( discount < 0.0 || discount > 1.0 ) {
            throw std::invalid_argument( "Invalid discount value" );
        }
    }

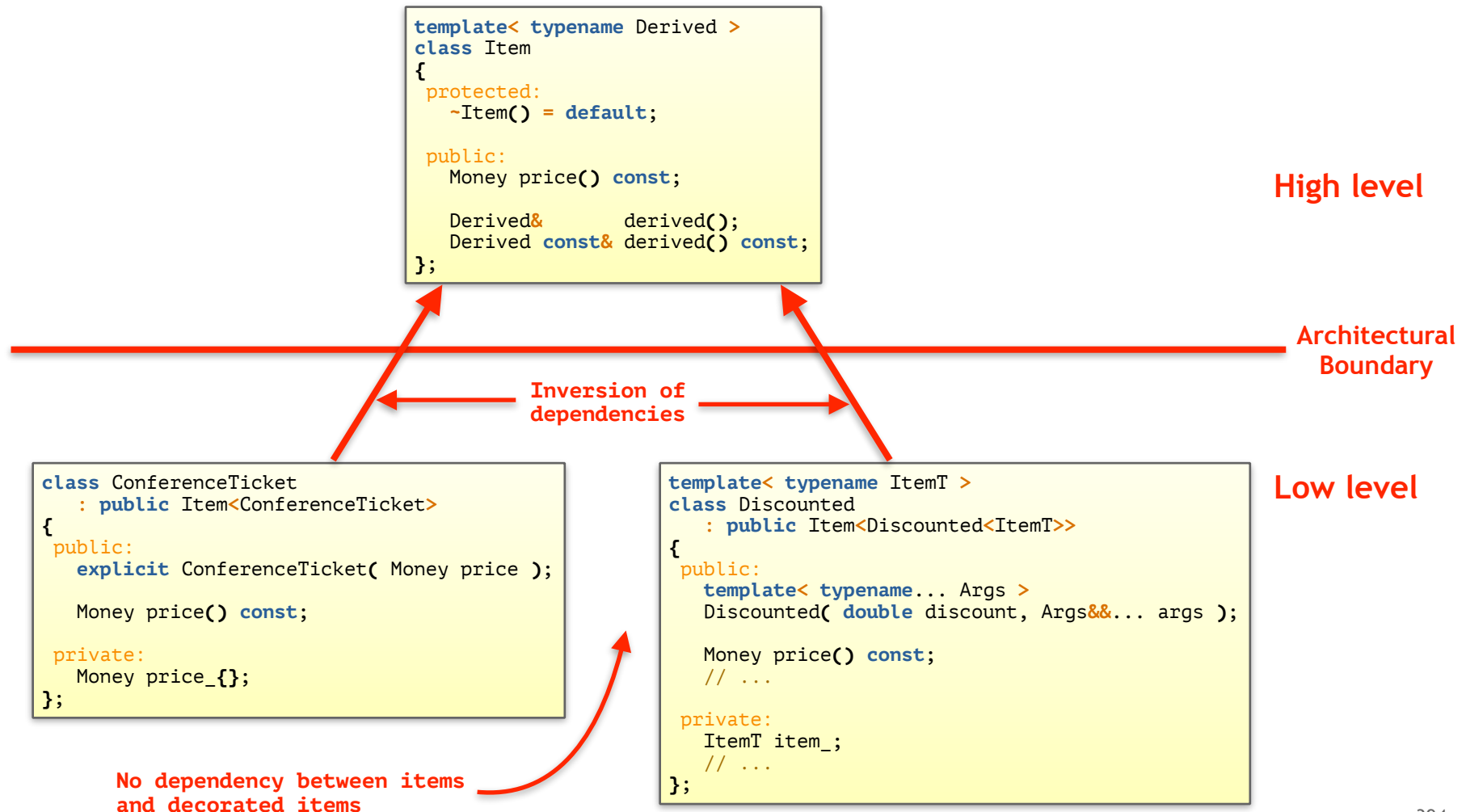
    Money price() const noexcept { return item_.price() * factor_; }

private:
    ItemT item_;
    double factor_{};
};
```

Alternative 3: A CRTP-Based Decorator

```
int main()
{
    Tax<Discount<ConferenceTicket>> item( 0.19, 0.2, "CppCon", Money{999.0F} );
    item.price();
    // ...
}
```

Alternative 3: A CRTP-Based Decorator



An Example from the Standard Library



std::vector

Defined in header `<vector>`

```
template<
    class T,
    class Allocator = std::allocator<T>
> class vector; (1)

namespace pmr {
    template <class T>
        using vector = std::vector<T, std::pmr::polymorphic_allocator<T>>; (2) (since C++17)
}
```

1) `std::vector` is a sequence container that encapsulates dynamic size arrays.

2) `std::pmr::vector` is an alias template that uses a [polymorphic allocator](#)

An Example from the Standard Library



std::pmr::memory_resource

Defined in header `<memory_resource>`

`class` `memory_resource`; (since C++17)

The class `std::pmr::memory_resource` is an abstract interface to an unbounded set of classes encapsulating memory resources.

Member functions

<code>(constructor)</code> (implicitly declared)	constructs a new <code>memory_resource</code> (public member function)
<code>(destructor)</code> [virtual]	destructs an <code>memory_resource</code> (virtual public member function)
<code>operator=</code> (implicitly declared)	Implicitly declared copy assignment operator (public member function)

Public member functions

<code>allocate</code>	allocates memory (public member function)
<code>deallocate</code>	deallocates memory (public member function)
<code>is_equal</code>	compare for equality with another <code>memory_resource</code> (public member function)

Private member functions

<code>do_allocate</code> [virtual]	allocates memory (virtual private member function)
<code>do_deallocate</code> [virtual]	deallocates memory (virtual private member function)
	compare for equality with another <code>memory_resource</code>

An Example from the Standard Library



The class `std::pmr::memory_resource` is an abstract interface to an unbounded set of classes encapsulating memory resources.

Member functions

(constructor) (implicitly declared)	constructs a new <code>memory_resource</code> (public member function)
(destructor) [virtual]	destructs an <code>memory_resource</code> (virtual public member function)
operator= (implicitly declared)	Implicitly declared copy assignment operator (public member function)

Public member functions

allocate	allocates memory (public member function)
deallocate	deallocates memory (public member function)
is_equal	compare for equality with another <code>memory_resource</code> (public member function)

Private member functions

do_allocate [virtual]	allocates memory (virtual private member function)
do_deallocate [virtual]	deallocates memory (virtual private member function)
do_is_equal [virtual]	compare for equality with another <code>memory_resource</code> (virtual private member function)

Non-member-functions

operator==	compare two <code>memory_resource</code> s
operator!= (removed in C++20)	(function)

An Example from the Standard Library



```
// ...
#include <memory_resource>

int main()
{
    std::array<std::byte,1000> raw; // Note: not initialized!

    return EXIT_SUCCESS;
}
```

An Example from the Standard Library



```
// ...
#include <memory_resource>

int main()
{
    std::array<std::byte, 1000> raw; // Note: not initialized!

    std::pmr::monotonic_buffer_resource buffer{ raw.data(), raw.size()
                                                , std::pmr::null_memory_resource() };

    return EXIT_SUCCESS;
}
```

An Example from the Standard Library



```
// ...
#include <memory_resource>

int main()
{
    std::array<std::byte, 1000> raw; // Note: not initialized!

    std::pmr::monotonic_buffer_resource buffer{ raw.data(), raw.size()
                                                , std::pmr::null_memory_resource() };

    std::pmr::vector<std::pmr::string> strings{ &buffer };

    return EXIT_SUCCESS;
}
```

An Example from the Standard Library



```
// ...
#include <memory_resource>

int main()
{
    std::array<std::byte, 1000> raw; // Note: not initialized!

    std::pmr::monotonic_buffer_resource buffer{ raw.data(), raw.size()
                                                , std::pmr::null_memory_resource() };

    std::pmr::vector<std::pmr::string> strings{ &buffer };

    strings.emplace_back( "String longer than what SSO can handle" );
    strings.emplace_back( "Another long string that goes beyond SSO" );
    strings.emplace_back( "A third long string that cannot be handled by SSO" );

    for( const auto& s : strings ) {
        std::cout << std::quoted(s) << '\n';
    }

    return EXIT_SUCCESS;
}
```

An Example from the Standard Library



```
// ...
#include <memory_resource>

int main()
{
    std::array<std::byte, 1000> raw; // Note: not initialized!

    std::pmr::monotonic_buffer_resource buffer{ raw.data(), raw.size()
                                                , std::pmr::null_memory_resource() };

    std::pmr::vector<std::pmr::string> strings{ &buffer };

    strings.emplace_back( "String longer than what SSO can handle" );
    strings.emplace_back( "Another long string that goes beyond SSO" );
    strings.emplace_back( "A third long string that cannot be handled by SSO" );

    for( const auto& s : strings ) {
        std::cout << std::quoted(s) << '\n';
    }

    return EXIT_SUCCESS;
}
```



Programming Exercise

Task (C_Cpp_Software_Design/Decorator/Decorator_Allocator):

Extend the following example for C++17 polymorphic memory resources:

Step 1: Instead of going out of memory via `std::pmr::null_memory_resource`, provide a `std::pmr::new_delete_resource` as backend allocator.

Step 2: Track the dynamic memory allocations via a custom memory resource called `MemoryTracker`.

Step 3: Introduce an allocator called `Mallocator`, which builds on `std::malloc` and `std::free`.

Step 4: Introduce an allocator called `AlignedAllocator`, which builds on `std::aligned_alloc` and `std::free`.

Step 5: Introduce a memory pool, which uses stack memory for allocation sizes below a certain threshold and dynamic memory otherwise.



Programming Task

Task (C_Cpp_Software_Design/Decorator/SynchronizedPool):

Compare the allocation behavior of the following two options:

Option 1: Use a `std::map` equipped with a `std::allocator`.

Option 2: Use a `std::pmr::map` equipped with a `std::pmr::unsynchronized_pool` allocator.

Programming Exercise: White Knight vs. Fire Dragon



Programming Exercise: White Knight vs. Fire Dragon

Task (C_Cpp_Software_Design/Decorator/Decorator_Fantasy):

Step 1: Help the knight “Roland” to win against the dragon “Darksmoke” by conjuring helpful magical equipment (e.g. IceEnchantment, StrengthEnchantment, MagicShield, ShoesOfAgility, or MagicShackles). Note that you are only allowed to add new code, but not to modify existing code.

Step 2: Refactor the classic Decorator solution by a value semantics based solution. Note that the general behavior should remain unchanged.

Step 3: Discuss the interface segregation of the Weapon, Armor, and Gear classes: what are the benefits and drawbacks of separating the `attack()`, `defense()`, and `speed()` functions?

Guidelines

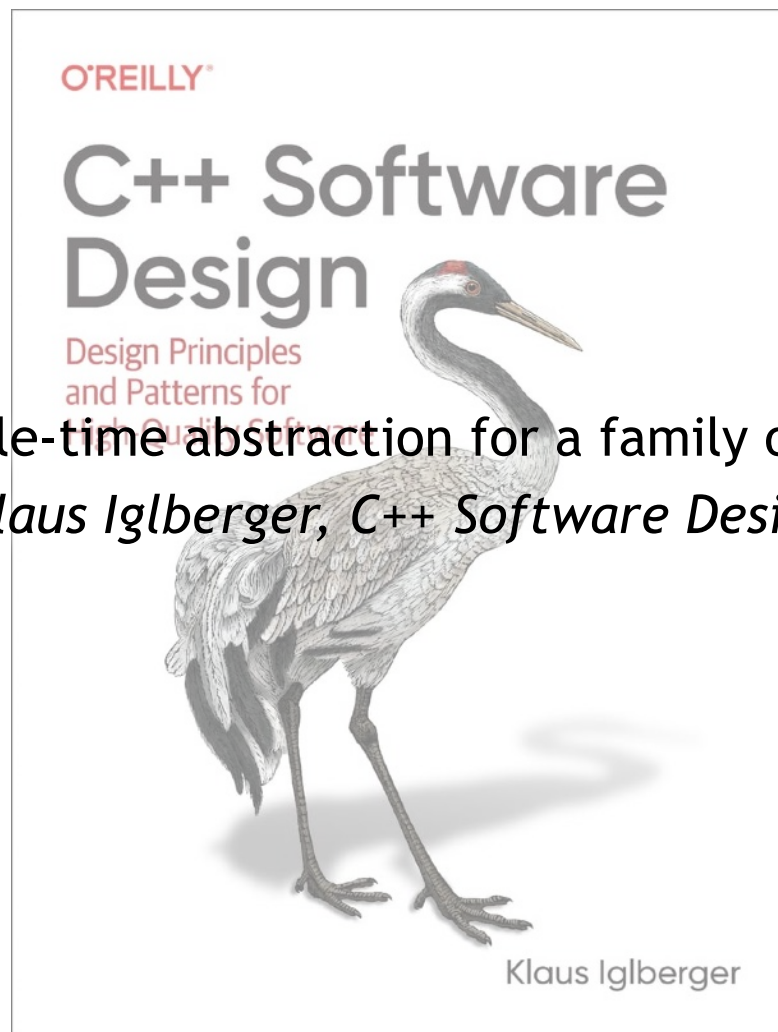
Guideline: Use the Decorator design pattern to attach new responsibilities to an object.

Guideline: Remember the Decorator design pattern as a non-intrusive alternative to subclassing.

Guideline: Prefer containment (composition/aggregation) to inheritance.

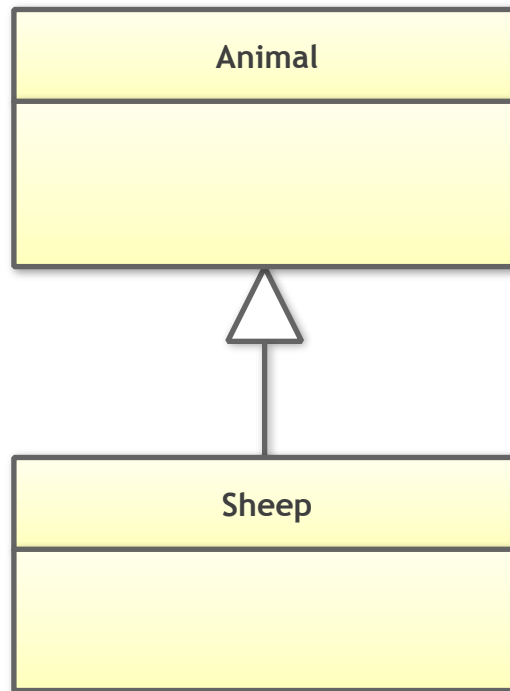
2.11. CRTP

The Intent



”Define a compile-time abstraction for a family of related types.”
(Klaus Iglberger, *C++ Software Design*)

The Overhead of Inheritance Hierarchies

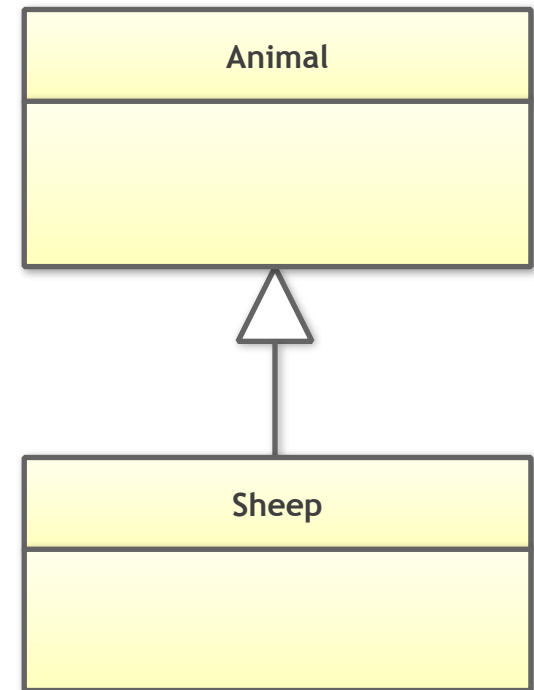


The Overhead of Inheritance Hierarchies

```
class Animal
{
public:
    virtual ~Animal() = default;

    virtual void make_sound() const = 0;
};

class Sheep : public Animal
{
public:
    ~Sheep();
    // ...
    void make_sound() const override { /*...*/ }
};
```



Virtual functions help to avoid downcasts, but are not cheap either.

The Overhead of Inheritance Hierarchies



The image shows a video player interface for a presentation at Cppcon 2022. The main title of the video is "The Hidden Performance Price of Virtual Functions" by IVICA BOGOSAVLJEVIC. The video player includes a progress bar at the bottom showing 0:00 / 49:28, and various control icons like play, pause, volume, and full screen. The Cppcon logo and the year 2022 are also visible.

The Hidden Performance Price of Virtual Functions

IVICA BOGOSAVLJEVIC

Cppcon
The C++ Conference

2022 | September 12th-16th

Full screen (f)

0:00 / 49:28 • Introduction >

The Overhead of Inheritance Hierarchies

Includes paid promotion >

Using Modern C++ to Eliminate Virtual Functions

JONATHAN GOPEL



Cppcon
The C++ Conference

20
22



September 12th-16th

▶ ▶| 🔊 0:01 / 1:01:28 • Introduction >

⏸ ⏮ ⏭ ⚙ ⏴ ⏵ ⏶ ⏷

The Overhead of Inheritance Hierarchies

Optimizing Away Virtual Functions May Be Pointless

SHACHAR SHEMESH



0:07 / 31:31

20
23



October 01 - 06

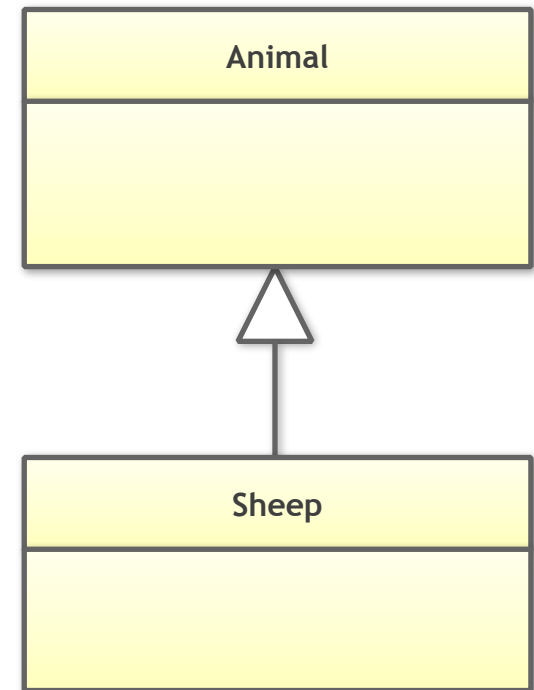


The Overhead of Inheritance Hierarchies

```
class Animal
{
public:
    virtual ~Animal() = default;

    virtual void make_sound() const = 0;
};

class Sheep : public Animal
{
public:
    ~Sheep();
    // ...
    void make_sound() const override { /*...*/ }
};
```



Virtual functions help to avoid downcasts, but are not cheap either.

CRTP - Curiously Recurring Template Pattern

Curiously Recurring Template Patterns

James O. Coplien

Software Production Research Dept., AT&T Bell Laboratories

1000 East Warrenton Rd., Naperville, IL 60566 USA

(708) 713-5384

cope@research.att.com

Abstract

Software design patterns capture recurring good practice in a domain. Many good patterns enjoy independent discovery by different people at different times. I have now seen three distinct uses of what I thought was an obscure template pattern. Each use arose in a unique domain; one instance was outside the C++ community. We can capture the technique in a pattern that solves a problem of factoring circular dependencies in code structure and behavior. The pattern form makes an otherwise opaque framework more accessible.

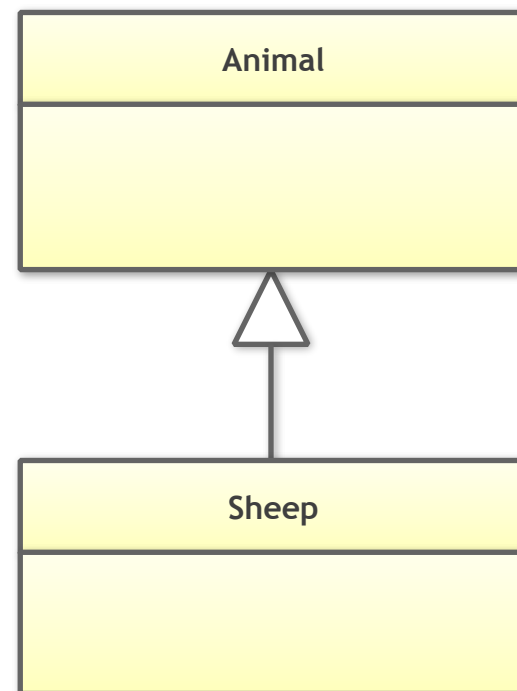
CRTP - Curiously Recurring Template Pattern

```
class Animal  
{
```

```
};
```

```
class Sheep  
{
```

```
};
```



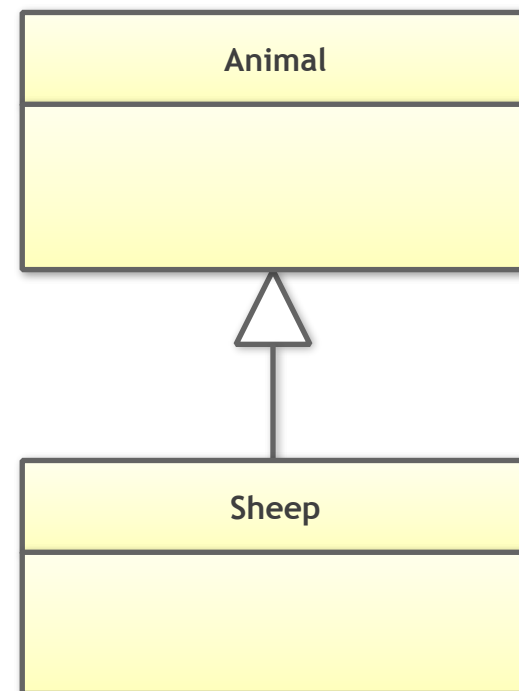
CRTP - Curiously Recurring Template Pattern

```
template< typename T >  
class Animal  
{
```

```
};
```

```
class Sheep  
{
```

```
};
```



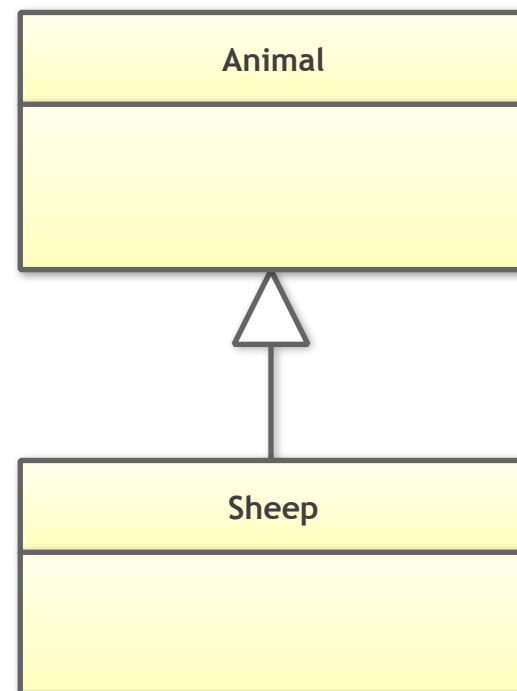
CRTP - Curiously Recurring Template Pattern

```
template< typename T >
class Animal
{

};

class Sheep : public Animal<Sheep>
{

};
```



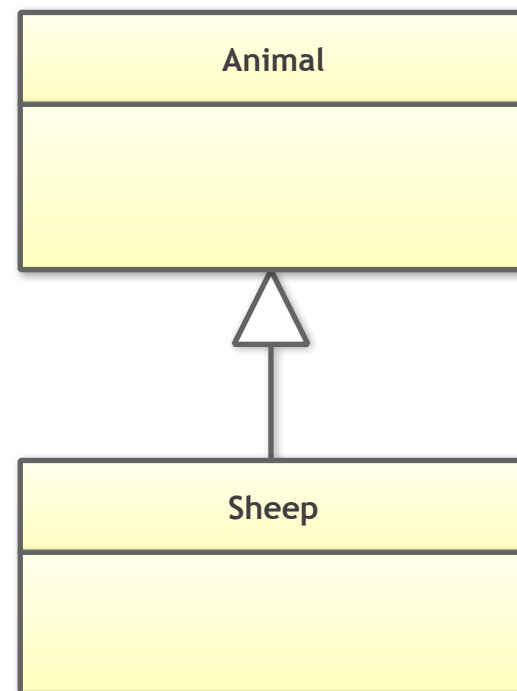
CRTP - Curiously Recurring Template Pattern

```
template< typename Derived >
class Animal
{

};

class Sheep : public Animal<Sheep>
{

};
```



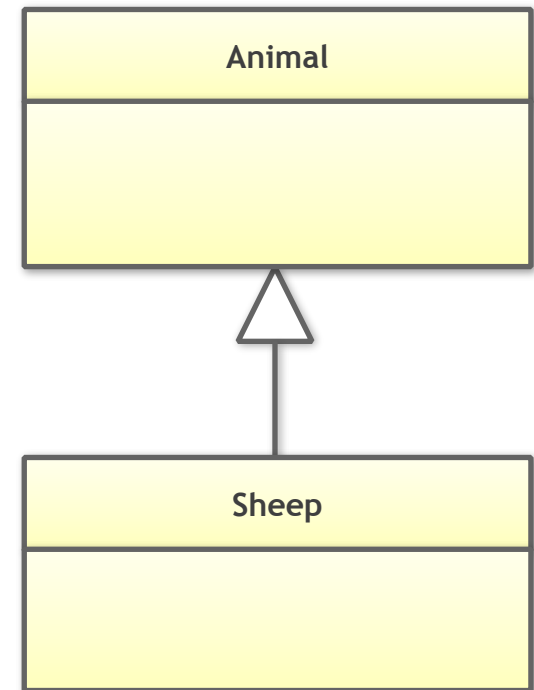
CRTP - Curiously Recurring Template Pattern

```
template< typename Derived >
class Animal
{
private:
    Animal() = default;
    ~Animal() = default;

};

class Sheep : public Animal<Sheep>
{

};
```



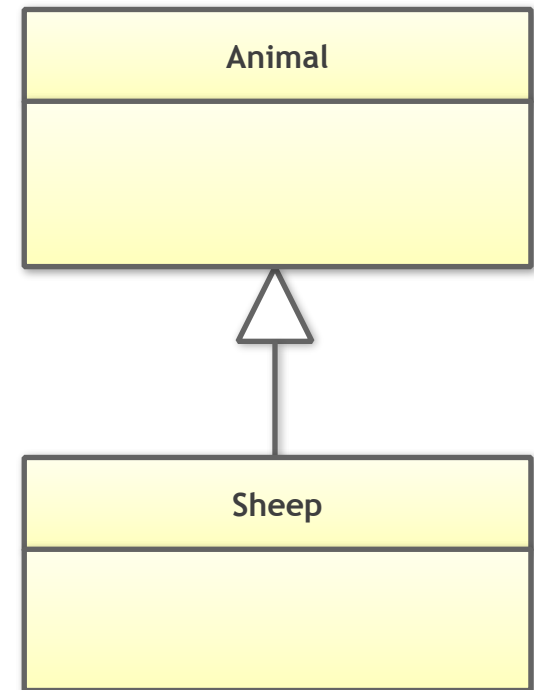
CRTP - Curiously Recurring Template Pattern

```
template< typename Derived >
class Animal
{
private:
    Animal() = default; // Protects against
    ~Animal() = default; // "wrong" Derived class
    friend Derived;

};

class Sheep : public Animal<Sheep>
{

};
```



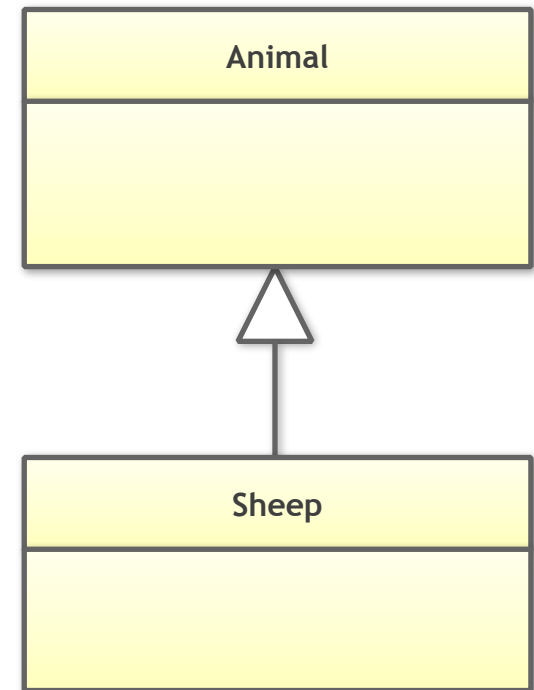
CRTP - Curiously Recurring Template Pattern

```
template< typename Derived >
class Animal
{
private:
    Animal() = default; // Protects against
    ~Animal() = default; // "wrong" Derived class
    friend Derived;

};

class Sheep : public Animal<Sheep>
{
public:
    //~Sheep(); Remember the Rule-of-5

};
```



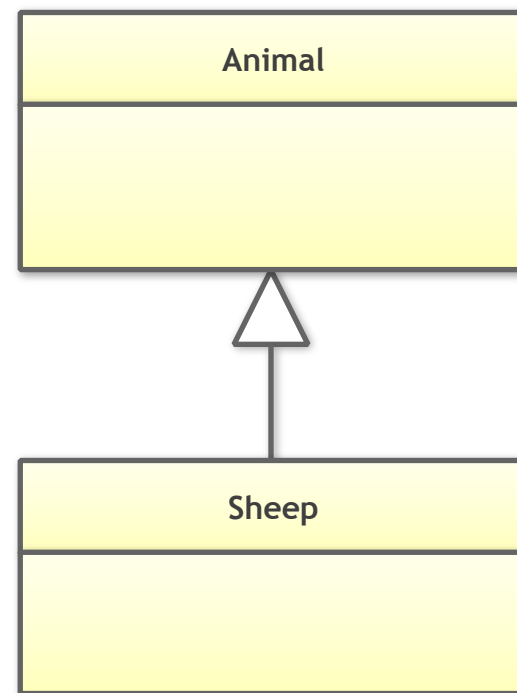
CRTP - Curiously Recurring Template Pattern

```
template< typename Derived >
class Animal
{
private:
    Animal() = default; // Protects against
    ~Animal() = default; // "wrong" Derived class
    friend Derived;

public:
    void make_sound() const {
        // ...
    }
};

class Sheep : public Animal<Sheep>
{
public:
    //~Sheep(); Remember the Rule-of-5

};
```

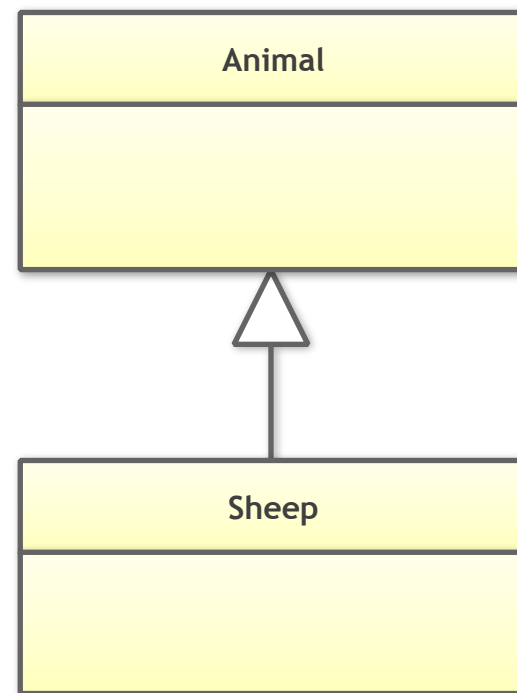


CRTP - Curiously Recurring Template Pattern

```
template< typename Derived >
class Animal
{
private:
    Animal() = default; // Protects against
    ~Animal() = default; // "wrong" Derived class
    friend Derived;

public:
    void make_sound() const {
        // ...
    }
};

class Sheep : public Animal<Sheep>
{
public:
    //~Sheep(); Remember the Rule-of-5
    // ...
    void make_sound() const { std::cout << "baa"; }
};
```



CRTP - Curiously Recurring Template Pattern

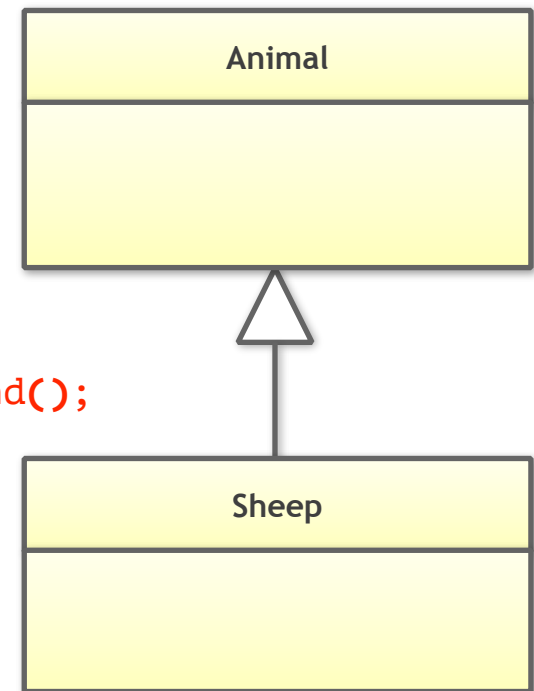
```

template< typename Derived >
class Animal
{
private:
    Animal() = default; // Protects against
    ~Animal() = default; // "wrong" Derived class
    friend Derived;

public:
    void make_sound() const {
        static_cast<Derived const*>(*this).make_sound();
    }
};

class Sheep : public Animal<Sheep>
{
public:
    //~Sheep(); Remember the Rule-of-5
    // ...
    void make_sound() const { std::cout << "baa"; }
};

```

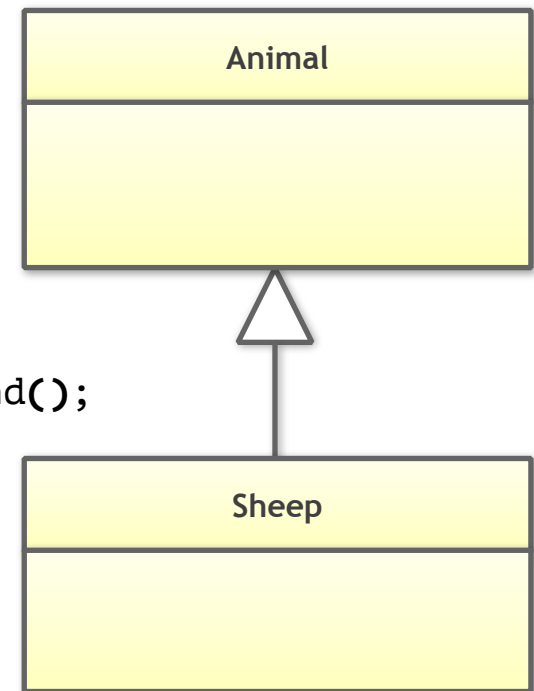


CRTP - Curiously Recurring Template Pattern

```
template< typename Derived >
class Animal
{
private:
    Animal() = default; // Protects against
    ~Animal() = default; // "wrong" Derived class
    friend Derived;

public:
    void make_sound() const {
        static_cast<Derived const*>(*this).make_sound();
    }
};

class Sheep : public Animal<Sheep>
{
public:
    //~Sheep(); Remember the Rule-of-5
    // ...
    void make_sound() const { std::cout << "baa"; }
};
```

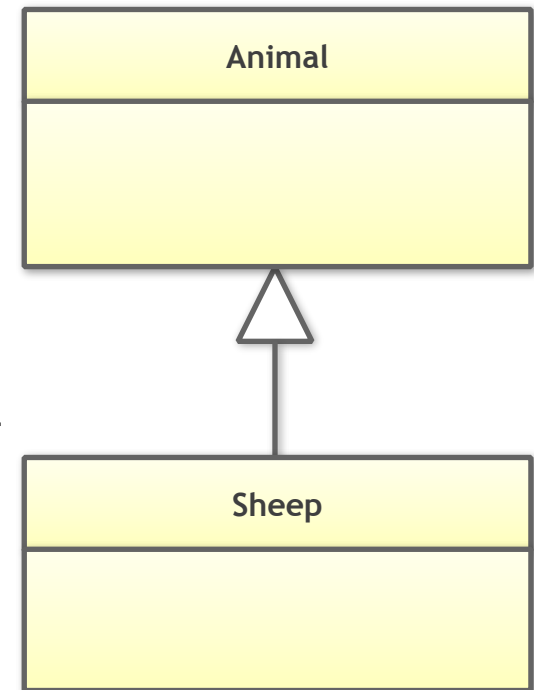


CRTP - Curiously Recurring Template Pattern

```
template< typename Derived >
class Animal
{
    // ...
    void make_sound() const;
};

class Sheep : public Animal<Sheep>
{
    // ...
    void make_sound() const { std::cout << "baa"; }
};

template< typename Derived >
void pet_animal( Animal<Derived> const& animal )
{
    // ...
    animal.make_sound();
    // ...
}
```



Works for all animals and does not incur any runtime costs (the function might even be inlined)



The Limitations of CRTP

1. Limitation: There Is No Common Base Class

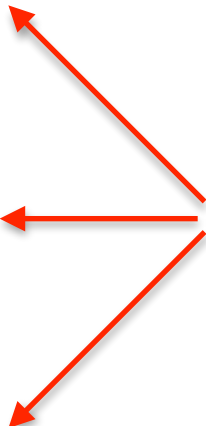
```
template< typename T >
class Animal
{
    // ...
};

class Sheep : public Animal<Sheep>
{
    // ...
};

class Dog : public Animal<Dog>
{
    // ...
};

class Cat : public Animal<Cat>
{
    // ...
};
```

// Different base classes,
// i.e. no common base class

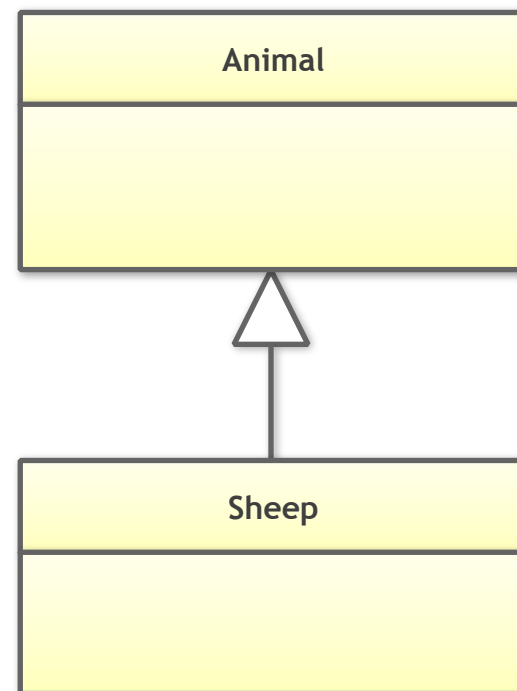


2. Limitation: Everything Is A Template

```
template< typename Derived >
class Animal
{
    // ...
    void make_sound() const;
};

class Sheep : public Animal<Sheep>
{
    // ...
    void make_sound() const { std::cout << "baa"; }
};

template< typename Derived >
void pet_animal( Animal<Derived> const& animal )
{
    // ...
    animal.make_sound();
    // ...
}
```



This function must be a function template to take any kind of animal. Thus CRTP can act like a virus: Everything touching CRTP is or becomes a template (including higher compile times).

Programming Task

Task (C_Cpp_Software_Design/CRTP/CRTP):

Step 1: Implement a common output operator and L2-norm for different kinds of `DenseVector` by means of the CRTP design pattern.

Step 2: Implement an addition operation between different kinds of `DenseVector`.

CRTP - Curiously Recurring Template Pattern

//--DynamicVector.h-----

```
template< typename T >
class DynamicVector
{
public:
    using value_type      = T;
    using iterator        = typename std::vector<T>::iterator;
    using const_iterator  = typename std::vector<T>::const_iterator;

    DynamicVector() = default;
    DynamicVector( std::initializer_list<T> init )
        : values_( std::begin(init), std::end(init) )
    {}

    size_t size() const { return values_.size(); }

    T&      operator[]( size_t index )      { return values_[index]; }
    T const& operator[]( size_t index ) const { return values_[index]; }

    const_iterator begin() const { return values_.begin(); }
    const_iterator end()   const { return values_.end(); }

private:
    std::vector<T> values_;
};
```

```
template< typename T >
```

CRTP - Curiously Recurring Template Pattern

```
//--DynamicVector.h-----  
  
template< typename T >  
class DynamicVector  
{  
public:  
    using value_type      = T;  
    using iterator        = typename std::vector<T>::iterator;  
    using const_iterator  = typename std::vector<T>::const_iterator;  
  
    DynamicVector() = default;  
    DynamicVector( std::initializer_list<T> init )  
        : values_( std::begin(init), std::end(init) )  
    {}  
  
    size_t size() const { return values_.size(); }  
  
    T&      operator[]( size_t index )      { return values_[index]; }  
    T const& operator[]( size_t index ) const { return values_[index]; }  
  
    const_iterator begin() const { return values_.begin(); }  
    const_iterator end()   const { return values_.end(); }  
  
private:  
    std::vector<T> values_;  
};
```

```
template< typename T >
```

CRTP - Curiously Recurring Template Pattern

```
const_iterator begin() const { return values_.begin(); }
const_iterator end()   const { return values_.end(); }

private:
    std::vector<T> values_;
};

template< typename T >
std::ostream& operator<< ( std::ostream& os, DynamicVector<T> const& vector )
{
    os << "(";
    for( auto const& item : vector ) {
        std::cout << " " << item;
    }
    os << " )";

    return os;
}

template< typename T >
constexpr decltype(auto) l2norm( DynamicVector<T> const& vector )
{
    using R = decltype( std::sqrt( std::declval<T>() ) );
    return std::sqrt( std::accumulate( std::begin(vector), std::end(vector), R{} ) );
}
```


CRTP - Curiously Recurring Template Pattern

```
template< typename T >
std::ostream& operator<<( std::ostream& os, DynamicVector<T> const& vector )
{
    os << "(";
    for( auto const& item : vector ) {
        std::cout << " " << item;
    }
    os << " )";

    return os;
}
```

```
template< typename T >
constexpr decltype(auto) l2norm( DynamicVector<T> const& vector )
{
    using R = decltype( std::sqrt( std::declval<T>() ) );
    return std::sqrt( std::accumulate( std::begin(vector), std::end(vector), R{} ) );
}
```

//--StaticVector.h-----

```
template< typename T, size_t Size >
class StaticVector
{
public:
    using value_type      = T;
    using iterator        = typename std::array<T,Size>::iterator;
    using const_iterator  = typename std::array<T,Size>::const_iterator;
```

CRTP - Curiously Recurring Template Pattern

```
//--StaticVector.h-----  
  
template< typename T, size_t Size >  
class StaticVector  
{  
public:  
    using value_type      = T;  
    using iterator        = typename std::array<T,Size>::iterator;  
    using const_iterator  = typename std::array<T,Size>::const_iterator;  
  
    constexpr StaticVector() = default;  
    constexpr StaticVector( std::initializer_list<T> init )  
    {  
        std::copy( std::begin(init), std::end(init), std::begin(values_) );  
    }  
  
    constexpr size_t size() const { return values_.size(); }  
  
    constexpr T&      operator[]( size_t index )      { return values_[index]; }  
    constexpr T const& operator[]( size_t index ) const { return values_[index]; }  
  
    constexpr const_iterator begin() const { return values_.begin(); }  
    constexpr const_iterator end()   const { return values_.end(); }  
  
private:  
    std::array<T,Size> values_;  
};
```

CRTP - Curiously Recurring Template Pattern

```
constexpr const_iterator begin() const { return values_.begin(); }
constexpr const_iterator end()   const { return values_.end(); }

private:
    std::array<T,Size> values_;
};

template< typename T, size_t Size >
std::ostream& operator<<( std::ostream& os, StaticVector<T,Size> const& vector )
{
    os << "(";
    for( auto const& item : vector ) {
        std::cout << " " << item;
    }
    os << " )";

    return os;
}

template< typename T, size_t Size >
constexpr decltype(auto) l2norm( StaticVector<T,Size> const& vector )
{
    using R = decltype( std::sqrt( std::declval<T>() ) );
    return std::sqrt( std::accumulate( std::begin(vector), std::end(vector), R{} ) );
}
```

CRTP - Curiously Recurring Template Pattern

```
template< typename T, size_t Size >
std::ostream& operator<<( std::ostream& os, StaticVector<T,Size> const& vector )
{
    os << "(";
    for( auto const& item : vector ) {
        std::cout << " " << item;
    }
    os << " )";

    return os;
}
```

```
template< typename T, size_t Size >
constexpr decltype(auto) l2norm( StaticVector<T,Size> const& vector )
{
    using R = decltype( std::sqrt( std::declval<T>() ) );
    return std::sqrt( std::accumulate( std::begin(vector), std::end(vector), R{} ) );
}
```

//--Main.cpp-----

```
int main()
{
    DynamicVector<int>    const a{ 1, 2, 3 };
    StaticVector<int,4UL> const b{ 4, 5, 6, 7 };

    std::cout << "\n"
               << " a = " << a << " l2-norm = " << l2norm(a) << "\n"
```

CRTP - Curiously Recurring Template Pattern

```
template< typename T, size_t Size >
constexpr decltype(auto) l2norm( StaticVector<T,Size> const& vector )
{
    using R = decltype( std::sqrt( std::declval<T>() ) );
    return std::sqrt( std::accumulate( std::begin(vector), std::end(vector), R{} ) );
}
```

```
//--Main.cpp-----
```

```
int main()
{
    DynamicVector<int>    const a{ 1, 2, 3 };
    StaticVector<int,4UL> const b{ 4, 5, 6, 7 };

    std::cout << "\n"
               << " a = " << a << ", L2-norm = " << l2norm(a) << "\n"
               << " b = " << b << ", L2-norm = " << l2norm(b) << "\n"
               << "\n";

    return EXIT_SUCCESS;
}
```

CRTP - Curiously Recurring Template Pattern

```
constexpr const_iterator begin() const { return values_.begin(); }
constexpr const_iterator end()   const { return values_.end(); }

private:
    std::array<T,Size> values_;
};

template< typename T, size_t Size >
std::ostream& operator<<( std::ostream& os, StaticVector<T,Size> const& vector )
{
    os << "(";
    for( auto const& item : vector ) {
        std::cout << " " << item;
    }
    os << " )";

    return os;
}

template< typename T, size_t Size >
constexpr decltype(auto) l2norm( StaticVector<T,Size> const& vector )
{
    using R = decltype( std::sqrt( std::declval<T>() ) );
    return std::sqrt( std::accumulate( std::begin(vector), std::end(vector), R{} ) );
}
```

CRTP - Curiously Recurring Template Pattern

```
const_iterator begin() const { return values_.begin(); }
const_iterator end()   const { return values_.end(); }

private:
    std::vector<T> values_;
};

template< typename T >
std::ostream& operator<< ( std::ostream& os, DynamicVector<T> const& vector )
{
    os << "(";
    for( auto const& item : vector ) {
        std::cout << " " << item;
    }
    os << " )";

    return os;
}

template< typename T >
constexpr decltype(auto) l2norm( DynamicVector<T> const& vector )
{
    using R = decltype( std::sqrt( std::declval<T>() ) );
    return std::sqrt( std::accumulate( std::begin(vector), std::end(vector), R{} ) );
}
```

CRTP - Curiously Recurring Template Pattern

//--DenseVector.h-----

```
template< typename VT >
std::ostream& operator<<( std::ostream& os, const VT& vector )
{
    const size_t size( vector.size() );

    os << "(";
    for( size_t i=0UL; i<size; ++i ) {
        std::cout << " " << vector[i];
    }
    os << " )";

    return os;
}
```

```
template< typename VT >
decltype(auto) l2norm( const VT& vector )
{
    using T = typename VT::value_type;
    using R = decltype( std::sqrt( std::declval<T>() ) );
    return std::sqrt( std::accumulate( std::begin(vector), std::end(vector), R{} ) );
}
```

//--DynamicVector.h-----

CRTP - Curiously Recurring Template Pattern

//--DenseVector.h-----

```
template< typename VT >
std::ostream& operator<<( std::ostream& os, const VT& vector )
{
    const size_t size( vector.size() );

    os << "(";
    for( size_t i=0UL; i<size; ++i ) {
        std::cout << " " << vector[i];
    }
    os << " )";

    return os;
}
```

```
template< typename VT >
decltype(auto) l2norm( const VT& vector )
{
    using T = typename VT::value_type;
    using R = decltype( std::sqrt( std::declval<T>() ) );
    return std::sqrt( std::accumulate( std::begin(vector), std::end(vector), R{} ) );
}
```

//--DynamicVector.h-----

CRTP - Curiously Recurring Template Pattern

```
//--DenseVector.h-----  
  
template< typename Derived >  
class DenseVector  
{  
private:  
    DenseVector() = default;  
    ~DenseVector() = default;  
    friend Derived;  
  
public:  
    constexpr Derived& derived() {  
        return static_cast<Derived&>( *this );  
    }  
    constexpr Derived const& derived() const {  
        return static_cast<Derived const&>( *this );  
    }  
  
    constexpr size_t size() const { return derived().size(); }  
  
    constexpr decltype(auto) operator[]( size_t index ) {  
        return derived()[index];  
    }  
    constexpr decltype(auto) operator[]( size_t index ) const {  
        return derived()[index];  
    }  
  
    decltype(auto) begin() const { return derived().begin(); }  
    decltype(auto) end() const { return derived().end(); }
```

CRTP - Curiously Recurring Template Pattern

```
//--DenseVector.h-----  
  
template< typename Derived >  
class DenseVector  
{  
private:  
    DenseVector() = default;  
    ~DenseVector() = default;  
    friend Derived;  
  
public:  
    constexpr Derived& derived() {  
        return static_cast<Derived&>( *this );  
    }  
    constexpr Derived const& derived() const {  
        return static_cast<Derived const&>( *this );  
    }  
  
    constexpr size_t size() const { return derived().size(); }  
  
    constexpr decltype(auto) operator[]( size_t index ) {  
        return derived()[index];  
    }  
    constexpr decltype(auto) operator[]( size_t index ) const {  
        return derived()[index];  
    }  
  
    decltype(auto) begin() const { return derived().begin(); }  
    decltype(auto) end() const { return derived().end(); }
```

CRTP - Curiously Recurring Template Pattern

```
//--DenseVector.h-----  
  
template< typename Derived >  
class DenseVector  
{  
private:  
    DenseVector() = default;  
    ~DenseVector() = default;  
    friend Derived;  
  
public:  
    constexpr Derived& derived() {  
        return static_cast<Derived&>( *this );  
    }  
    constexpr Derived const& derived() const {  
        return static_cast<Derived const&>( *this );  
    }  
  
    constexpr size_t size() const { return derived().size(); }  
  
    constexpr decltype(auto) operator[]( size_t index ) {  
        return derived()[index];  
    }  
    constexpr decltype(auto) operator[]( size_t index ) const {  
        return derived()[index];  
    }  
  
    decltype(auto) begin() const { return derived().begin(); }  
    decltype(auto) end() const { return derived().end(); }
```

CRTP - Curiously Recurring Template Pattern

```
template< typename Derived >
class DenseVector
{
private:
    DenseVector() = default;
    ~DenseVector() = default;
    friend Derived;

public:
    constexpr Derived& derived() {
        return static_cast<Derived&>( *this );
    }
    constexpr Derived const& derived() const {
        return static_cast<Derived const&>( *this );
    }

    constexpr size_t size() const { return derived().size(); }

    constexpr decltype(auto) operator[] ( size_t index ) {
        return derived()[index];
    }
    constexpr decltype(auto) operator[] ( size_t index ) const {
        return derived()[index];
    }

    decltype(auto) begin() const { return derived().begin(); }
    decltype(auto) end() const { return derived().end(); }
};
```

CRTP - Curiously Recurring Template Pattern

```
decltype(auto) end() const { return derived().end(); }
};
```

```
template< typename Derived >
std::ostream& operator<<( std::ostream& os, DenseVector<Derived> const& vector )
{
    size_t const size( vector.size() );

    os << "(";
    for( size_t i=0UL; i<size; ++i ) {
        std::cout << " " << vector[i];
    }
    os << " )";

    return os;
}
```

```
template< typename Derived >
decltype(auto) l2norm( DenseVector<Derived> const& vector )
{
    using T = typename Derived::value_type;
    using R = decltype( std::sqrt( std::declval<T>() ) );
    return std::sqrt( std::accumulate( std::begin(vector), std::end(vector), R{} ) );
}
```

CRTP - Curiously Recurring Template Pattern

```
//--DynamicVector.h-----  
  
template< typename T >  
class DynamicVector : public DenseVector< DynamicVector<T> >  
{  
public:  
    using value_type      = T;  
    using iterator        = typename std::vector<T>::iterator;  
    using const_iterator  = typename std::vector<T>::const_iterator;  
  
    DynamicVector() = default;  
    DynamicVector( std::initializer_list<T> init )  
        : values_( std::begin(init), std::end(init) )  
    {}  
  
    size_t size() const { return values_.size(); }  
  
    T& operator[]( size_t index ) { return values_[index]; }  
    T const& operator[]( size_t index ) const { return values_[index]; }  
  
    const_iterator begin() const { return values_.begin(); }  
    const_iterator end() const { return values_.end(); }  
  
private:  
    std::vector<T> values_;  
};  
  
//--StaticVector.h-----
```

CRTP - Curiously Recurring Template Pattern

```
//--StaticVector.h-----  
  
template< typename T, size_t Size >  
class StaticVector : public DenseVector< StaticVector<T,Size> >  
{  
public:  
    using value_type      = T;  
    using iterator        = typename std::array<T,Size>::iterator;  
    using const_iterator  = typename std::array<T,Size>::const_iterator;  
  
    constexpr StaticVector() = default;  
    constexpr StaticVector( std::initializer_list<T> init )  
    {  
        std::copy( std::begin(init), std::end(init), std::begin(values_) );  
    }  
  
    constexpr size_t size() const { return values_.size(); }  
  
    constexpr T& operator[]( size_t index ) { return values_[index]; }  
    constexpr T const& operator[]( size_t index ) const { return values_[index]; }  
  
    constexpr const_iterator begin() const { return values_.begin(); }  
    constexpr const_iterator end() const { return values_.end(); }  
  
private:  
    std::array<T,Size> values_;  
};
```


CRTP - Curiously Recurring Template Pattern

```
constexpr T& operator[]( size_t index ) { return values_[index]; }
constexpr T const& operator[]( size_t index ) const { return values_[index]; }

constexpr const_iterator begin() const { return values_.begin(); }
constexpr const_iterator end() const { return values_.end(); }

private:
    std::array<T,Size> values_;
};

//--Main.cpp-----

int main()
{
    DynamicVector<int> const a{ 1, 2, 3 };
    StaticVector<int,4UL> const b{ 4, 5, 6, 7 };

    std::cout << "\n"
                << " a = " << a << ", L2-norm = " << l2norm(a) << "\n"
                << " b = " << b << ", L2-norm = " << l2norm(b) << "\n"
                << "\n";

    return EXIT_SUCCESS;
}
```

The Future of CRTP?



Cppcon 2021
The C++ Conference



October 24-29



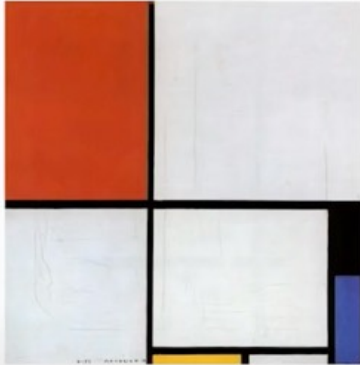




Ben Deane

Deducing this Patterns

DEDUCING **this** PATTERNS



BEN DEANE / **@ben_deane**

CPPCON 2021

1 / 64



0:44 / 1:02:41

The Future of CRTP?

 Cppcon 2021
The C++ Conference
October 24-29







Ben Deane

Deducing this Patterns

THE FEATURE

Here's what we came up with:

```
struct S {  
    template <typename Self>  
    auto func(this Self&& self);  
};
```

By the end of this talk, you will understand this slide
and (more importantly) its **implications**.

One of the satisfying things about this feature is that it
doesn't introduce new magic - as we shall see.

9 / 64

   5:39 / 1:02:41       

The Future of CRTP?

```
template< typename Derived >
class Animal
{
private:
    // ... Private default ctor and dtor

public:
    void make_sound() const {
        static_cast<Derived const*>(*this).make_sound_impl();
    }
};

class Sheep : public Animal<Sheep>
{
public:
    void make_sound_impl() const { std::cout << "baa"; }
};

int main()
{
    Sheep sheep;
    Animal<Sheep>& animal = sheep;

    sheep.make_sound();
    animal.make_sound();
}
```

The Future of CRTP?

```
class Animal
{
public:
    template< typename Self >
    void make_sound( this Self const& self )
    {
        self.make_sound_impl();
    }
};

class Sheep : public Animal
{
public:
    void make_sound_impl() const { std::cout << "baa"; }
};

int main()
{
    Sheep sheep;
    Animal& animal = sheep;

    sheep.make_sound();
    animal.make_sound();
}
```

No template parameter anymore 😊

The Future of CRTP?



Cppcon 2021
The C++ Conference



October 24-29







So CRTP really just ... I guess ... goes away.

```
Self& a(this Self&& self) { /* ... */; return self; }

template <typename Self>
Self& b(this Self&& self) { /* ... */; return self; }
};

struct Special : Builder {
    template <typename Self>
    Self& c(this Self&& self) { /* ... */; return self; }

    template <typename Self>
    Self& d(this Self&& self) { /* ... */; return self; }
};

struct Super : Special {
    template <typename Self>
    Self& e(this Self&& self) { /* ... */; return self; }
};
```

COMPLER

All the same flat pattern.

25 / 64

Ben Deane

Deducing this Patterns

19:49 / 1:02:41

II CC HD

The Future of CRTP?



Cppcon 2023
The C++ Conference



October 01 - 06



23



erg

Rudy Merriam

A Journey Into
Non-Virtual Polymorphism

C++23: Explicit Object Parameter AKA, Deducing This

```
struct Shape {  
    template<typename T>  
    void draw(this T&& self) { self.draw_impl();}  
};  
  
struct Rectangle : public Shape {  
    void draw_impl() const { std::cout << "Rectangle\n"; }  
};
```

Cppcon 2023 | Rud Merriam | A Journey into Non-Virtual Polymorphism

41

Video Sponsorship Provided By:



43:48 / 48:49

The Future of CRTP?

```
template <class Derived>  
struct Animal {  
    void speak(this auto&& self) { self.speak_impl(); }  
};  
  
struct Cat : public Animal<Cat> {  
    void speak_impl() { std::cout << "meow"; }  
};  
  
struct Dog : public Animal<Dog> {  
    void speak_impl() { std::cout << "woof"; }  
};  
  
int main() {  
    std::unique_ptr<Animal> myAnimal(new Cat);  
    myAnimal->speak(); // prints "meow"  
}
```

Copyright (c) Timur Doumler | [@timur_audio](https://twitter.com/timur_audio) | <https://timur.audio>

56



~~NDC { TechTown }~~

▶ 19:37 / 59:24



NDC
Conferences
Subscribe

The Future of CRTP?

```
class Animal
{
public:
    template< typename Self >
    void make_sound( this Self const& self )
    {
        self.make_sound_impl();
    }
};

class Sheep : public Animal
{
public:
    void make_sound_impl() const { std::cout << "baa"; }
};

int main()
{
    Sheep sheep;
    Animal& animal = sheep;

    sheep.make_sound();
    animal.make_sound();    // Compilation error!
}
```

The Future of CRTP?



Ben Deane

Deducing this Patterns

OK, THAT WAS A SIMPLE WIN

There's more. Template deduction is unchanged.

```
struct Base {  
    auto func1(this const Base& self);  
  
    template <typename Self>  
    auto func2(this Self&& self);  
};  
  
struct Derived : Base {};  
  
void example() {  
    Base b();  
    b.func1(); // self has type const Base&  
    b.func2(); // self has type Base&  
  
    Derived d();  
    d.func1(); // self has type const Base&  
    d.func2(); // self has type Derived&  
}
```

When called via the base class, the explicit object parameter is always deduced to be the type of the base class (the static type).

19 / 64

15:12 / 1:02:41



The Future of CRTP?



Ben Deane

Deducing this Patterns

CRTP

Pre P0847, CRTP is a case of following the pattern.

```
template <typename T>
struct NumericalFunctions {
    void scale(double multiplier) {
        T& underlying = static_cast<T&>(*this);
        underlying.setValue(underlying.getValue() * multiplier);
    }
};

struct Sensitivity : NumericalFunctions<Sensitivity> {
    double getValue() const;
    void setValue(double value);
};
```

(From <https://www.fluentcpp.com/2017/05/12/curiously-recurring-template-pattern/>)

20 / 64

The Two Forms of CRTP

Static Interfaces

- ... provide a base class for a related set/family of types;
- ... define a common interface;
- ... are used via the base class interface;
- ... introduce an abstraction and is a design pattern.

Mixins

- ... provide implementation details for the derived class;
- ... do **not** define a common interface;
- ... are not used via the base class interface;
- ... do not introduce an abstraction, hence is no design pattern.

Mixins

```
template< typename Derived >
struct NumericalFunctions
{
    void scale( double multiplicator )
    {
        Derived& underlying = static_cast<Derived&>(*this);
        underlying.setValue( underlying.getValue() * multiplicator );
    }
};

struct Sensitivity : public NumericalFunctions<Sensitivity>
{
    double getValue() const { return value; }
    void setValue( double v ) { value = v; }
    double value;
};

int main()
{
    Sensitivity s{ 1.2 };
    s.scale( 2.0 );

    std::println( std::cout, "s.getValue() = {}", s.getValue() );
}
```



Mixins

```
struct NumericalFunctions
{
    void scale( this auto&& self, double multiplicator )
    {
        self.setValue( self.getValue() * multiplicator );
    }
};

struct Sensitivity : public NumericalFunctions
{
    double getValue() const { return value; }
    void setValue( double v ) { value = v; }
    double value;
};

int main()
{
    Sensitivity s{ 1.2 };
    s.scale( 2.0 );

    std::println( std::cout, "s.getValue() = {}", s.getValue() );
}
```



Programming Task

Task (C_Cpp_Software_Design/CRTP/Mixins):

Understand the classic form of Mixin (Implementation) CRTP, before switching to the C++23 form using explicit object parameters (aka "Deducing This").

Programming Task

Task (C_Cpp_Software_Design/CRTP/StrongType_CRTP): Extend the `StrongType` class template by a CRTP-based mechanism to enable the customization of a specific strong type with user-defined operations. For instance, the `Meter` type should support addition and subtraction, it should be printable and equality comparable.

Static Interfaces

```
template< typename Derived >
class Animal
{
private:
    // ... Private default ctor and dtor

public:
    void make_sound() const {
        static_cast<Derived const*>(*this).make_sound_impl();
    }
};

class Sheep : public Animal<Sheep>
{
public:
    void make_sound_impl() const { std::cout << "baa"; }
};

int main()
{
    Sheep sheep;
    Animal<Sheep>& animal = sheep;

    sheep.make_sound();
    animal.make_sound();
}
```

Static Interfaces

```
class Animal
{
public:
    template< typename Self >
    void make_sound( this Self const& self )
    {
        self.make_sound_impl();
    }
};
```

```
class Sheep : public Animal
{
public:
    void make_sound_impl() const { std::cout << "baa"; }
};
```

```
int main()
{
    Sheep sheep;
    Animal& animal = sheep;

    sheep.make_sound();
    animal.make_sound();
}
```

Cannot compile since the 'Self' type
cannot be deduced to be the dynamic
type





Programming Task

Task (C_Cpp_Software_Design/CRTP/StaticInterfaces):

Understand the classic form of Interface CRTP, before switching to the C++23 form using explicit object parameters (aka "Deducing This").



Programming Task

Task (C_Cpp_Software_Design/CRTP/CRTP_Concept):

Step 1: Implement a common output operator and L2-norm for different kinds of `DenseVector` by means of C++20 concepts.

Step 2: Implement an addition operation between different kinds of `DenseVector`.

CRTP - Curiously Recurring Template Pattern

```
//--DenseVector.h-----  
  
class DenseVectorTag  
{  
};  
  
template< typename T >  
concept DenseVector = std::derived_from<T,DenseVectorTag>;  
  
template< DenseVector VectorType >  
std::ostream& operator<<( std::ostream& os, VectorType const& vector )  
{  
    size_t const size( vector.size() );  
  
    os << "(";  
    for( size_t i=0UL; i<size; ++i ) {  
        os << " " << vector[i];  
    }  
    os << " )";  
  
    return os;  
}  
  
template< DenseVector VectorType >  
constexpr decltype(auto) l2norm( VectorType const& vector )  
{  
    using T = typename VectorType::value_type;  
    return std::sqrt( std::inner_product( std::begin(vector), std::end(vector)  
                                          , std::begin(vector), T{} ) );  
}
```

CRTP - Curiously Recurring Template Pattern

```
//--DenseVector.h-----  
  
class DenseVectorTag  
{  
};  
  
template< typename T >  
concept DenseVector = std::derived_from<T,DenseVectorTag>;  
  
template< DenseVector VectorType >  
std::ostream& operator<<( std::ostream& os, VectorType const& vector )  
{  
    size_t const size( vector.size() );  
  
    os << "(";  
    for( size_t i=0UL; i<size; ++i ) {  
        os << " " << vector[i];  
    }  
    os << " )";  
  
    return os;  
}  
  
template< DenseVector VectorType >  
constexpr decltype(auto) l2norm( VectorType const& vector )  
{  
    using T = typename VectorType::value_type;  
    return std::sqrt( std::inner_product( std::begin(vector), std::end(vector)  
                                          , std::begin(vector), T{} ) );  
}
```

CRTP - Curiously Recurring Template Pattern

```
//--DenseVector.h-----  
  
class DenseVectorTag  
{  
};  
  
template< typename T >  
concept DenseVector = std::derived_from<T,DenseVectorTag>;  
  
template< DenseVector VectorType >  
std::ostream& operator<<( std::ostream& os, VectorType const& vector )  
{  
    size_t const size( vector.size() );  
  
    os << "(";  
    for( size_t i=0UL; i<size; ++i ) {  
        os << " " << vector[i];  
    }  
    os << " )";  
  
    return os;  
}  
  
template< DenseVector VectorType >  
constexpr decltype(auto) l2norm( VectorType const& vector )  
{  
    using T = typename VectorType::value_type;  
    return std::sqrt( std::inner_product( std::begin(vector), std::end(vector)  
                                          , std::begin(vector), T{} ) );  
}
```

CRTP - Curiously Recurring Template Pattern

```
//--DenseVector.h-----  
  
class DenseVectorTag  
{  
};  
  
template< typename T >  
concept DenseVector = std::derived_from<T,DenseVectorTag>;  
  
template< DenseVector VectorType >  
std::ostream& operator<< ( std::ostream& os, VectorType const& vector )  
{  
    size_t const size( vector.size() );  
  
    os << "(";  
    for( size_t i=0UL; i<size; ++i ) {  
        os << " " << vector[i];  
    }  
    os << " )";  
  
    return os;  
}  
  
template< DenseVector VectorType >  
constexpr decltype(auto) l2norm( VectorType const& vector )  
{  
    using T = typename VectorType::value_type;  
    return std::sqrt( std::inner_product( std::begin(vector), std::end(vector)  
                                          , std::begin(vector), T{} ) );  
}
```


CRTP - Curiously Recurring Template Pattern

```
class DenseVectorTag
{};

template< typename T >
concept DenseVector = std::derived_from<T,DenseVectorTag>;

template< DenseVector VectorType >
std::ostream& operator<<( std::ostream& os, VectorType const& vector )
{
    size_t const size( vector.size() );

    os << "(";
    for( size_t i=0UL; i<size; ++i ) {
        os << " " << vector[i];
    }
    os << " )";

    return os;
}

template< DenseVector VectorType >
constexpr decltype(auto) l2norm( VectorType const& vector )
{
    using T = typename VectorType::value_type;
    return std::sqrt( std::inner_product( std::begin(vector), std::end(vector)
                                          , std::begin(vector), T{} ) );
}
```

CRTP - Curiously Recurring Template Pattern

```
//--DynamicVector.h-----  
  
template< typename T >  
class DynamicVector : public DenseVectorTag  
{  
public:  
    using value_type      = T;  
    using iterator        = typename std::vector<T>::iterator;  
    using const_iterator  = typename std::vector<T>::const_iterator;  
  
    DynamicVector() = default;  
    DynamicVector( std::initializer_list<T> init )  
        : values_( std::begin(init), std::end(init) )  
    {}  
  
    size_t size() const { return values_.size(); }  
  
    T& operator[]( size_t index ) { return values_[index]; }  
    T const& operator[]( size_t index ) const { return values_[index]; }  
  
    const_iterator begin() const { return values_.begin(); }  
    const_iterator end() const { return values_.end(); }  
  
private:  
    std::vector<T> values_;  
};
```

CRTP - Curiously Recurring Template Pattern

```
//--StaticVector.h-----  
  
template< typename T, size_t Size >  
class StaticVector : public DenseVectorTag  
{  
public:  
    using value_type      = T;  
    using iterator        = typename std::array<T,Size>::iterator;  
    using const_iterator  = typename std::array<T,Size>::const_iterator;  
  
    constexpr StaticVector() = default;  
    constexpr StaticVector( std::initializer_list<T> init )  
    {  
        std::copy( std::begin(init), std::end(init), std::begin(values_) );  
    }  
  
    constexpr size_t size() const { return values_.size(); }  
  
    constexpr T& operator[]( size_t index ) { return values_[index]; }  
    constexpr T const& operator[]( size_t index ) const { return values_[index]; }  
  
    constexpr const_iterator begin() const { return values_.begin(); }  
    constexpr const_iterator end() const { return values_.end(); }  
  
private:  
    std::array<T,Size> values_;  
};
```

CRTP - Curiously Recurring Template Pattern

```
constexpr T& operator[] ( size_t index ) { return values_[index]; }
constexpr T const& operator[] ( size_t index ) const { return values_[index]; }

constexpr const_iterator begin() const { return values_.begin(); }
constexpr const_iterator end() const { return values_.end(); }

private:
    std::array<T,Size> values_;
};

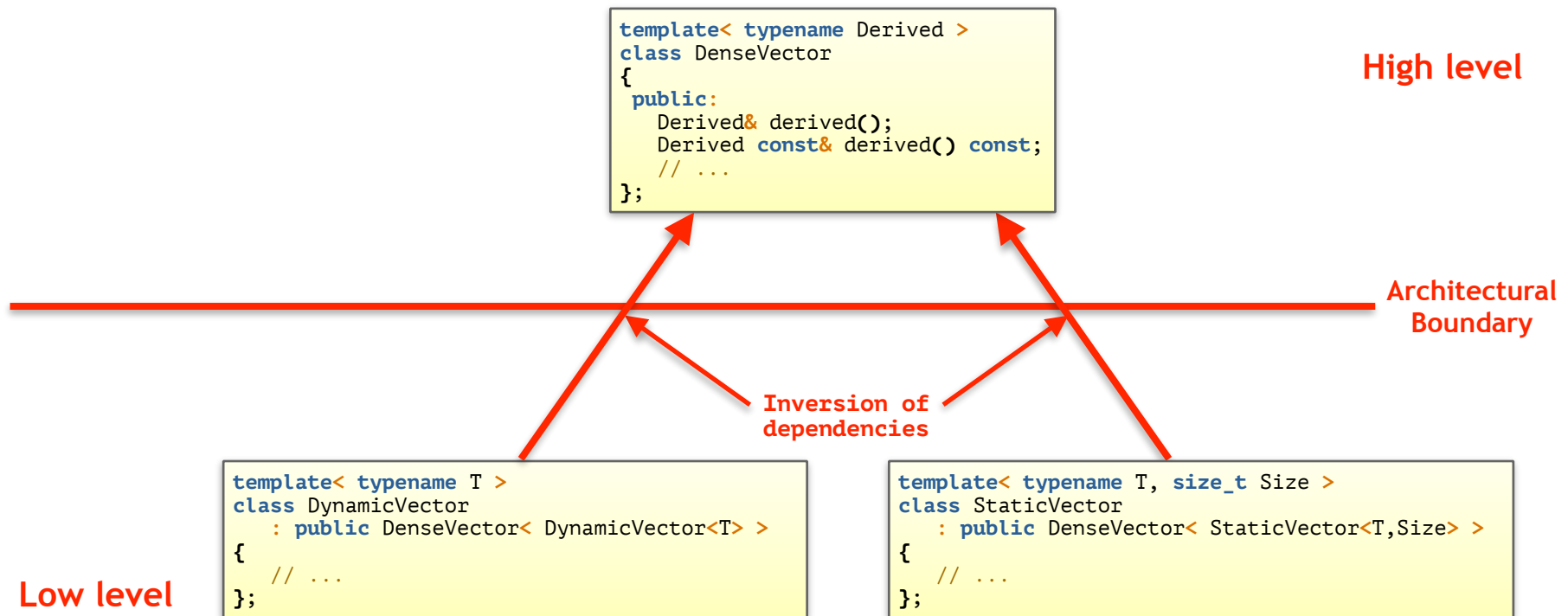
//--Main.cpp-----

int main()
{
    DynamicVector<int> const a{ 1, 2, 3 };
    StaticVector<int,4UL> const b{ 4, 5, 6, 7 };

    std::cout << "\n"
                << " a = " << a << ", L2-norm = " << l2norm(a) << "\n"
                << " b = " << b << ", L2-norm = " << l2norm(b) << "\n"
                << "\n";

    return EXIT_SUCCESS;
}
```

Dependency Structure



CRTP — Advantages/Disadvantages

Use CRTP if ...

- ... you don't need a **common base class**;
- ... you functions that use the base class may be **templates**;
- ... you don't need to **explicitly use** the base class;
- ... the **virtual call overhead** is not acceptable.

Don't use CRTP if ...

- ... you need a **common base class**;
- ... you need code to reside in **source files**;
- ... you need to use the base class across **architectural boundaries**;
- ... you could just as well use a **concept**.

Guidelines

Guideline: Use CRTP to introduce a compile-time abstraction for an extensible group of types.

2.13. Expression Templates

Expression Templates



This paper was presented at the 1994 C++ World Conference in Austin, Texas, and later appeared in the June 1995 issue of [C++ Report](#) Magazine. It has been reprinted in the book "C++ Gems" edited by Stanley Lippman.

T. Veldhuizen, "Expression Templates," *C++ Report*, Vol. 7 No. 5 June 1995, pp. 26-31

C++ Report (ISSN 1040-6042) is published nine times per year by SIGS Publications Inc., 71 West 23rd St., 3rd Floor, New York, NY 10010. © Copyright 1995 SIGS Publications Inc.

[Back to the Blitz++ Home Page](#)

[Back to Todd's Home Page](#)

Expression Templates

Todd Veldhuizen

Abstract: Expression Templates is a C++ technique for passing expressions as function arguments. The expression can be inlined into the function body, which results in faster and more convenient code than C-style callback functions. This technique can also be used to evaluate vector and matrix expressions in a single pass without temporaries. In preliminary benchmark results, one compiler evaluates vector expressions at 95-99.5% efficiency of hand-coded C using this technique (for long vectors). The speed is 2-15 times that of a conventional C++ vector class.

Introduction

Passing an expression to a function is a common occurrence. In C, expressions are usually passed using a pointer to a callback function containing the expression. For example, the standard C library routines `qsort()`, `bsearch()`, and `bsearch()` accept an argument `int (*cmp)(void*, void*)` which points to a user-defined function to compare two elements. Another common example is passing mathematical expressions to functions. The problem with callback functions is that repeated calls generate a lot of overhead, especially if the expression which the function evaluates is simple. Callback functions also slow pipelined machines, since the function calls force the processor to stall.

The technique of expression templates allows expressions to be passed to functions as an argument and inlined into the function body. Here are some examples:

```
// Integrate a function from 0 to 10
DoublePlaceholder x;
double result = integrate(x/(1.0+x), 0.0, 10.0);

// Count the elements between 0 and 100 in a
collection of int
List<int> myCollection;
IntPlaceholder y;
int n = myCollection.count(y >= 0 && y <= 100);

// Fill a vector with a sampled sine function
```

Expression Templates

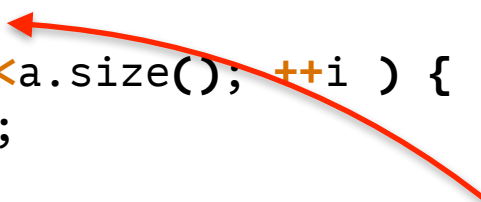
```
class DynamicVector  
{ /*...*/ };
```

```
int main()  
{  
    DynamicVector a, b, c;  
    // ... Initialization of vector a and b  
  
    c = a + b;  
  
    // ...  
}
```

Expression Templates

```
inline DynamicVector
operator+( DynamicVector const& a, DynamicVector const& b )
{
    if( a.size() != b.size() )
        throw std::invalid_argument( "Sizes don't match" );

    Vector tmp( a.size() );
    for( std::size_t i=0; i<a.size(); ++i ) {
        tmp[i] = a[i] + b[i];
    }
    return tmp;
}
```



Temporary vector: includes
a memory allocation, copy
operation, and a deallocation

Expression Templates

```
class DynamicVector  
{ /*...*/ };
```

```
int main()  
{  
    DynamicVector a, b, c;  
    // ... Initialization of vector a and b  
  
    for( size_t i=0U; i<size; ++i ) {  
        c[i] = a[i] + b[i];  
    }  
  
    // ...  
}
```

Expression Templates

The idea of expression templates:

- Defer the computation until it is required (lazy evaluation)
- Instead of returning a temporary vector, return a proxy object
- The proxy object represents the result of the addition
- The result is computed when accessing the proxy

Expression Templates

```
template< typename D1, typename D2 >
class VecVecAddExpr : public DenseVector<VecVecAddExpr<D1,D2>>
{
public:
    VecVecAddExpr( D1 const& lhs, D2 const& rhs )
        : lhs_( lhs )
        , rhs_( rhs )
    {}

    std::size_t size() const { return lhs_.size(); }

    decltype(auto) operator[]( std::size_t i ) const
    { return lhs_[i] + rhs_[i]; }

private:
    D1 const& lhs_; // Reference to the left-hand side operand
    D2 const& rhs_; // Reference to the right-hand side operand
};
```



CRTP base class

Expression Templates

```
template< typename D1, typename D2 >
VecVecAddExpr<D1,D2>
operator+( DenseVector<D1> const& a, DenseVector<D2> const& b )
{
    if( a.size() != b.size() )
        throw std::invalid_argument( "Sizes don't match" );

    return VecVecAddExpr<D1,D2>( a.derived(), b.derived() );
}
```



CRTP base class

Expression Templates

```
template< typename T >
class DynamicVector
    : public DenseVector< DynamicVector<T> >
{
public:
    // ...
    template< typename D >
    Vector& operator=( DenseVector<D> const& expr ) {
        resize( expr.size() );
        for( std::size_t i=0U; i<expr.size(); ++i )
            values_[i] = expr[i];
        return *this;
    }

private:
    // ...
    std::vector<T> values_;
};
```

C RTP base class

**This is where the magic happens:
the given expression is evaluated!**

Programming Task

Task (C_Cpp_Software_Design/Expression_Templates/ExpressionTemplates):

Step 1: Use Expression Templates to implement the addition operation. Benchmark the addition of two small (in-cache) and two large (out-of-cache) vectors.

Step 2: Use Expression Templates to implement the subtraction operation and a `sqrt()` operation for dense vectors.

Expression Templates

Let's consider the STL ...

The Ranges Algorithms

Non-modifying sequence operations

Defined in header `<algorithm>`

<code>all_of</code> (C++11) <code>any_of</code> (C++11) <code>none_of</code> (C++11)	checks if a predicate is <code>true</code> for all, any or none of the elements in a range (function template)
<code>ranges::all_of</code> (C++20) <code>ranges::any_of</code> (C++20) <code>ranges::none_of</code> (C++20)	checks if a predicate is <code>true</code> for all, any or none of the elements in a range (niebloid)
<code>for_each</code>	applies a function to a range of elements (function template)
<code>ranges::for_each</code> (C++20)	applies a function to a range of elements (niebloid)
<code>for_each_n</code> (C++17)	applies a function object to the first n elements of a sequence (function template)
<code>ranges::for_each_n</code> (C++20)	applies a function object to the first n elements of a sequence (niebloid)
<code>count</code> <code>count_if</code>	returns the number of elements satisfying specific criteria (function template)
<code>ranges::count</code> (C++20) <code>ranges::count_if</code> (C++20)	returns the number of elements satisfying specific criteria (niebloid)
<code>mismatch</code>	finds the first position where two ranges differ (function template)
<code>ranges::mismatch</code> (C++20)	finds the first position where two ranges differ (niebloid)
<code>find</code> <code>find_if</code> <code>find_if_not</code> (C++11)	finds the first element satisfying specific criteria (function template)
<code>ranges::find</code> (C++20) <code>ranges::find_if</code> (C++20) <code>ranges::find_if_not</code> (C++20)	finds the first element satisfying specific criteria (niebloid)

The Ranges Algorithms

```
void print( std::ranges::input_range auto const& range )
{
    for( auto const& value : range ) {
        std::cout << ' ' << value;
    }
    std::cout << '\n';
}

int main()
{
    // Note: std::views is a standard shortcut for std::ranges::views

    std::vector<int> const numbers{ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 };

    auto scaledEvenNumbers = numbers
        | std::views::filter([](int n){ return n%2==0; })
        | std::views::transform([](int n) { return n*2; })
        | std::views::take( 3UL );

    print( scaledEvenNumbers );
}
```

Programming Task

Task (C_Cpp_Software_Design/Expression_Templates/Ranges_std):
Modify the given example of the `ranges_v3` library to compute the first four squares of odd numbers.

STL Algorithms

Algorithms... That sounds like a lot of fun!

Programming Task

Task (C_Cpp_Software_Design/Expression_Templates/Ranges): Fix the implementation of the `TransformExpr` expression to apply the given operation on the given range.

Programming Task

Task (C_Cpp_Software_Design/Expression_Templates/Ranges_Benchmark): Copy-and-paste the following code into quick-bench.com. Benchmark the time to filter and transform a certain number of integers.

Expression Templates

Non-modifying sequence operations

Defined in header `<algorithm>`

<code>all_of</code> (C++11) <code>any_of</code> (C++11) <code>none_of</code> (C++11)	checks if a predicate is <code>true</code> for all, any or none of the elements in a range (function template)
<code>ranges::all_of</code> (C++20) <code>ranges::any_of</code> (C++20) <code>ranges::none_of</code> (C++20)	checks if a predicate is <code>true</code> for all, any or none of the elements in a range (niebloid)
<code>for_each</code>	applies a function to a range of elements (function template)
<code>ranges::for_each</code> (C++20)	applies a function to a range of elements (niebloid)
<code>for_each_n</code> (C++17)	applies a function object to the first n elements of a sequence (function template)
<code>ranges::for_each_n</code> (C++20)	applies a function object to the first n elements of a sequence (niebloid)
<code>count</code> <code>count_if</code>	returns the number of elements satisfying specific criteria (function template)
<code>ranges::count</code> (C++20) <code>ranges::count_if</code> (C++20)	returns the number of elements satisfying specific criteria (niebloid)
<code>mismatch</code>	finds the first position where two ranges differ (function template)
<code>ranges::mismatch</code> (C++20)	finds the first position where two ranges differ (niebloid)
<code>find</code> <code>find_if</code> <code>find_if_not</code> (C++11)	finds the first element satisfying specific criteria (function template)
<code>ranges::find</code> (C++20) <code>ranges::find_if</code> (C++20) <code>ranges::find_if_not</code> (C++20)	finds the first element satisfying specific criteria (niebloid)

Programming Task

Task (C_Cpp_Software_Design/Expression_Templates/Niebloid):
Consider the use of `swap()` in the following example. Implement an ADL-resistant version of the `swap()` algorithm in the standard namespace.

Guidelines

Guideline: Use Expression Templates to introduce lazy evaluation of operations to allow optimizations.

Guideline: Consider Expression Templates as a static polymorphism example for the Decorator design pattern.

Things to Remember

- Design for change/extensions
- Minimize couplings wherever possible
- Consider modern programming techniques to break inheritance relationships
- Prefer containment (composition/aggregation) to inheritance
- Prefer value semantics based solutions
- Keep it simple (KISS)

Literature



References (Value Semantics)

- Jon Kalb, “Back to Basics: Object-Oriented Programming”. CppCon 2019 (<https://www.youtube.com/watch?v=32tDTD9UJCE>)
- John Lakos, “Value Semantics: It ain’t about the syntax! (Part I)”. CppCon 2015 (<https://www.youtube.com/watch?v=W3xl1HJUy7Q>)
- Klaus Iglberger, “Back to Basics: Value Semantics”. CppCon 2022 (<https://www.youtube.com/watch?v=G9MxNwUoSt0>)
- Dave Abrahams, “Values: Regularity, Independence, Projection, and the Future of Programming”. CppCon 2022 (<https://www.youtube.com/watch?v=QthAU-t3PQ4>)

References (General Design Patterns)

- Klaus Iglberger, “Design Patterns - Facts and Misconceptions”. CppCon 2021 (<https://www.youtube.com/watch?v=OvO2NR7pXjg>)
- Klaus Iglberger, “Design Patterns - The Most Common Misconceptions (1 of N)”. Meeting C++ 2023 (<https://www.youtube.com/watch?v=w-fkFyrbWbs>)
- Klaus Iglberger, “Design Patterns - The Most Common Misconceptions (2 of N)”. Meeting C++ 2024 (<https://www.youtube.com/watch?v=pmdwAf6hCWg>)
- Scott Wlaschin, “Function Programming Design Patterns”. NDC London 2014 (<https://www.youtube.com/watch?v=E8l19uA-wGY>)

References (Type Erasure)

- Sean Parent, “Inheritance is the Base Class of Evil”. GoingNative 2013 (<http://channel9.msdn.com/Events/GoingNative/2013/Inheritance-Is-The-Base-Class-of-Evil>)
- Arthur O’Dwyer, “Back to Basics: Type Erasure”. CppCon 2019 (<https://www.youtube.com/watch?v=tbUCHifyT24>)
- Klaus Iglberger, “Breaking Dependencies: Type Erasure - A Design Analysis”. Meeting C++ 2021 (<https://www.youtube.com/watch?v=jKt6A3wnDyl>)
- Klaus Iglberger, “Breaking Dependencies: Type Erasure - The Implementation Details”. CppCon 2022 (<https://www.youtube.com/watch?v=qn6OqefuH08>)
- Zach Laine, “Pragmatic Type Erasure: Solving OOP Problems with an Elegant Design Pattern”. CppCon 2014 (<https://www.youtube.com/watch?v=0l0FD3N5cgM>)
- Sy Brand, “Dynamic Polymorphism with Metaclasses and Code Injection”, CppCon 2020 (https://www.youtube.com/watch?v=8c6BAQcYF_E)
- Eduardo Madrid, “Not Leaving Performance On The Jump Table”, CppCon 2020 (https://www.youtube.com/watch?v=e8SyxB3_mnw)

References (CRTP)

- Klaus Iglberger, “Design Patterns - The Most Common Misconceptions (2 of N)”. CppCon 2024 (<https://www.youtube.com/watch?v=pmdwAf6hCWg>)

References (Expression Templates)

- Joel Falcou, “Expression Templates - Past, Present, Future (Part 1)”. CppCon 2015 (<https://www.youtube.com/watch?v=liVl5oSU5B8>)
- Eric Niebler, “Ranges for the Standard Library”. CppCon 2015 (<https://www.youtube.com/watch?v=mFUXNMfaciE>)
- Eric Niebler: Customization Point Design in C++11 and Beyond (<http://ericniebler.com/2014/10/21/customization-point-design-in-c11-and-beyond/>)

References (Visitor)

- Klaus Iglberger, “Breaking Dependencies - The Visitor Design Pattern”. CppCon 2022 (<https://www.youtube.com/watch?v=PEcy1vYHb8A>)
- Mateusz Pusz, “Effective replacement of dynamic polymorphism with std::variant”. CppCon 2018 (<https://www.youtube.com/watch?v=gKbORJtnVu8>)

email: klaus.iglberger@cpp-training.com

LinkedIn: [linkedin.com/in/klaus-iglberger](https://www.linkedin.com/in/klaus-iglberger)

Xing: [xing.com/profile/Klaus_Iglberger/cv](https://www.xing.com/profile/Klaus_Iglberger/cv)